FDPS Tutorial

Ataru Tanikawa, Daisuke Namekata, Natsuki Hosono, Masaki Iwasawa, Keigo Nitadori, Takayuki Munanushi, and Junichiro Makino

Paricle Simulator Research Team, RIKEN Center for Computational Science, RIKEN

0 Contents

1 Change log **6**

2 Overview **7**

3 Getting Started **8**
3.1 Environment . . . 8
3.2 Necessary software . . . 8
3.2.1 Standard functions . . . 8
3.2.1.1 Single thread . . . 8
3.2.1.2 Parallel processing . . . 8
3.2.1.2.1 OpenMP . . . 8
3.2.1.2.2 MPI . . . 8
3.2.1.2.3 MPI+OpenMP . . . 8
3.2.2 Extensions . . . 9
3.2.2.1 Particle Mesh . . . 9
3.3 Install . . . 9
3.3.1 How to get the software . . . 9
3.3.1.1 The latest version . . . 9
3.3.1.2 Previous versions . . . 10
3.3.2 How to install . . . 10
3.4 How to compile and run the sample codes . . . 10
3.4.1 Gravitational N-body simulation . . . 10
3.4.1.1 Summary . . . 10
3.4.1.2 Move to the directory with the sample code . . . 10
3.4.1.3 Edit Makefile . . . 10
3.4.1.4 Run make . . . 11
3.4.1.5 Run the sample code . . . 11
3.4.1.6 Analysis of the result . . . 12
3.4.1.7 To use Phantom-GRAPE for x86 . . . 12

3.4.1.8 To use PIKG . . . 14
3.4.1.9 To use NVIDIA GPUs . . . 14
3.4.2 SPH simulation code . . . 14
3.4.2.1 Summary . . . 14
3.4.2.2 Move to the directory with the sample code . . . 14
3.4.2.3 Edit Makefile . . . 14
3.4.2.4 Run make . . . 14
3.4.2.5 Run the sample code . . . 15
3.4.2.6 Analysis of the result . . . 15

4 How to Use **16**
4.1 *N*-body simulation code . . . 16
4.1.1 Location of source files and file structure . . . 16
4.1.2 User-defined types and user-defined functions . . . 16
4.1.2.1 FullParticle type . . . 16
4.1.2.2 calcForceEpEp . . . 18
4.1.3 The main body of the user program . . . 19
4.1.3.1 Including the header file of FDPS . . . 19
4.1.3.2 Initialization and Termination of FDPS . . . 19
4.1.3.3 Creation and initialization of FDPS objects . . . 19
4.1.3.3.1 Creation of FDPS objects . . . 19
4.1.3.3.2 Initialization of DomainInfo object . . . 19
4.1.3.3.3 Initialization of ParticleSystem object . . . 20
4.1.3.3.4 Initialization of Tree object . . . 20
4.1.3.4 Time integration loop . . . 20
4.1.3.4.1 Domain Decomposition . . . 20
4.1.3.4.2 Particle Exchange . . . 21
4.1.3.4.3 Interaction Calculation . . . 21
4.1.3.4.4 Time integration . . . 21
4.1.4 Log file . . . 22
4.2 SPH simulation code with fixed smoothing length . . . 22
4.2.1 Location of source files and file structure . . . 22
4.2.2 User-defined types and user-defined functions . . . 22
4.2.2.1 FullParticle type . . . 22
4.2.2.2 EssentialParticleI(J) type . . . 24
4.2.2.3 Force type . . . 24
4.2.2.4 calcForceEpEp . . . 25
4.2.3 The main body of the user program . . . 26
4.2.3.1 Including the header file of FDPS . . . 26
4.2.3.2 Initialization and termination of FDPS . . . 27
4.2.3.3 Creation and initialization of FDPS objects . . . 27
4.2.3.3.1 Creation of necessary FDPS objects . . . 27
4.2.3.3.2 Initialization of the domain information object . . . 27
4.2.3.3.3 Initialization of ParticleSystem object . . . 28
4.2.3.3.4 Initialization of Tree objects . . . 28
4.2.3.4 Time integration loop . . . 28

4.2.3.4.1 Domain Decomposition . . . 28
4.2.3.4.2 Particle Exchange . . . 28
4.2.3.4.3 Interaction Calculation . . . 29
4.2.4 Compilation of the program . . . 29
4.2.5 Execution . . . 29
4.2.6 Log and output files . . . 29
4.2.7 Visualization . . . 29

5 Sample Codes **30**
5.1 N-body simulation . . . 30
5.2 SPH simulation with fixed smoothing length . . . 40

6 Extensions **51**
6.1 P^3M code . . . 51
6.1.1 Location of sample code and working directory . . . 51
6.1.2 User-defined types . . . 51
6.1.2.1 FullParticle type . . . 51
6.1.2.2 EssentialParticleI type . . . 52
6.1.2.3 Force type . . . 53
6.1.2.4 calcForceEpEp . . . 53
6.1.2.5 calcForceEpSp . . . 55
6.1.3 Main body of the sample code . . . 56
6.1.4 Include header files . . . 56
6.1.4.1 Initialization and Termination of FDPS . . . 57
6.1.4.2 Creation and initialization of FDPS objects . . . 57
6.1.4.2.1 Creation of necessary FDPS objects . . . 57
6.1.4.2.2 Initialization of FDPS objects . . . 57
6.1.4.3 Generation of a distribution of particles . . . 59
6.1.4.3.1 Domain Decomposition . . . 59
6.1.4.3.2 Particle Exchange . . . 60
6.1.4.4 Interaction Calculation . . . 60
6.1.4.5 Calculation of relative energy error . . . 62
6.1.5 Compile . . . 62
6.1.6 Run . . . 62
6.1.7 Check the result . . . 62
6.2 TreePM code . . . 64
6.2.1 Location of the sample code and the working directory . . . 64
6.2.2 Required header files . . . 64
6.2.3 User-defined classes . . . 65
6.2.3.1 FullParticle type . . . 65
6.2.3.2 EssentialParticleI type . . . 69
6.2.3.3 EssentialParticleJ type . . . 70
6.2.3.4 Force type . . . 71
6.2.3.5 calcForceEpEp type . . . 71
6.2.4 Main body of the program . . . 72
6.2.4.1 Initialization and Termination of FDPS . . . 73

6.2.4.2 Creation and Initialization of FDPS objects . . . 73
6.2.4.2.1 Creation of necessary FDPS objects . . . 73
6.2.4.2.2 Initialization of FDPS objects . . . 73
6.2.4.3 Initial Condition . . . 74
6.2.4.3.1 Domain Decomposition . . . 74
6.2.4.3.2 Particle Exchange . . . 75
6.2.4.4 Interaction Calculation . . . 75
6.2.4.5 Time Integration . . . 75
6.2.5 Compile . . . 75
6.2.6 Execution . . . 76
6.2.7 Confirmation of the result . . . 76

7 Practical Applications **77**
7.1 *N*-body/SPH code . . . 77
7.1.1 How to run the sample code . . . 77
7.1.1.1 Move to the directory the sample code is placed . . . 77
7.1.1.2 File structure of the sample code . . . 78
7.1.1.3 Edit Makefile . . . 78
7.1.1.4 Create particle data using MAGI . . . 81
7.1.1.5 Run make . . . 82
7.1.1.6 Run the sample code . . . 82
7.1.1.7 Analysis of the result . . . 82
7.1.2 Springel's SPH scheme . . . 83
7.1.3 User-defined types . . . 84
7.1.3.1 FullParticle type . . . 85
7.1.3.2 EssentialParticle type . . . 88
7.1.3.3 Force type . . . 90
7.1.4 Interaction functions . . . 91
7.1.4.1 Interaction function for the gravity calculation . . . 91
7.1.4.2 Interaction function for the density calculation . . . 93
7.1.4.3 Interaction function for the calculation of pressure-gradient acceleration . . . 96
7.1.5 Main body of the sample code . . . 97
7.1.5.1 Include the header file of FDPS . . . 98
7.1.5.2 Initialization and and termination of FDPS . . . 98
7.1.5.3 Creation and initialization of FDPS objects . . . 98
7.1.5.3.1 Creation and initialization of ParticleSystem objects 98
7.1.5.3.2 Creation and initialization of DomainInfo object . . 99
7.1.5.3.3 Creation and initialization of TreeForForce objects . 99
7.1.5.4 Setting initial condition . . . 99
7.1.5.5 Domain decomposition . . . 100
7.1.5.6 Particle exchange . . . 101
7.1.5.7 Interaction calculations . . . 101
7.1.5.8 Time integration . . . 103

8 User Supports **104**
8.1 Compile-time problem 104
8.2 Run-time problem 104
8.3 Other cases 104

9 License **105**

1 Change log

- 2015/03/17 English version created
- 2015/06/04 Spell-checked complete version
- 2016/01/18 Description of GPU version added (Sec. 3.4.1.9)
- 2018/07/11
  - Typographical error correction in Section 3:
    - Update the information of compilers tested (using FDPS ver. 4.1a)
    - The extension of output file of the SPH sample code is wrong
  - Typographical error correction in Section 4:
    - The section number of DomainInfo class in the specification document is wrong (Sec. 4.1)
    - The arguments of calcForceAllAndWriteBack() in the document is not consistent with the source code (Sec. 4.1)
  - Typographical error in Section 6:
    - Description of the PP part is not consistent with the source code (Sec. 6.1)
    - Description of the file structure is not consistent with the sample code (Sec. 6.2)
    - The arguments of decomposeDomainAll() in the document is not consistent with the source code (Sec. 6.2)
- 2018/08/22 Description of N-body/SPH sample code added (Sec. 7)
- 2019/07/19 Description of N-body/SPH sample code updated (Sec. 7)
- 2020/08/28
  - Web links for the initial condition files used in TreePM & N-body/SPH sample codes are changed (Sec. 6.2, 7.1)
- 2020/09/02
  - Description for `initialize` of TreeForForce class that leads to misunderstanding is fixed. The sample codes are also fixed.
- 2023/07/18
  - Updated the software requirements. Ruby added, gcc version changed to 8.x.

2 Overview

In this section, we present the overview of Framework for Developing Particle Simulator (FDPS) . FDPS is an application-development framework which helps the application programmers and researchers to develop simulation codes for particle systems. What FDPS does are calculation of the particle-particle interactions and all of the necessary works to parallelize that part on distributed-memory parallel computers wit near-ideal load balancing, using hybrid parallel programming model (uses both MPI and OpenMP). Low-cost part of the simulation program, such as the integration of the orbits of particles using the calculated interaction, is taken care by the user-written part of the code.

FDPS support two- and three-dimensional Cartesian coordinates. Supported boundary conditions are open and periodic. For each coordinate, the user can select open or periodic boundary.

The user should specify the functional form of the particle-particle interaction. FDPS divides the interactions into two categories: long-range and short-range. The difference between two categories is that if the grouping of distant particles is used to speedup calculation (long-range) or not (short range).

The long-range force is further divided into two subcategories: with and without a cutoff scale. The long range force without cutoff is what is used for gravitational N-body simulations with open boundary. For periodic boundary, one would usually use TreePM, P^3M, PME or other variant, for which the long-range force with cutoff can be used.

The short-range force is divided to four subcategories. By definition, the short-range force has some cutoff length. If the cutoff length is a constant which does not depend on the identity of particles, the force belongs to "constant" class. If the cutoff depends on the source or receiver of the force, it is of "scatter" or "gather" classes. Finally, if the cutoff depends on both the source and receiver in the symmetric way, its class is "symmetric". Example of a "constant" interaction is the Lennard-Jones potential. Other interactions appear, for example, SPH calculation with adaptive kernel size.

The user writes the code for particle-particle interaction kernel and orbital integration using C++ language .

3 Getting Started

In this section, we describe the first steps you need to do to start using FDPS . We explain the environment (the supported operating systems), the necessary software (compilers etc), and how to compile and run the sample codes.

3.1 Environment

FDPS works on Linux, Mac OS X, Windows (with Cygwin).

3.2 Necessary software

In this section, we describe software necessary to use FDPS, first for standard functions, and then for extensions.

3.2.1 Standard functions

we describe software necessary to use standard functions of FDPS. First for the case of single-thread execution, then for multithread, then for multi-nodes.

3.2.1.1 Single thread

- make
- A C++ compiler (We have tested with gcc version 8.x and later)
- Ruby (necessary for PIKG. We have tested with version 3.0.2)

3.2.1.2 Parallel processing

3.2.1.2.1 OpenMP

- make
- A C++ compiler with OpenMP support (We have tested with gcc version 8)

3.2.1.2.2 MPI

- make
- A C++ compiler which supports MPI version 1.3 or later. (We have tested with Open MPI 1.6.4 and K compiler version 1.2.0)

3.2.1.2.3 MPI+OpenMP

- make
- A C++ compiler which supports OpenMP and MPI version 1.3 or later. (We have tested with Open MPI 1.6.4 and K compiler version 1.2.0)

3.2.2 Extensions

Current extension for FDPS is the "Particle Mesh" module. We describe the necessary software for it below.

3.2.2.1 Particle Mesh

- make
- A C++ compiler which supports OpenMP and MPI version 1.3 or later. (We have tested with Open MPI 1.6.4)
- FFTW 3.3 or later

3.3 Install

In this section we describe how to get the FDPS software and how to build it.

3.3.1 How to get the software

We first describe how to get the latest version, and then previous versions. We recommend to use the latest version.

3.3.1.1 The latest version

You can use one of the following ways.

- Using browsers
  1. Click "Download ZIP" in https://github.com/FDPS/FDPS to download `FDPS-master.zip`
  2. Move the zip file to the directory under which you want to install FDPS and unzip the file (or place the files using some GUI).
- Using CLI (Command line interface)
  - Using Subversion:

```
$ svn co --depth empty https://github.com/FDPS/FDPS
$ cd FDPS
$ svn up trunk
```

  - Using Git

```
$ git clone git://github.com/FDPS/FDPS.git
```

3.3.1.2 Previous versions

You can get previous versions using browsers.

- Previous versions are listed in https://github.com/FDPS/FDPS/releases. Click the version you want to download it.
- Extract the files under the directory you want.

3.3.2 How to install

There is no need for configure or setup.

3.4 How to compile and run the sample codes

We provide two samples: one for gravitational N-body simulation and the other for SPH. We first describe gravitational N-body simulation and then SPH. Sample codes do not use extensions.

3.4.1 Gravitational N-body simulation

3.4.1.1 Summary

Through the following steps one can use this sample.

- Move to the directory `$(FDPS)/sample/c++/nbody`. Here, `$(FDPS)` denotes the highest-level directory for FDPS (Note that `FDPS` is not an environmental variable). The actual value of `$(FDPS)` depends on the way you acquire the software. If you used the browser, the last part is "FDPS-master". If you used Subversion or Git, it is "trunk" or "FDPS", respectively.
- Edit `Makefile` in the current directory (`$(FDPS)/sample/c++/nbody`).
- Run the `make` command to create the executable `nbody.out`.
- Run `nbody.out`
- Check the output.

In addition, we describe the way to use Phantom-GRAPE for x86.

3.4.1.2 Move to the directory with the sample code

Move to `$(FDPS)/sample/c++/nbody`.

3.4.1.3 Edit Makefile

Edit Makefile following the description below. The changes depend on if you use OpenMP and/or MPI.

- Without both OpenMP and MPI

  - Set the variable `CC` the command to run your C++ compiler

- With OpenMP but not with MPI
  - Set the variable `CC` the command to run your C++ compiler
  - Uncomment the line `CFLAGS += -DPARTICLE_SIMULATOR_THREAD_PARALLEL -fopenmp`. If you use Intel compiler, replace `-fopenmp` by `-qopenmp` or `-openmp` depending on the version of the compiler.

- With MPI but not with OpenMP
  - Set the variable `CC` the command to run your MPI C++ compiler
  - Uncomment the line `CFLAGS += -DPARTICLE_SIMULATOR_MPI_PARALLEL`

- With both OpenMP and MPI
  - Set the variable `CC` the command to run your MPI C++ compiler
  - Uncomment the line `CFLAGS += -DPARTICLE_SIMULATOR_THREAD_PARALLEL -fopenmp`. If you use Intel compiler, replace `-fopenmp` by `-qopenmp` or `-openmp` depending on the version of the compiler.
  - Uncomment the line `CFLAGS += -DPARTICLE_SIMULATOR_MPI_PARALLEL`

3.4.1.4 Run make

Type "make" to run `make`.

3.4.1.5 Run the sample code

- If you are not using MPI, run the following in CLI (terminal)

```
$ ./nbody.out
```

- If you are using MPI, run the following in CLI (terminal)

```
$ MPIRUN -np NPROC ./nbody.out
```

  Here, `MPIRUN` should be `mpirun` or `mpiexec` depending on your MPI configuration, and `NPROC` is the number of processes you will use.

  Upon normal completion, the following output log should appear in stderr. The exact value of the energy error may depend on the system, but it is okay if its absolute value is of the order of 1×10^{-3}.

```
time:  9.6250000 energy error: -4.512836e-03
time:  9.7500000 energy error: -4.440746e-03
time:  9.8750000 energy error: -4.652358e-03
time: 10.0000000 energy error: -4.605855e-03
MemoryPool::finalize() is completed!
******** FDPS has successfully finished. ********
```

3.4.1.6 Analysis of the result

In the directory `result`, files "000x.dat" have been created. These files store the distribution of particles. Here, x is an integer (from 0 to 9) and it indicates time. The output file format is that in each line, index of particle, mass, position (x, y, z) and velocity (vx, vy, vz) are listed.

What is simulated with the default sample is the cold collapse of an uniform sphere with radius three expressed using 1024 particles. Using gnuplot, you can see the particle distribution in the xy plane at time=9:

```
$ gnuplot
$ plot "result/0009.dat" using 3:4
```

By plotting the particle distributions at other times, you can see how the initially uniform sphere contracts and then expands again. (Figure 1).

To increase the number of particles to 10000, try: (without MPI)

```
$ ./nbody.out -N 10000
```

3.4.1.7 To use Phantom-GRAPE for x86

If you are using a computer with Intel or AMD x86 CPU, you can use Phantom-GRAPE for x86.

Move to the directory `$(FDPS)/src/phantom_grape_x86/G5/newton/libpg5`, edit the Makefile there (if necessary), and run make to build the Phantom-GRAPE library `libpg5.a`.

Then go back to directory `$(FDPS)/sample/c++/nbody`, edit Makefile and remove "#" at the top of the line
"#use_phantom_grape_x86 = yes", and (after removing the existing executable) run make again. (Same for with and without OpenMP or MPI). You can run the executable in the same way as that for the executable without Phantom GRAPE.

The performance test on a machine with Intel Core i5-3210M CPU @2.50GHz (2 cores, 4 threads) indicates that, for N=8192, the code with Phantom GRAPE is faster than that without Phantom GRAPE by a factor a bit less than five.

The following is the sample command line:

```
$ ./nbody.out -N 8192 -n 256
```

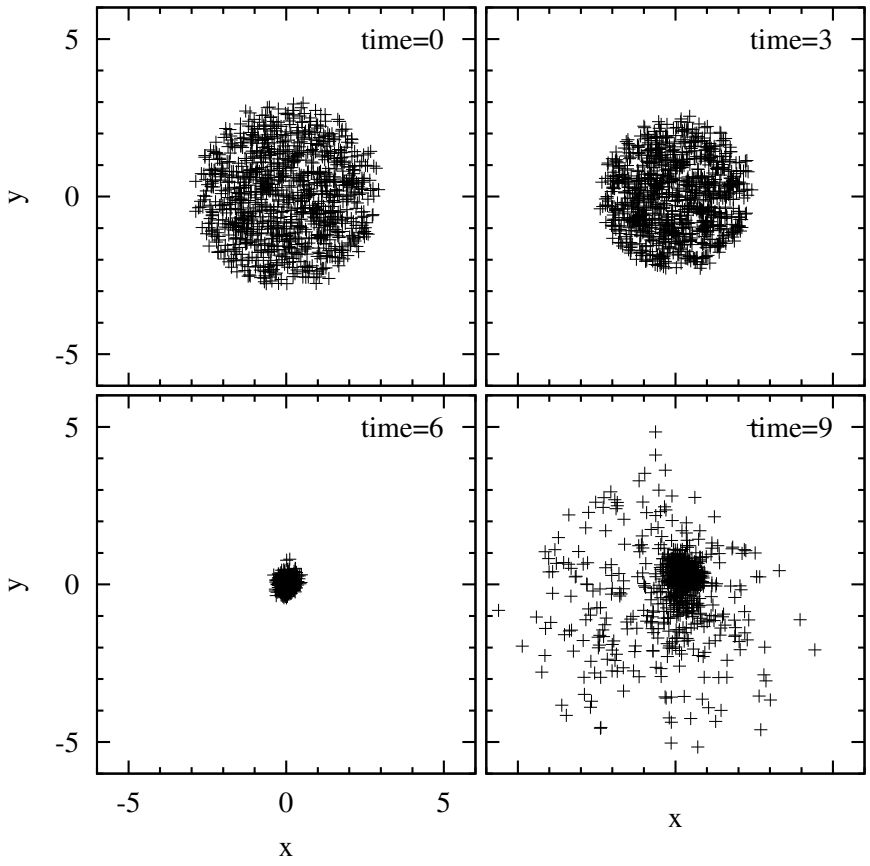


Figure 1:

3.4.1.8 To use PIKG

PIKG (https://github.com/FDPS/PIKG) is a tool to generate a highly-optimized, two-body inter-particle interaction calculation kernel for particle simulations from a simple description of the interaction using a DSL (Domain Specific Language).

In order to use kernels generated by PIKG, open Makefile in directory `$(FDPS)/sample/c++/nbody` and remove `#` at the top of the line `#use_pikg_x86 = yes`. Then, (after removing the existing executable) run `make pikg`. (Same for with and without OpenMP or MPI). You can run the executable in the same way as that for the executable without PIKG.

In the default, PIKG generates kernels in `reference` mode. In this mode, unoptimized kernels are generated. To generate kernels optimized for specific architectures such as AVX2 and AVX-512, change `CONVERSION_TYPE` in Makefile and remove `#` at the top of the line containing `*FLAGS` (where `*` is the usual regular expression symbol).

3.4.1.9 To use NVIDIA GPUs

The sample program includes the interaction kernel written in Cuda for NVIDIA GPUs.

Uncomment the line "#use_cuda_gpu = yes" in file `Makefile` in `$(FDPS)/sample/c++/nbody`and assign to CUDA_HOME in Makefile a value appropriate to your environment. You can then run make to obtain the executable (OpenMP and MPI are also supported). The executable can be tested in the same way as the non-GPU version.

3.4.2 SPH simulation code

3.4.2.1 Summary

Through the following steps one can use this sample.

- Move to the directory `$(FDPS)/sample/c++/sph`.
- Edit Makefile in the current directory (`$(FDPS)/sample/c++/sph`).
- Run make command to create the executable `sph.out`.
- Run `sph.out`.
- Check the output.

3.4.2.2 Move to the directory with the sample code

Move to `$(FDPS)/sample/c++/sph`.

3.4.2.3 Edit Makefile

Edit Makefile following the same description described in § 3.4.1.3.

3.4.2.4 Run make

Type "make" to run `make`.

3.4.2.5 Run the sample code

- If you are not using MPI, run the following in CLI (terminal)

```
$ ./sph.out
```

- If you are using MPI, run the following in CLI (terminal)

```
$ MPIRUN -np NPROC ./sph.out
```

Here, `MPIRUN` should be `mpirun` or `mpiexec` depending on your MPI configuration, and `NPROC` is the number of processes you will use.

Upon normal completion, the following output log should appear in stderr.

```
******** FDPS has successfully finished. ********
```

3.4.2.6 Analysis of the result

In the directory `result`, files "000x.txt" have been created. These files store the distribution of particles. Here, x is an integer (from 0 to 9) and it indicates time. The output file format is that in each line, index of particle, mass, position (x, y, z), velocity (vx, vy, vz), density, internal energy and pressure are listed.

What is simulated is the three-dimensional shock-tube problem. Using gnuplot, you can see the plot of the x-coordinate and density of particles at time=40:

```
$ gnuplot
$ plot "result/0040.txt" using 3:9
```

When the sample worked correctly, a figure similar to Figure 2 should appear.

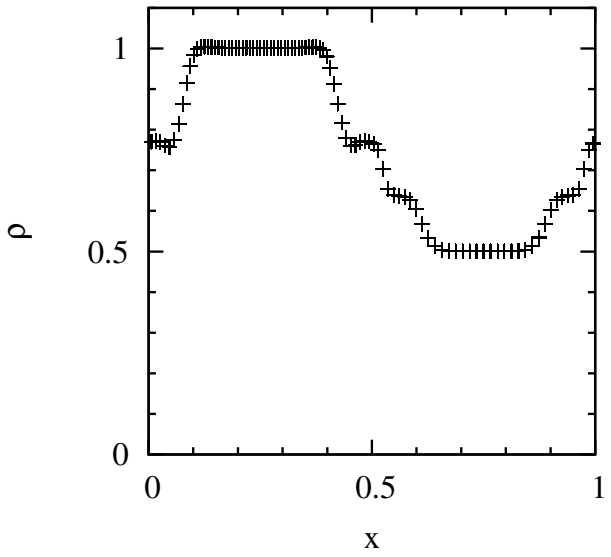


Figure 2:

4 How to Use

In this section, we describe the sample codes used in previous section (§ 3) in more detail. Especially, the explanation will focus mainly on derived data types that users must define (hereafter, **user-defined types**) and how to use APIs of FDPS. In order to avoid duplication of explanation, some matters are explained in § 4.1 only, where we explain the N-body sample code. Therefore, we recommend users who are interested in SPH simulation only to read § 4.1.

4.1 N-body simulation code

4.1.1 Location of source files and file structure

The source files of the sample code are in the directory `$(FDPS)/sample/c++/nbody`. The sample code consists of `user-defined.hpp` where user-defined types and interaction function are described, and `nbody.cpp` where the other parts of N-body simulation code are implemented. In addition to these, there is a Makefile for GCC, `Makefile`.

4.1.2 User-defined types and user-defined functions

In this section, we describe the details of classes and `void` functions that users must define when performing an N-body simulation with FDPS.

4.1.2.1 FullParticle type

You must define a FullParticle type. FullParticle type should contain all physical quantities necessary for an N-body simulation. Listing 1 shows the implementation of FullParticle type

in our sample code (see `user-defined.hpp`).

Listing 1: FullParticle type

```
1  class FPGrav{
2  public:
3      PS::S64    id;
4      PS::F64    mass;
5      PS::F64vec pos;
6      PS::F64vec vel;
7      PS::F64vec acc;
8      PS::F64    pot;
9
10     static PS::F64 eps;
11
12     PS::F64vec getPos() const {
13         return pos;
14     }
15
16     PS::F64 getCharge() const {
17         return mass;
18     }
19
20     void copyFromFP(const FPGrav & fp){
21         mass = fp.mass;
22         pos  = fp.pos;
23     }
24
25     void copyFromForce(const FPGrav & force) {
26         acc = force.acc;
27         pot = force.pot;
28     }
29
30     void clear() {
31         acc = 0.0;
32         pot = 0.0;
33     }
34
35     void writeAscii(FILE* fp) const {
36         fprintf(fp, "%ld\t%g\t%g\t%g\t%g\t%g\t%g\t%g\n",
37                 this->id, this->mass,
38                 this->pos.x, this->pos.y, this->pos.z,
39                 this->vel.x, this->vel.y, this->vel.z);
40     }
41
42     void readAscii(FILE* fp) {
43         fscanf(fp, "%ld\t%lf\t%lf\t%lf\t%lf\t%lf\t%lf\t%lf\n",
44                &this->id, &this->mass,
45                &this->pos.x, &this->pos.y, &this->pos.z,
46                &this->vel.x, &this->vel.y, &this->vel.z);
47         }
48
49 };
```

Note that FullParticle type is used as EssentialParticleI type, EssentialParticleJ type, and Force type in this sample code. FullParticle type must have member functions `copyfromFP()`

and `copyFromForce()` to copy data. It should have member functions `getCharge()` (returns the particle mass), `getPos()` (returns the particle position), and `setPos()` (sets the particle position). In this code, we also define member functions `writeAscii()` and `readAscii()`, which are necessary to use file I/O functions of FDPS. The member function `clear()` is also necessary, which zero-clear the acceleration and potential.

4.1.2.2 calcForceEpEp

You must define an interaction function calcForceEpEp as `void` function in C++. It should contain actual code for the calculation of interaction between particles. Listing 2 shows the implementation of calcForceEpEp (see `user-defined.hpp`).

Listing 2: Function calcForceEpEp

```
#else

template <class TParticleJ>
void CalcGravity(const FPGrav * ep_i,
                 const PS::S32 n_ip,
                 const TParticleJ * ep_j,
                 const PS::S32 n_jp,
                 FPGrav * force) {
    PS::F64 eps2 = FPGrav::eps * FPGrav::eps;
    for(PS::S32 i = 0; i < n_ip; i++){
        PS::F64vec xi = ep_i[i].getPos();
        PS::F64vec ai = 0.0;
        PS::F64 poti = 0.0;
        for(PS::S32 j = 0; j < n_jp; j++){
            PS::F64vec rij    = xi - ep_j[j].getPos();
            PS::F64    r3_inv = rij * rij + eps2;
            PS::F64    r_inv  = 1.0/sqrt(r3_inv);
            r3_inv  = r_inv * r_inv;
            r_inv  *= ep_j[j].getCharge();
            r3_inv *= r_inv;
            ai     -= r3_inv * rij;
            poti   -= r_inv;
        }
        force[i].acc += ai;
        force[i].pot += poti;
```

Here, we show the implementation for the case that the code is executed on CPUs without the Phantom-GRAPE library.

In this sample, we implemented calcForceEpEp using function template[1)]. Its dummy arguments are an array of EssentialParticleI type, the number of EssentialParticleI type variables, an array of EssentialParticleJ type, the number of EssentialParticleJ type variables, an array of Force type.

[1)] Function template is, as the name suggests, template of function, in which we can use variables of general data types listed in the angle brackets of `templete <...>` to define function. This characteristic is very different from normal functions in which we must use variables of specific data types such as `int`, `double`, `struct foo`, etc. All of general data types used in a function template is replaced by specific data types at the compile-time because function template must be called with the template argument in which we must describe a list of specific data types.

4.1.3 The main body of the user program

In this section, we describe the functions a user should write to implement gravitational N-body calculation using FDPS. The main function is described in the file `nbody.cpp`.

4.1.3.1 Including the header file of FDPS

To make the standard features of FDPS available, we must include header file `particle_simulator.hpp`.

Listing 3: Including header file `particle_simulator.hpp`

```
1 #include <particle_simulator.hpp>
```

4.1.3.2 Initialization and Termination of FDPS

First, users must initialize FDPS by the following code.

Listing 4: Initialization of FDPS

```
1 PS::Initialize(argc, argv);
```

Once started, FDPS should be terminated explicitly. In the sample code, FDPS should be terminated just before the termination of the program. To achieve this, user should write the following code at the end of the main function.

Listing 5: Termination of FDPS

```
1 PS::Finalize();
```

4.1.3.3 Creation and initialization of FDPS objects

Once succeed the initialization, the user needs to create objects used to talk to FDPS. In this section, we describe how to create and initialize these objects.

4.1.3.3.1 Creation of FDPS objects

In an N-body simulation, one needs to create objects of ParticleSystem type, DomainInfo type, and TreeForForceLong type (hereinafter, we call it Tree type for simplicity). The following is the code to create them (see the main function in `nbody.cpp`).

Listing 6: Creation of FDPS objects

```
1 PS::DomainInfo dinfo;
2 PS::ParticleSystem<FPGrav> system_grav;
3 PS::TreeForForceLong<FPGrav, FPGrav, FPGrav>::Monopole tree_grav;
```

4.1.3.3.2 Initialization of DomainInfo object

Once create the objects, user must initialize these objects. In this sample code, since the boundary condition is not periodic, users have only to call the API initialize to initialize the objects.

Listing 7: Initialization of DomainInfo object

```
1 const PS::F32 coef_ema = 0.3;
2 dinfo.initialize(coef_ema);
```

Note that the first argument of API `initialize` is a smoothing factor of an exponential moving average operation that is performed in the domain decomposition procedure. The definition of this factor is described in the specification of FDPS (see § 9.1.2 in `doc_spec_cpp_en.pdf`).

4.1.3.3.3 Initialization of ParticleSystem object

Next, you must initialize a ParticleSystem object. This is done by calling the API `initialize` .

Listing 8: Initialization of ParticleSystem object

```
1 system_grav.initialize();
```

4.1.3.3.4 Initialization of Tree object

Next, we must initialize a Tree object. The initialization of a Tree object is done by calling the API `initialize`. This API should be given an integer for determining the initial sizes of internally-used arrays. A number comparable to the number of local particles is enough. In this sample, we set the number of local particles (`n_loc`):

Listing 9: Initialization of Tree object

```
1 tree_grav.initialize(n_loc, theta, n_leaf_limit, n_group_limit);
```

The `initialize` method has three optional arguments. Here, we pass these arguments explicitly.

- `theta` — the so-called opening angle criterion for the tree method.
- `n_leaf_limit` — the upper limit for the number of particles in the leaf nodes.
- `n_group_limit` — the upper limit for the number of particles with which the particles use the same interaction list for the force calculation.

4.1.3.4 Time integration loop

In this section we describe the structure of the time integration loop.

4.1.3.4.1 Domain Decomposition

First, the computational domain is decomposed, using the current distribution of particles. In the sample, this is done by API decomposeDomainAll of the DomainInfo object:

Listing 10: Domain Decomposition

```
1 if (n_loop % 4 == 0){
2     dinfo.decomposeDomainAll(system_grav);
3 }
```

In this sample code, we perform domain decomposition once in 4 main loops in order to reduce the computational cost.

4.1.3.4.2 Particle Exchange

Then, particles are exchanged between processes so that they belong to the process for the domain of their coordinates. To do so, users can use API `exchangeParticle` of ParticleSystem object.

Listing 11: Particle Exchange

```
1 system_grav.exchangeParticle(dinfo);
```

4.1.3.4.3 Interaction Calculation

After the domain decomposition and the particle exchange, an interaction calculation is done. To do so, users can use API `calcForceAllAndWriteBack` of Tree object.

Listing 12: Interaction Calculation

```
1 tree_grav.calcForceAllAndWriteBack(CalcGravity<FPGrav>,
2                                    CalcGravity<PS::SPJMonopole>,
3                                    system_grav,
4                                    dinfo);
```

Note that the content of the description `<...>` in the arguments of this method represents a template argument.

4.1.3.4.4 Time integration

In this sample code, we use the Leapfrog method to integrate the particle system in time. In this method, the time evolution operator can be expressed as $K(\frac{\Delta t}{2})D(\Delta t)K(\frac{\Delta t}{2})$, where Δt is the timestep, $K(\Delta t)$ is the 'kick' operator that integrates the velocities of particles from t to $t + \Delta t$, $D(\Delta t)$ is the 'drift' operator that integrates the positions of particles from t to $t + \Delta t$ (e.g. see Springel [2005,MNRAS,364,1105]). In the sample code, these operators are implemented as the `void` functions `kick` and `drift`.

At the beginning of the main loop, the positions and the velocities of the particles are updated by the operator $D(\Delta t)K(\frac{\Delta t}{2})$:

Listing 13: Calculation of $D(\Delta t)K(\frac{\Delta t}{2})$ operator

```
1 kick(system_grav, dt * 0.5);
2 drift(system_grav, dt);
```

After the force calculation, the velocities of the particles are updated by the operator $K(\frac{\Delta t}{2})$:

Listing 14: Calculation of $K(\frac{\Delta t}{2})$ operator

```
1 kick(system_grav, dt * 0.5);
```

4.1.4 Log file

Once the calculation starts successfully, the time and the energy error are printed in the standard output. The first step is shown in the bellow example.

Listing 15: standard output

```
time:     0.0000000000E+000, energy error:    -0.0000000000E+000
```

4.2 SPH simulation code with fixed smoothing length

In this section, we describe the sample code used in the previous section (§ 3), a standard SPH code with fixed smoothing length, in detail.

4.2.1 Location of source files and file structure

The source files of the sample code are in the directory `$(FDPS)/sample/c++/sph`. The sample code consists of `main.cpp` and a Makefile for GCC, `Makefile`.

4.2.2 User-defined types and user-defined functions

In this section, we describe the derived data types and subroutines that users must define when performing SPH simulations by using of FDPS.

4.2.2.1 FullParticle type

Users must define a FullParticle type as a user-defined type. The FullParticle type must contain all physical quantities of an SPH particle necessary for the simulation. Listing 16 shows an example implementation of the `FullParticle` type in our sample code (see `main.cpp`).

Listing 16: FullParticle type

```
struct FP{
   PS::F64 mass;
   PS::F64vec pos;
   PS::F64vec vel;
   PS::F64vec acc;
   PS::F64 dens;
   PS::F64 eng;
   PS::F64 pres;
   PS::F64 smth;
   PS::F64 snds;
   PS::F64 eng_dot;
   PS::F64 dt;
   PS::S64 id;
   PS::F64vec vel_half;
   PS::F64 eng_half;
   void copyFromForce(const Dens& dens){
      this->dens = dens.dens;
   }
   void copyFromForce(const Hydro& force){
```

```
20        this->acc     = force.acc;
21        this->eng_dot = force.eng_dot;
22        this->dt      = force.dt;
23    }
24    PS::F64 getCharge() const{
25        return this->mass;
26    }
27    PS::F64vec getPos() const{
28        return this->pos;
29    }
30    PS::F64 getRSearch() const{
31        return kernelSupportRadius * this->smth;
32    }
33    void setPos(const PS::F64vec& pos){
34        this->pos = pos;
35    }
36    void writeAscii(FILE* fp) const{
37        fprintf(fp,
38                "%lld\t%lf\t%lf\t%lf\t%lf\t%lf\t"
39                "%lf\t%lf\t%lf\t%lf\t%lf\n",
40                this->id, this->mass,
41                this->pos.x, this->pos.y, this->pos.z,
42                this->vel.x, this->vel.y, this->vel.z,
43                this->dens, this->eng, this->pres);
44    }
45    void readAscii(FILE* fp){
46        fscanf(fp,
47               "%lld\t%lf\t%lf\t%lf\t%lf\t%lf\t"
48               "%lf\t%lf\t%lf\t%lf\t%lf\n",
49               &this->id, &this->mass,
50               &this->pos.x, &this->pos.y, &this->pos.z,
51               &this->vel.x, &this->vel.y, &this->vel.z,
52               &this->dens, &this->eng, &this->pres);
53    }
54    void setPressure(){
55        const PS::F64 hcr = 1.4;
56        pres = (hcr - 1.0) * dens * eng;
57        snds = sqrt(hcr * pres / dens);
58    }
59 };
```

FullParticle type must have a member function `copyFromForce` to copy the results from the Force type (explained later). It should have member functions `getCharge()` (returns the particle mass), `getPos()` (returns the particle position), `getRSearch()` (returns the search radius for neighbor particles), and `setPos()` (sets the position). In this sample code, we make use of file I/O functions of FDPS, which requires a user to define member functions `writeAscii()` and `readAscii()`. In addition to them, member function `setPressure()` is defined. This member function calculates the pressure from the equation of states. This function is not used by FDPS, but used within the user code.

4.2.2.2 EssentialParticleI(J) type

Users must define an EssentialParticleI type. An EssentialParticleI type must contain all necessary physical quantities to compute the Force as an i-particle in its member variables. Moreover in this sample code, it also doubles as an EssentialParticleJ type and all necessary physical quantities as a j-particle as well need to be included in the member variables. Hereinafter, we simply call this EssentialParticle type. Listing 17 shows an example of EssentialParticle type of this sample code (see `main.cpp`):

Listing 17: EssentialParticle type

```
1  struct EP{
2     PS::F64vec pos;
3     PS::F64vec vel;
4     PS::F64    mass;
5     PS::F64    smth;
6     PS::F64    dens;
7     PS::F64    pres;
8     PS::F64    snds;
9     void copyFromFP(const FP& rp){
10       this->pos  = rp.pos;
11       this->vel  = rp.vel;
12       this->mass = rp.mass;
13       this->smth = rp.smth;
14       this->dens = rp.dens;
15       this->pres = rp.pres;
16       this->snds = rp.snds;
17    }
18    PS::F64vec getPos() const{
19       return this->pos;
20    }
21    PS::F64 getRSearch() const{
22       return kernelSupportRadius * this->smth;
23    }
24    void setPos(const PS::F64vec& pos){
25       this->pos = pos;
26    }
27 };
```

EssentialParticle type should have member function `copyFromFP` to copy necessary quantities from FullParticle type described above. Also, it should have member functions `getPos()`, `getRSearch()`, and `setPos()`.

4.2.2.3 Force type

Users must define a Force type. A Force type must contain all the resultant physical quantities after performing the Force computations. In this sample code, we have 2 force computations; one for the density and the other for the fluid interactions. Thus, we have to define 2 different Force types. In Listing 18, we show an example of the Force types in this sample code.

Listing 18: Force type

```
1  class Dens{
```

```
2     public:
3     PS::F64 dens;
4     PS::F64 smth;
5     void clear(){
6        dens = 0;
7     }
8  };
9  class Hydro{
10    public:
11    PS::F64vec acc;
12    PS::F64 eng_dot;
13    PS::F64 dt;
14    void clear(){
15       acc = 0;
16       eng_dot = 0;
17    }
18 };
```

A Force type should have member function `clear()`, which zero-clears or initializes member variables that store the result of some accumulation operation.

In this sample, the `Dens` class has a member variable `smth` that stands for the smoothing length of a SPH particle, which is actually unnecessary for a SPH simulation with a *fixed* smoothing length. However, we leave it in the sample code because it would be useful for a user to extend this sample code to a SPH simulation code with *variable* smoothing length. In the formulation by Springel [2005,MNRAS,364,1105] (one of the most popular formulation of SPH with variable smoothing length), it is required to calculate the mass density and the smoothing length simultaneously. If you adopt this formulation, you need to let Force type have a member variable that represents smoothing length as in this sample code. The member function `clear` in the `Dens` class does not zero-clear `smth` because this sample code assume a fixed smoothing length (the density calculation will fail if `smth` is zero-cleared!).

The `Hydro` class has a member variable `dt` that stands for a timestep of each particle. In this sample, `dt` is not zero-cleared because `dt` is not a quantity that stores the result of some accumulation operation and therefore zero-clear is unnecessary.

4.2.2.4 calcForceEpEp

Users must define a `void` function calcForceEpEp in C++ which specifies the interaction between particles. It should contain actual code for the calculation of interaction between particles. Listing 19 shows the implementation of calcForceEpEp (see `main.cppuser_defined.cuser_defined.F90` .

Listing 19: Function calcForceEpEp type

```
1  class CalcDensity{
2     public:
3     void operator () (const EP* const ep_i, const PS::S32 Nip,
4                       const EP* const ep_j, const PS::S32 Njp,
5                       Dens* const dens){
6        for(PS::S32 i = 0 ; i < Nip ; ++i){
7           dens[i].clear();
8           for(PS::S32 j = 0 ; j < Njp ; ++j){
9              const PS::F64vec dr = ep_j[j].pos - ep_i[i].pos;
```

```
10              dens[i].dens += ep_j[j].mass * W(dr, ep_i[i].smth);
11           }
12        }
13     }
14  };
15
16  class CalcHydroForce{
17     public:
18     void operator () (const EP* const ep_i, const PS::S32 Nip,
19                       const EP* const ep_j, const PS::S32 Njp,
20                       Hydro* const hydro){
21        for(PS::S32 i = 0; i < Nip ; ++ i){
22           hydro[i].clear();
23           PS::F64 v_sig_max = 0.0;
24           for(PS::S32 j = 0; j < Njp ; ++j){
25              const PS::F64vec dr = ep_i[i].pos - ep_j[j].pos;
26              const PS::F64vec dv = ep_i[i].vel - ep_j[j].vel;
27              const PS::F64 w_ij = (dv * dr < 0) ? dv * dr / sqrt(dr * dr) :
                     0;
28              const PS::F64 v_sig = ep_i[i].snds + ep_j[j].snds - 3.0 * w_ij
                     ;
29              v_sig_max = std::max(v_sig_max, v_sig);
30              const PS::F64 AV = - 0.5 * v_sig * w_ij / (0.5 * (ep_i[i].dens
                      + ep_j[j].dens));
31              const PS::F64vec gradW_ij = 0.5 * (gradW(dr, ep_i[i].smth) +
                     gradW(dr, ep_j[j].smth));
32              hydro[i].acc     -= ep_j[j].mass * (ep_i[i].pres / (ep_i[i].
                     dens * ep_i[i].dens) + ep_j[j].pres / (ep_j[j].dens *
                     ep_j[j].dens) + AV) * gradW_ij;
33              hydro[i].eng_dot += ep_j[j].mass * (ep_i[i].pres / (ep_i[i].
                     dens * ep_i[i].dens) + 0.5 * AV) * dv * gradW_ij;
34           }
35           hydro[i].dt = C_CFL * 2.0 * ep_i[i].smth / v_sig_max;
36        }
37     }
38  };
```

Users can define calcForceEpEp using a functor (function object). In this sample, we use a functor instead of `void` function. In any case, the arguments of calcForceEpEp are an array of EssentialParticleI type, the number of EssentialParticleI type variables, an array of EssentialParticleJ type, the number of EssentialParticleJ variables, an array of Force type. As described above, two Force classes, one for density and the other for actual hydrodynamic interaction, are used in this code. Thus, two calcForceEpEp should be defined.

4.2.3 The main body of the user program

In this section, we describe `void` functions and functions to be called from the main function of the user program when a user want to do an SPH simulation using FDPS .

4.2.3.1 Including the header file of FDPS

To make the standard features of FDPS available, we must include header file `particle_simulator.hpp`.

Listing 20: Including header file `particle_simulator.hpp`

```
1 #include <particle_simulator.hpp>
```

4.2.3.2 Initialization and termination of FDPS

You should first initialize FDPS by the following code.

Listing 21: Initialization of FDPS

```
1 PS::Initialize(argc, argv);
```

Once started, FDPS should be explicitly terminated. In this sample, FDPS is terminated just before the termination of the program. To achieve this, you write the following code at the end of the main function.

Listing 22: Termination of FDPS

```
1 PS::Finalize();
```

4.2.3.3 Creation and initialization of FDPS objects

After the initialization of FDPS, a user need to create the objects used to talk to FDPS. In this section we describe how to create and initialize these objects.

4.2.3.3.1 Creation of necessary FDPS objects

In an SPH simulation code, one needs to create objects for particles, for domain information, for interaction calculation of Gather type (for density calculation using gather type interaction), and for interaction calculation of Symmetry type (for hydrodynamic interaction calculation using symmetric type interaction). The following is the code to create to them.

Listing 23: Creation of FDPS Objects

```
1 PS::ParticleSystem<FP> sph_system;
2 PS::DomainInfo dinfo;
3 PS::TreeForForceShort<Dens, EP, EP>::Gather dens_tree;
4 PS::TreeForForceShort<Hydro, EP, EP>::Symmetry hydr_tree;
```

4.2.3.3.2 Initialization of the domain information object

FDPS objects created by a user code should be initialized. Here, we describe the necessary procedures required to initialize a DomainInfo object. First, we need to call API `initialize` of DomainInfo object. After the initialization of the object, the type of the boundary and the size of the simulation box should be set by calling APIs `setBoundaryCondition` and `setPosRootDomain` of DomainInfo object. In this code, we use the periodic boundary for all of x, y and z directions.

Listing 24: Initialization of DomainInfo object

```
1 dinfo.initialize();
```

```
2 dinfo.setBoundaryCondition(PS::BOUNDARY_CONDITION_PERIODIC_XYZ);
3 dinfo.setPosRootDomain(PS::F64vec(0.0, 0.0, 0.0),
4                        PS::F64vec(box.x, box.y, box.z));
```

4.2.3.3.3 Initialization of ParticleSystem object

Next, we need to initialize the ParticleSystem object. This is done by the following single line of code:

Listing 25: Initialization of ParticleSystem object

```
1 sph_system.initialize();
```

4.2.3.3.4 Initialization of Tree objects

Finally, Tree objects should be initialized. This is done by calling API `initialize`. This API should be given an integer for determining the initial sizes of internally-used arrays. A number comparable to the number of local particles is enough. In this sample, we set three times the number of local particles:

Listing 26: Initialization of tree objects

```
1 dens_tree.initialize(n_loc);
2 hydr_tree.initialize(n_loc);
```

4.2.3.4 Time integration loop

In this section we describe the structure of the time integration loop.

4.2.3.4.1 Domain Decomposition

First, the computational domain is decomposed, using the current distribution of particles. To do so, the API `decomposeDomainAll` of DomainInfo object is called.

Listing 27: Domain Decomposition

```
1 dinfo.decomposeDomainAll(sph_system);
```

4.2.3.4.2 Particle Exchange

Then particles are exchanged between processes so that they belong to the process for the domain of their coordinates. To do so, the following API `exchangeParticle` of `ParticleSystem` object is used.

Listing 28: Particle Exchange

```
1 sph_system.exchangeParticle(dinfo);
```

4.2.3.4.3 Interaction Calculation

After the domain decomposition and particle exchange, interaction calculation is done. To do so, the following API `calcForceAllAndWriteBack` of Tree object is used.

Listing 29: Interaction Calculation

```
1 dens_tree.calcForceAllAndWriteBack(CalcDensity(), sph_system, dinfo);
2 hydr_tree.calcForceAllAndWriteBack(CalcHydroForce(), sph_system, dinfo);
```

4.2.4 Compilation of the program

Run `make` at the working directory. You can use the Makefile attached to the sample code.

```
$ make
```

4.2.5 Execution

To run the code without MPI, you should execute the following command in the command shell.

```
$ ./sph.out
```

To run the code using MPI, you should execute the following command in the command shell, or follow the document of your system.

```
$ MPIRUN -np NPROC ./sph.out
```

Here, `MPIRUN` represents the command to run your program using MPI such as `mpirun` or `mpiexec`, and `NPROC` is the number of MPI processes.

4.2.6 Log and output files

Log and output files are created under `result` directory.

4.2.7 Visualization

In this section, we describe how to visualize the calculation result using `gnuplot`. To enter the interactive mode of `gnuplot`, execute the following command.

```
$ gnuplot
```

In the interactive mode, you can visualize the result. In the following example, using the 40th snapshot file, we create the plot in which the abscissa is the x coordinate of particles and the ordinate is the density of particles.

```
gnuplot> plot "result/0040.txt" u 3:9
```

5 Sample Codes

5.1 N-body simulation

In this section, we show a sample code for the N-body simulation. This code is the same as what we described in section 4. One can create a working code by cut and paste this code and compile and link the resulted source program.

Listing 30: Sample code of N-body simulation (`user-defined.hpp`)

```
#pragma once
class FileHeader{
public:
    PS::S64 n_body;
    PS::F64 time;
    PS::S32 readAscii(FILE * fp) {
        fscanf(fp, "%lf\n", &time);
        fscanf(fp, "%ld\n", &n_body);
        return n_body;
    }
    void writeAscii(FILE* fp) const {
        fprintf(fp, "%e\n", time);
        fprintf(fp, "%ld\n", n_body);
    }
};

class FPGrav{
public:
    PS::S64    id;
    PS::F64    mass;
    PS::F64vec pos;
    PS::F64vec vel;
    PS::F64vec acc;
    PS::F64    pot;

    static PS::F64 eps;

    PS::F64vec getPos() const {
        return pos;
    }

    PS::F64 getCharge() const {
        return mass;
    }

    void copyFromFP(const FPGrav & fp){
        mass = fp.mass;
        pos  = fp.pos;
    }

    void copyFromForce(const FPGrav & force) {
        acc = force.acc;
        pot = force.pot;
    }

```

```
46      void clear() {
47          acc = 0.0;
48          pot = 0.0;
49      }
50
51      void writeAscii(FILE* fp) const {
52          fprintf(fp, "%ld\t%g\t%g\t%g\t%g\t%g\t%g\t%g\n",
53                  this->id, this->mass,
54                  this->pos.x, this->pos.y, this->pos.z,
55                  this->vel.x, this->vel.y, this->vel.z);
56      }
57
58      void readAscii(FILE* fp) {
59          fscanf(fp, "%ld\t%lf\t%lf\t%lf\t%lf\t%lf\t%lf\t%lf\n",
60                 &this->id, &this->mass,
61                 &this->pos.x, &this->pos.y, &this->pos.z,
62                 &this->vel.x, &this->vel.y, &this->vel.z);
63          }
64
65  };
66
67
68  #ifdef ENABLE_PHANTOM_GRAPE_X86
69
70
71  template <class TParticleJ>
72  void CalcGravity(const FPGrav * iptcl,
73                   const PS::S32 ni,
74                   const TParticleJ * jptcl,
75                   const PS::S32 nj,
76                   FPGrav * force) {
77      const PS::S32 nipipe = ni;
78      const PS::S32 njpipe = nj;
79      PS::F64 (*xi)[3] = (PS::F64 (*)[3])malloc(sizeof(PS::F64) * nipipe *
            PS::DIMENSION);
80      PS::F64 (*ai)[3] = (PS::F64 (*)[3])malloc(sizeof(PS::F64) * nipipe *
            PS::DIMENSION);
81      PS::F64  *pi     = (PS::F64  *    )malloc(sizeof(PS::F64) * nipipe);
82      PS::F64 (*xj)[3] = (PS::F64 (*)[3])malloc(sizeof(PS::F64) * njpipe *
            PS::DIMENSION);
83      PS::F64  *mj     = (PS::F64  *    )malloc(sizeof(PS::F64) * njpipe);
84      for(PS::S32 i = 0; i < ni; i++) {
85          xi[i][0] = iptcl[i].getPos()[0];
86          xi[i][1] = iptcl[i].getPos()[1];
87          xi[i][2] = iptcl[i].getPos()[2];
88          ai[i][0] = 0.0;
89          ai[i][1] = 0.0;
90          ai[i][2] = 0.0;
91          pi[i]    = 0.0;
92      }
93      for(PS::S32 j = 0; j < nj; j++) {
94          xj[j][0] = jptcl[j].getPos()[0];
95          xj[j][1] = jptcl[j].getPos()[1];
96          xj[j][2] = jptcl[j].getPos()[2];
97          mj[j]    = jptcl[j].getCharge();
```

```
 98          xj[j][0] = jptcl[j].pos[0];
 99          xj[j][1] = jptcl[j].pos[1];
100          xj[j][2] = jptcl[j].pos[2];
101          mj[j]    = jptcl[j].mass;
102      }
103      PS::S32 devid = PS::Comm::getThreadNum();
104      g5_set_xmjMC(devid, 0, nj, xj, mj);
105      g5_set_nMC(devid, nj);
106      g5_calculate_force_on_xMC(devid, xi, ai, pi, ni);
107      for(PS::S32 i = 0; i < ni; i++) {
108          force[i].acc[0] += ai[i][0];
109          force[i].acc[1] += ai[i][1];
110          force[i].acc[2] += ai[i][2];
111          force[i].pot    -= pi[i];
112      }
113      free(xi);
114      free(ai);
115      free(pi);
116      free(xj);
117      free(mj);
118  }
119
120  #elif USE_PIKG_KERNEL
121  struct Epi{
122    PS::F32vec pos;
123  };
124  struct Epj{
125    PS::F32vec pos;
126    PS::F32    mass;
127  };
128  struct Force{
129    PS::F32vec acc;
130    PS::F32    pot;
131  };
132
133
134
135  #include "kernel_pikg.hpp"
136
137  template <class TParticleJ>
138  void CalcGravity(const FPGrav * ep_i,
139                   const PS::S32 n_ip,
140                   const TParticleJ * ep_j,
141                   const PS::S32 n_jp,
142                   FPGrav * force) {
143    Epi epi[n_ip];
144    Force f[n_ip];
145    for(int i=0;i<n_ip;i++){
146      epi[i].pos = (PS::F32vec)(ep_i[i].pos - ep_i[0].pos);
147
148      f[i].acc = force[i].acc;
149      f[i].pot = force[i].pot;
150    }
151    Epj epj[n_jp];
152    for(int i=0;i<n_jp;i++){
```

```
153     epj[i].pos = (PS::F32vec)(ep_j[i].pos - ep_i[0].pos);
154     epj[i].mass = ep_j[i].mass;
155   }
156   CalcGravityEpEp(FPGrav::eps*FPGrav::eps)(epi,n_ip,epj,n_jp,f);
157   for(int i=0;i<n_ip;i++){
158     force[i].acc = f[i].acc;
159     force[i].pot = f[i].pot;
160   }
161 }
162 #else
163
164 template <class TParticleJ>
165 void CalcGravity(const FPGrav * ep_i,
166                  const PS::S32 n_ip,
167                  const TParticleJ * ep_j,
168                  const PS::S32 n_jp,
169                  FPGrav * force) {
170     PS::F64 eps2 = FPGrav::eps * FPGrav::eps;
171     for(PS::S32 i = 0; i < n_ip; i++){
172         PS::F64vec xi = ep_i[i].getPos();
173         PS::F64vec ai = 0.0;
174         PS::F64 poti = 0.0;
175         for(PS::S32 j = 0; j < n_jp; j++){
176             PS::F64vec rij    = xi - ep_j[j].getPos();
177             PS::F64    r3_inv = rij * rij + eps2;
178             PS::F64    r_inv  = 1.0/sqrt(r3_inv);
179             r3_inv  = r_inv * r_inv;
180             r_inv  *= ep_j[j].getCharge();
181             r3_inv *= r_inv;
182             ai     -= r3_inv * rij;
183             poti   -= r_inv;
184         }
185         force[i].acc += ai;
186         force[i].pot += poti;
187     }
188 }
189
190 #endif
```

Listing 31: Sample code of N-body simulation (`nbody.cpp`)

```
 1 #include <iostream>
 2 #include <fstream>
 3 #include <unistd.h>
 4 #include <sys/stat.h>
 5 #include <particle_simulator.hpp>
 6 #ifdef ENABLE_PHANTOM_GRAPE_X86
 7 #include <gp5util.h>
 8 #endif
 9 #ifdef ENABLE_GPU_CUDA
10 #define MULTI_WALK
11 #include "force_gpu_cuda.hpp"
12 #endif
13 #include "user-defined.hpp"
14
```

```
15 void makeColdUniformSphere(const PS::F64 mass_glb, const PS::S64 n_glb,
       const PS::S64 n_loc, PS::F64 *&mass, PS::F64vec *&pos, PS::F64vec *&
       vel,
16                            const PS::F64 eng = -0.25, const PS::S32 seed =
                                  0) {
17     assert(eng < 0.0);
18     {
19         PS::MTTS mt;
20         mt.init_genrand(0);
21         for (PS::S32 i = 0; i < n_loc; i++) {
22             mass[i] = mass_glb / n_glb;
23             const PS::F64 radius = 3.0;
24             do {
25                 pos[i][0] = (2. * mt.genrand_res53() - 1.) * radius;
26                 pos[i][1] = (2. * mt.genrand_res53() - 1.) * radius;
27                 pos[i][2] = (2. * mt.genrand_res53() - 1.) * radius;
28             } while (pos[i] * pos[i] >= radius * radius);
29             vel[i][0] = 0.0;
30             vel[i][1] = 0.0;
31             vel[i][2] = 0.0;
32         }
33     }
34
35     PS::F64vec cm_pos = 0.0;
36     PS::F64vec cm_vel = 0.0;
37     PS::F64 cm_mass = 0.0;
38     for (PS::S32 i = 0; i < n_loc; i++) {
39         cm_pos += mass[i] * pos[i];
40         cm_vel += mass[i] * vel[i];
41         cm_mass += mass[i];
42     }
43     cm_pos /= cm_mass;
44     cm_vel /= cm_mass;
45     for (PS::S32 i = 0; i < n_loc; i++) {
46         pos[i] -= cm_pos;
47         vel[i] -= cm_vel;
48     }
49 }
50
51 template <class Tpsys>
52 void setParticlesColdUniformSphere(Tpsys &psys, const PS::S32 n_glb, PS::
       S32 &n_loc) {
53     n_loc = n_glb;
54     psys.setNumberOfParticleLocal(n_loc);
55
56     PS::F64 *mass = new PS::F64[n_loc];
57     PS::F64vec *pos = new PS::F64vec[n_loc];
58     PS::F64vec *vel = new PS::F64vec[n_loc];
59     const PS::F64 m_tot = 1.0;
60     const PS::F64 eng = -0.25;
61     makeColdUniformSphere(m_tot, n_glb, n_loc, mass, pos, vel, eng);
62     for (PS::S32 i = 0; i < n_loc; i++) {
63         psys[i].mass = mass[i];
64         psys[i].pos = pos[i];
65         psys[i].vel = vel[i];
```

```
66          psys[i].id = i;
67      }
68      delete[] mass;
69      delete[] pos;
70      delete[] vel;
71  }
72
73  template <class Tpsys>
74  void kick(Tpsys &system, const PS::F64 dt) {
75      PS::S32 n = system.getNumberOfParticleLocal();
76      for (PS::S32 i = 0; i < n; i++) {
77          system[i].vel += system[i].acc * dt;
78      }
79  }
80
81  template <class Tpsys>
82  void drift(Tpsys &system, const PS::F64 dt) {
83      PS::S32 n = system.getNumberOfParticleLocal();
84      for (PS::S32 i = 0; i < n; i++) {
85          system[i].pos += system[i].vel * dt;
86      }
87  }
88
89  template <class Tpsys>
90  void calcEnergy(const Tpsys &system, PS::F64 &etot, PS::F64 &ekin, PS::F64
          &epot, const bool clear = true) {
91      if (clear) {
92          etot = ekin = epot = 0.0;
93      }
94      PS::F64 etot_loc = 0.0;
95      PS::F64 ekin_loc = 0.0;
96      PS::F64 epot_loc = 0.0;
97      const PS::S32 nbody = system.getNumberOfParticleLocal();
98      for (PS::S32 i = 0; i < nbody; i++) {
99          ekin_loc += system[i].mass * system[i].vel * system[i].vel;
100         epot_loc += system[i].mass * (system[i].pot + system[i].mass /
                FPGrav::eps);
101     }
102     ekin_loc *= 0.5;
103     epot_loc *= 0.5;
104     etot_loc = ekin_loc + epot_loc;
105     etot = PS::Comm::getSum(etot_loc);
106     epot = PS::Comm::getSum(epot_loc);
107     ekin = PS::Comm::getSum(ekin_loc);
108 }
109
110 void printHelp() {
111     std::cerr << "o: dir name of output (default: ./result)" << std::endl;
112     std::cerr << "t: theta (default: 0.5)" << std::endl;
113     std::cerr << "T: time_end (default: 10.0)" << std::endl;
114     std::cerr << "s: time_step (default: 1.0 / 128.0)" << std::endl;
115     std::cerr << "d: dt_diag (default: 1.0 / 8.0)" << std::endl;
116     std::cerr << "D: dt_snap (default: 1.0)" << std::endl;
117     std::cerr << "l: n_leaf_limit (default: 8)" << std::endl;
118     std::cerr << "n: n_group_limit (default: 64)" << std::endl;
```

```
119     std::cerr << "N: n_tot (default: 1024)" << std::endl;
120     std::cerr << "h: help" << std::endl;
121 }
122
123 void makeOutputDirectory(char *dir_name) {
124     struct stat st;
125     PS::S32 ret;
126     if (PS::Comm::getRank() == 0) {
127         if (stat(dir_name, &st) != 0) {
128             ret = mkdir(dir_name, 0777);
129         } else {
130             ret = 0;  // the directory named dir_name already exists.
131         }
132     }
133     PS::Comm::broadcast(&ret, 1);
134     if (ret == 0) {
135         if (PS::Comm::getRank() == 0) fprintf(stderr, "Directory \"%s\" is
                successfully made.\n", dir_name);
136     } else {
137         if (PS::Comm::getRank() == 0) fprintf(stderr, "Directory %s fails
                to be made.\n", dir_name);
138         PS::Abort();
139     }
140 }
141
142 PS::F64 FPGrav::eps = 1.0 / 32.0;
143
144 int main(int argc, char *argv[]) {
145     std::cout << std::setprecision(15);
146     std::cerr << std::setprecision(15);
147
148     PS::Initialize(argc, argv);
149     PS::F64 theta = 0.5;
150     PS::S32 n_leaf_limit = 8;
151     PS::S32 n_group_limit = 64;
152     PS::F64 time_end = 10.0;
153     PS::F64 dt = 1.0 / 128.0;
154     PS::F64 dt_diag = 1.0 / 8.0;
155     PS::F64 dt_snap = 1.0;
156     PS::S32 n_steps_reuse = 4;
157     bool clear_force = true;
158     char dir_name[1024];
159     PS::S64 n_tot = 1024;
160     PS::S32 c;
161     sprintf(dir_name, "./result");
162     opterr = 0;
163     while ((c = getopt(argc, argv, "i:o:d:D:t:T:l:n:N:hs:")) != -1) {
164         switch (c) {
165             case 'o':
166                 strncat(dir_name, optarg, 1000);
167                 break;
168             case 't':
169                 theta = atof(optarg);
170                 std::cerr << "theta =" << theta << std::endl;
171                 break;
```

```
172             case 'T':
173                 time_end = atof(optarg);
174                 std::cerr << "time_end = " << time_end << std::endl;
175                 break;
176             case 's':
177                 dt = atof(optarg);
178                 std::cerr << "time_step = " << dt << std::endl;
179                 break;
180             case 'd':
181                 dt_diag = atof(optarg);
182                 std::cerr << "dt_diag = " << dt_diag << std::endl;
183                 break;
184             case 'D':
185                 dt_snap = atof(optarg);
186                 std::cerr << "dt_snap = " << dt_snap << std::endl;
187                 break;
188             case 'l':
189                 n_leaf_limit = atoi(optarg);
190                 std::cerr << "n_leaf_limit = " << n_leaf_limit << std::
                        endl;
191                 break;
192             case 'n':
193                 n_group_limit = atoi(optarg);
194                 std::cerr << "n_group_limit = " << n_group_limit << std::
                        endl;
195                 break;
196             case 'N':
197                 n_tot = atoi(optarg);
198                 std::cerr << "n_tot = " << n_tot << std::endl;
199                 break;
200             case 'h':
201                 if (PS::Comm::getRank() == 0) {
202                     printHelp();
203                 }
204                 PS::Finalize();
205                 return 0;
206             default:
207                 if (PS::Comm::getRank() == 0) {
208                     std::cerr << "No such option! Available options are 
                            here." << std::endl;
209                     printHelp();
210                 }
211                 PS::Abort();
212         }
213     }
214
215     makeOutputDirectory(dir_name);
216
217     std::ofstream fout_eng;
218
219     if (PS::Comm::getRank() == 0) {
220         char sout_de[1280];
221         sprintf(sout_de, "%s/t-de.dat", dir_name);
222         fout_eng.open(sout_de);
223         fprintf(stdout, "This is a sample program of N-body simulation on 
```

```
                FDPS!\n");
224         fprintf(stdout, "Number of processes: %d\n", PS::Comm::
                getNumberOfProc());
225         fprintf(stdout, "Number of threads per process: %d\n", PS::Comm::
                getNumberOfThread());
226     }
227
228     PS::ParticleSystem<FPGrav> system_grav;
229     system_grav.initialize();
230     PS::S32 n_loc = 0;
231     PS::F32 time_sys = 0.0;
232     if (PS::Comm::getRank() == 0) {
233         setParticlesColdUniformSphere(system_grav, n_tot, n_loc);
234     } else {
235         system_grav.setNumberOfParticleLocal(n_loc);
236     }
237
238     const PS::F32 coef_ema = 0.3;
239     PS::DomainInfo dinfo;
240     dinfo.initialize(coef_ema);
241     dinfo.decomposeDomainAll(system_grav);
242     system_grav.exchangeParticle(dinfo);
243     n_loc = system_grav.getNumberOfParticleLocal();
244
245 #if defined(ENABLE_PHANTOM_GRAPE_X86)
246     g5_open();
247     g5_set_eps_to_all(FPGrav::eps);
248 #endif
249
250     PS::TreeForForceLong<FPGrav, FPGrav, FPGrav>::Monopole tree_grav;
251     tree_grav.initialize(n_loc, theta, n_leaf_limit, n_group_limit);
252     //tree_grav.let_usage_mode = PS::LET_USAGE_MODE::CONSTRUCT_GLOBAL_TREE
            ;
253     tree_grav.let_usage_mode_ = PS::LET_USAGE_MODE::CONSTRUCT_LET_TREE;
254
255     /*
256     // CHECKED
257     tree_grav.setParticleLocalTree(system_grav);
258     tree_grav.makeLocalTree(dinfo);
259     tree_grav.makeGlobalTree(dinfo);
260     tree_grav.calcMomentGlobalTree();
261     tree_grav.calcForce(CalcGravity<FPGrav>, CalcGravity<PS::SPJMonopole>)
            ;
262     for(int i=0; i<system_grav.getNumberOfParticleLocal(); i++){
263         system_grav[i].copyFromForce(tree_grav.getForce(i));
264         std::cout<<"i= "<<i<<" acc= "<<system_grav[i].acc<<std::endl;
265     }
266     PS::Finalize();
267     return 0;
268     // CHECKED
269     */
270
271 #if defined(MULTI_WALK_PTCL)
272     const PS::S32 n_walk_limit = 4;
273     const PS::S32 tag_max = 1;
```

```
274     //tree_grav.calcForceAllAndWriteBackMultiWalk(DispatchKernelWithSP,
            RetrieveKernel, tag_max, system_grav, dinfo, n_walk_limit);
275     tree_grav.calcForceAllAndWriteBackMultiWalkVA(DispatchKernelWithSP,
            RetrieveKernel, tag_max, system_grav, dinfo, n_walk_limit);
276 #elif defined(ENABLE_GPU_CUDA)
277     const PS::S32 n_walk_limit = 4;
278     const PS::S32 tag_max = 1;
279     //tree_grav.calcForceAllAndWriteBackMultiWalkIndex(
            DispatchKernelWithSPIndex, RetrieveKernel, tag_max, system_grav,
             dinfo, n_walk_limit);
280     tree_grav.calcForceAllAndWriteBackMultiWalkIndexVA(
            DispatchKernelWithSPIndex, RetrieveKernel, tag_max, system_grav,
             dinfo, n_walk_limit);
281 #else
282     tree_grav.calcForceAllAndWriteBack(CalcGravity<FPGrav>, CalcGravity<PS
            ::SPJMonopole>, system_grav, dinfo);
283     //tree_grav.calcForceAllAndWriteBackVA(CalcGravity<FPGrav>,
            CalcGravity<PS::SPJMonopole>, system_grav, dinfo);
284     /*
285     for(int i=0; i<system_grav.getNumberOfParticleLocal(); i++){
286         system_grav[i].copyFromForce(tree_grav.getForce(i));
287         std::cout<<"i= "<<i<<" acc= "<<system_grav[i].acc<<std::endl;
288     }
289     PS::Finalize();
290     return 0;
291     */
292 #endif
293     PS::F64 Epot0, Ekin0, Etot0, Epot1, Ekin1, Etot1;
294     calcEnergy(system_grav, Etot0, Ekin0, Epot0);
295     PS::F64 time_diag = 0.0;
296     PS::F64 time_snap = 0.0;
297     PS::S64 n_loop = 0;
298     PS::S32 id_snap = 0;
299     while (time_sys <= time_end) {
300         if ((time_sys >= time_snap) || ((time_sys + dt) - time_snap) > (
                time_snap - time_sys)) {
301             char filename[1280];
302             sprintf(filename, "%s/%04d.dat", dir_name, id_snap++);
303             FileHeader header;
304             header.time = time_sys;
305             header.n_body = system_grav.getNumberOfParticleGlobal();
306             system_grav.writeParticleAscii(filename, header);
307             time_snap += dt_snap;
308         }
309
310         calcEnergy(system_grav, Etot1, Ekin1, Epot1);
311
312         if (PS::Comm::getRank() == 0) {
313             if ((time_sys >= time_diag) || ((time_sys + dt) - time_diag) >
                    (time_diag - time_sys)) {
314                 fout_eng << time_sys << "␣␣␣" << (Etot1 - Etot0) / Etot0
                        << std::endl;
315                 fprintf(stdout, "time:␣%10.7f␣energy␣error:␣%+e\n",
                        time_sys, (Etot1 - Etot0) / Etot0);
316                 time_diag += dt_diag;
```

```
317             }
318         }
319
320         kick(system_grav, dt * 0.5);
321
322         time_sys += dt;
323         drift(system_grav, dt);
324
325         auto list_mode = PS::REUSE_LIST;
326         if (n_loop % n_steps_reuse == 0) {
327             dinfo.decomposeDomainAll(system_grav);
328             system_grav.exchangeParticle(dinfo);
329             list_mode = PS::MAKE_LIST_FOR_REUSE;
330         }
331 #if defined(MULTI_WALK_PTCL)
332         //tree_grav.calcForceAllAndWriteBackMultiWalk(DispatchKernelWithSP
                , RetrieveKernel, tag_max, system_grav, dinfo, n_walk_limit,
                 true);
333         tree_grav.calcForceAllAndWriteBackMultiWalkVA(DispatchKernelWithSP
                , RetrieveKernel, tag_max, system_grav, dinfo, n_walk_limit,
                 clear_force, list_mode);
334 #elif defined(ENABLE_GPU_CUDA)
335         //tree_grav.calcForceAllAndWriteBackMultiWalkIndex(
                DispatchKernelWithSPIndex, RetrieveKernel, tag_max,
                system_grav, dinfo, n_walk_limit, true);
336         tree_grav.calcForceAllAndWriteBackMultiWalkIndexVA(
                DispatchKernelWithSPIndex, RetrieveKernel, tag_max,
                system_grav, dinfo, n_walk_limit, clear_force, list_mode);
337 #else
338         //tree_grav.calcForceAllAndWriteBack(CalcGravity<FPGrav>,
                CalcGravity<PS::SPJMonopole>, system_grav, dinfo);
339         tree_grav.calcForceAllAndWriteBackVA(CalcGravity<FPGrav>,
                CalcGravity<PS::SPJMonopole>, system_grav, dinfo,
                clear_force, list_mode);
340 #endif
341
342         kick(system_grav, dt * 0.5);
343
344         n_loop++;
345     }
346
347 #if defined(ENABLE_PHANTOM_GRAPE_X86)
348     g5_close();
349 #endif
350
351     PS::Finalize();
352     return 0;
353 }
```

5.2 SPH simulation with fixed smoothing length

In this section, we show a sample code for the SPH simulation with fixed smoothing length. This code is the same as what we described in section 4. One can create a working code by cut and paste this code and compile and link the resulted source program.

Listing 32: Sample code of SPH simulation

```
// Include FDPS header
#include <particle_simulator.hpp>
// Include the standard C++ headers
#include <cmath>
#include <cstdio>
#include <iostream>
#include <vector>
#include <sys/stat.h>

/* Parameters */
const short int Dim = 3;
const PS::F64 SMTH = 1.2;
const PS::U32 OUTPUT_INTERVAL = 10;
const PS::F64 C_CFL = 0.3;

/* Kernel Function */
const PS::F64 pi = atan(1.0) * 4.0;
const PS::F64 kernelSupportRadius = 2.5;

PS::F64 W(const PS::F64vec dr, const PS::F64 h){
   const PS::F64 H = kernelSupportRadius * h;
   const PS::F64 s = sqrt(dr * dr) / H;
   const PS::F64 s1 = (1.0 - s < 0) ? 0 : 1.0 - s;
   const PS::F64 s2 = (0.5 - s < 0) ? 0 : 0.5 - s;
   PS::F64 r_value = pow(s1, 3) - 4.0 * pow(s2, 3);
   //if # of dimension == 3
   r_value *= 16.0 / pi / (H * H * H);
   return r_value;
}

PS::F64vec gradW(const PS::F64vec dr, const PS::F64 h){
   const PS::F64 H = kernelSupportRadius * h;
   const PS::F64 s = sqrt(dr * dr) / H;
   const PS::F64 s1 = (1.0 - s < 0) ? 0 : 1.0 - s;
   const PS::F64 s2 = (0.5 - s < 0) ? 0 : 0.5 - s;
   PS::F64 r_value = - 3.0 * pow(s1, 2) + 12.0 * pow(s2, 2);
   //if # of dimension == 3
   r_value *= 16.0 / pi / (H * H * H);
   return dr * r_value / (sqrt(dr * dr) * H + 1.0e-6 * h);
}

/* Class Definitions */
//** Force Class (Result Class)
class Dens{
   public:
   PS::F64 dens;
   PS::F64 smth;
   void clear(){
      dens = 0;
   }
};
class Hydro{
   public:
   PS::F64vec acc;
```

```
55     PS::F64 eng_dot;
56     PS::F64 dt;
57     void clear(){
58        acc = 0;
59        eng_dot = 0;
60     }
61  };
62
63  //** Full Particle Class
64  struct FP{
65     PS::F64 mass;
66     PS::F64vec pos;
67     PS::F64vec vel;
68     PS::F64vec acc;
69     PS::F64 dens;
70     PS::F64 eng;
71     PS::F64 pres;
72     PS::F64 smth;
73     PS::F64 snds;
74     PS::F64 eng_dot;
75     PS::F64 dt;
76     PS::S64 id;
77     PS::F64vec vel_half;
78     PS::F64 eng_half;
79     void copyFromForce(const Dens& dens){
80        this->dens = dens.dens;
81     }
82     void copyFromForce(const Hydro& force){
83        this->acc     = force.acc;
84        this->eng_dot = force.eng_dot;
85        this->dt      = force.dt;
86     }
87     PS::F64 getCharge() const{
88        return this->mass;
89     }
90     PS::F64vec getPos() const{
91        return this->pos;
92     }
93     PS::F64 getRSearch() const{
94        return kernelSupportRadius * this->smth;
95     }
96     void setPos(const PS::F64vec& pos){
97        this->pos = pos;
98     }
99     void writeAscii(FILE* fp) const{
100       fprintf(fp,
101               "%lld\t%lf\t%lf\t%lf\t%lf\t%lf\t"
102               "%lf\t%lf\t%lf\t%lf\t%lf\n",
103               this->id, this->mass,
104               this->pos.x, this->pos.y, this->pos.z,
105               this->vel.x, this->vel.y, this->vel.z,
106               this->dens, this->eng, this->pres);
107    }
108    void readAscii(FILE* fp){
109       fscanf(fp,
```

```
110                 "%lld\t%lf\t%lf\t%lf\t%lf\t%lf\t"
111                 "%lf\t%lf\t%lf\t%lf\t%lf\n",
112                 &this->id, &this->mass,
113                 &this->pos.x, &this->pos.y, &this->pos.z,
114                 &this->vel.x, &this->vel.y, &this->vel.z,
115                 &this->dens, &this->eng, &this->pres);
116     }
117     void setPressure(){
118         const PS::F64 hcr = 1.4;
119         pres = (hcr - 1.0) * dens * eng;
120         snds = sqrt(hcr * pres / dens);
121     }
122 };
123
124 //** Essential Particle Class
125 struct EP{
126     PS::F64vec pos;
127     PS::F64vec vel;
128     PS::F64    mass;
129     PS::F64    smth;
130     PS::F64    dens;
131     PS::F64    pres;
132     PS::F64    snds;
133     void copyFromFP(const FP& rp){
134         this->pos  = rp.pos;
135         this->vel  = rp.vel;
136         this->mass = rp.mass;
137         this->smth = rp.smth;
138         this->dens = rp.dens;
139         this->pres = rp.pres;
140         this->snds = rp.snds;
141     }
142     PS::F64vec getPos() const{
143         return this->pos;
144     }
145     PS::F64 getRSearch() const{
146         return kernelSupportRadius * this->smth;
147     }
148     void setPos(const PS::F64vec& pos){
149         this->pos = pos;
150     }
151 };
152
153 class FileHeader{
154     public:
155     PS::S32 Nbody;
156     PS::F64 time;
157     int readAscii(FILE* fp){
158         fscanf(fp, "%lf\n", &time);
159         fscanf(fp, "%d\n", &Nbody);
160         return Nbody;
161     }
162     void writeAscii(FILE* fp) const{
163         fprintf(fp, "%e\n", time);
164         fprintf(fp, "%d\n", Nbody);
```

```
165     }
166 };
167
168 struct boundary{
169     PS::F64 x, y, z;
170 };
171
172
173 /* Force Functors */
174 class CalcDensity{
175     public:
176     void operator () (const EP* const ep_i, const PS::S32 Nip,
177                       const EP* const ep_j, const PS::S32 Njp,
178                       Dens* const dens){
179         for(PS::S32 i = 0 ; i < Nip ; ++i){
180             dens[i].clear();
181             for(PS::S32 j = 0 ; j < Njp ; ++j){
182                 const PS::F64vec dr = ep_j[j].pos - ep_i[i].pos;
183                 dens[i].dens += ep_j[j].mass * W(dr, ep_i[i].smth);
184             }
185         }
186     }
187 };
188
189 class CalcHydroForce{
190     public:
191     void operator () (const EP* const ep_i, const PS::S32 Nip,
192                       const EP* const ep_j, const PS::S32 Njp,
193                       Hydro* const hydro){
194         for(PS::S32 i = 0; i < Nip ; ++ i){
195             hydro[i].clear();
196             PS::F64 v_sig_max = 0.0;
197             for(PS::S32 j = 0; j < Njp ; ++j){
198                 const PS::F64vec dr = ep_i[i].pos - ep_j[j].pos;
199                 const PS::F64vec dv = ep_i[i].vel - ep_j[j].vel;
200                 const PS::F64 w_ij = (dv * dr < 0) ? dv * dr / sqrt(dr * dr) :
                        0;
201                 const PS::F64 v_sig = ep_i[i].snds + ep_j[j].snds - 3.0 * w_ij
                        ;
202                 v_sig_max = std::max(v_sig_max, v_sig);
203                 const PS::F64 AV = - 0.5 * v_sig * w_ij / (0.5 * (ep_i[i].dens
                        + ep_j[j].dens));
204                 const PS::F64vec gradW_ij = 0.5 * (gradW(dr, ep_i[i].smth) +
                       gradW(dr, ep_j[j].smth));
205                 hydro[i].acc     -= ep_j[j].mass * (ep_i[i].pres / (ep_i[i].
                       dens * ep_i[i].dens) + ep_j[j].pres / (ep_j[j].dens *
                       ep_j[j].dens) + AV) * gradW_ij;
206                 hydro[i].eng_dot += ep_j[j].mass * (ep_i[i].pres / (ep_i[i].
                       dens * ep_i[i].dens) + 0.5 * AV) * dv * gradW_ij;
207             }
208             hydro[i].dt = C_CFL * 2.0 * ep_i[i].smth / v_sig_max;
209         }
210     }
211 };
212
```

```
213 void makeOutputDirectory(char * dir_name) {
214     struct stat st;
215     PS::S32 ret;
216     if (PS::Comm::getRank() == 0) {
217         if (stat(dir_name, &st) != 0) {
218             ret = mkdir(dir_name, 0777);
219         } else {
220             ret = 0; // the directory named dir_name already exists.
221         }
222     }
223     PS::Comm::broadcast(&ret, 1);
224     if (ret == 0) {
225         if (PS::Comm::getRank() == 0)
226             fprintf(stderr, "Directory␣\"%s\"␣is␣successfully␣made.\n",
                    dir_name);
227     } else {
228         if (PS::Comm::getRank() == 0)
229             fprintf(stderr, "Directory␣%s␣fails␣to␣be␣made.\n", dir_name);
230         PS::Abort();
231     }
232 }
233
234 void SetupIC(PS::ParticleSystem<FP>& sph_system, PS::F64 *end_time,
        boundary *box){
235     // Set the interparticle distance and the box size
236     const PS::F64 dx = 1.0 / 128.0;
237     box->x = 1.0;
238     box->y = box->z = box->x / 8.0;
239     // Count # of particles in the computational domain
240     PS::S32 nx_left {0}, ny_left {0}, nz_left {0};
241     for(PS::F64 x = 0 ; x < box->x * 0.5 ; x += dx) nx_left++;
242     for(PS::F64 y = 0 ; y < box->y ; y += dx) ny_left++;
243     for(PS::F64 z = 0 ; z < box->z ; z += dx) nz_left++;
244     PS::S32 nx_right {0}, ny_right {0}, nz_right {0};
245     for(PS::F64 x = box->x * 0.5 ; x < box->x * 1.0 ; x += dx * 2.0)
            nx_right++;
246     for(PS::F64 y = 0 ; y < box->y ; y += dx) ny_right++;
247     for(PS::F64 z = 0 ; z < box->z ; z += dx) nz_right++;
248     const PS::S64 n_ptcl_glb = (nx_left * ny_left * nz_left)
249                              + (nx_right * ny_right * nz_right);
250     // Set # of local particles
251     const PS::S32 n_proc = PS::Comm::getNumberOfProc();
252     const PS::S32 my_rank = PS::Comm::getRank();
253     const PS::S32 n_ptcl_quot = n_ptcl_glb / n_proc; // quotient
254     const PS::S32 n_ptcl_rem = n_ptcl_glb % n_proc; // remainder
255     if (my_rank == 0) {
256         std::cout << "n_ptcl_glb␣␣=␣" << n_ptcl_glb << std::endl;
257         std::cout << "n_ptcl_quot␣=␣" << n_ptcl_quot << std::endl;
258         std::cout << "n_ptcl_rem␣␣=␣" << n_ptcl_rem << std::endl;
259     }
260     const PS::S32 n_ptcl_loc = n_ptcl_quot + ((my_rank < n_ptcl_rem) ? 1 :
            0);
261     sph_system.setNumberOfParticleLocal(n_ptcl_loc);
262     const PS::S64 i_head = n_ptcl_quot * my_rank
263                          + ((my_rank > (n_ptcl_rem-1)) ? n_ptcl_rem :
```

```
                                    my_rank);
264     const PS::S64 i_tail = i_head + n_ptcl_loc;
265     for (PS::S32 rank = 0; rank < n_proc; rank++) {
266        if (rank == my_rank) {
267            std::cout << "my_rank␣=␣" << my_rank
268                      << ",␣n_ptcl_loc␣=␣" << n_ptcl_loc
269                      << ",␣i_head␣=␣" << i_head
270                      << ",␣i_tail␣=␣" << i_tail
271                      << std::endl;
272        }
273        PS::Comm::barrier();
274     }
275     // Set local particles
276     const PS::F64 sv = (box->x * box->y * box->z)
277                      / static_cast<PS::F64>(n_ptcl_glb);
278     PS::S64 i = 0;
279     for(PS::F64 x = 0 ; x < box->x * 0.5 ; x += dx){
280        for(PS::F64 y = 0 ; y < box->y ; y += dx){
281           for(PS::F64 z = 0 ; z < box->z ; z += dx){
282              if (i_head <= i && i < i_tail) {
283                  const PS::S32 ii = i - i_head;
284                  sph_system[ii].pos.x = x;
285                  sph_system[ii].pos.y = y;
286                  sph_system[ii].pos.z = z;
287                  sph_system[ii].dens = 1.0;
288                  sph_system[ii].mass = 0.75 * sv;
289                  sph_system[ii].eng  = 2.5;
290                  sph_system[ii].id   = i;
291                  sph_system[ii].smth = 0.012;
292              }
293              i++;
294           }
295        }
296     }
297     for(PS::F64 x = box->x * 0.5 ; x < box->x * 1.0 ; x += dx * 2.0){
298        for(PS::F64 y = 0 ; y < box->y ; y += dx){
299           for(PS::F64 z = 0 ; z < box->z ; z += dx){
300              if(i_head <= i && i < i_tail) {
301                  const PS::S32 ii = i - i_head;
302                  sph_system[ii].pos.x = x;
303                  sph_system[ii].pos.y = y;
304                  sph_system[ii].pos.z = z;
305                  sph_system[ii].dens = 0.5;
306                  sph_system[ii].mass = 0.75 * sv;
307                  sph_system[ii].eng  = 2.5;
308                  sph_system[ii].id   = i;
309                  sph_system[ii].smth = 0.012;
310              }
311              i++;
312           }
313        }
314     }
315     assert(i == n_ptcl_glb);
316     // Set the end time
317     *end_time = 0.12;
```

```
318     // Fin.
319     if (my_rank == 0) {
320         std::cout << "SetupIC() is completed." << std::endl;
321     }
322 }
323
324 void Initialize(PS::ParticleSystem<FP>& sph_system){
325     for(PS::S32 i = 0 ; i < sph_system.getNumberOfParticleLocal() ; ++ i){
326         sph_system[i].setPressure();
327     }
328 }
329
330 PS::F64 getTimeStepGlobal(const PS::ParticleSystem<FP>& sph_system){
331     PS::F64 dt = 1.0e+30; //set VERY LARGE VALUE
332     for(PS::S32 i = 0 ; i < sph_system.getNumberOfParticleLocal() ; ++ i){
333         dt = std::min(dt, sph_system[i].dt);
334     }
335     return PS::Comm::getMinValue(dt);
336 }
337
338 void InitialKick(PS::ParticleSystem<FP>& sph_system, const PS::F64 dt){
339     for(PS::S32 i = 0 ; i < sph_system.getNumberOfParticleLocal() ; ++ i){
340         sph_system[i].vel_half = sph_system[i].vel + 0.5 * dt * sph_system[i
                ].acc;
341         sph_system[i].eng_half = sph_system[i].eng + 0.5 * dt * sph_system[i
                ].eng_dot;
342     }
343 }
344
345 void FullDrift(PS::ParticleSystem<FP>& sph_system, const PS::F64 dt){
346     // time becomes t + dt;
347     for(PS::S32 i = 0 ; i < sph_system.getNumberOfParticleLocal() ; ++ i){
348         sph_system[i].pos += dt * sph_system[i].vel_half;
349     }
350 }
351
352 void Predict(PS::ParticleSystem<FP>& sph_system, const PS::F64 dt){
353     for(PS::S32 i = 0 ; i < sph_system.getNumberOfParticleLocal() ; ++ i){
354         sph_system[i].vel += dt * sph_system[i].acc;
355         sph_system[i].eng += dt * sph_system[i].eng_dot;
356     }
357 }
358
359 void FinalKick(PS::ParticleSystem<FP>& sph_system, const PS::F64 dt){
360     for(PS::S32 i = 0 ; i < sph_system.getNumberOfParticleLocal() ; ++ i){
361         sph_system[i].vel = sph_system[i].vel_half + 0.5 * dt * sph_system[i
                ].acc;
362         sph_system[i].eng = sph_system[i].eng_half + 0.5 * dt * sph_system[i
                ].eng_dot;
363     }
364 }
365
366 void setPressure(PS::ParticleSystem<FP>& sph_system){
367     for(PS::S32 i = 0 ; i < sph_system.getNumberOfParticleLocal() ; ++ i){
368         sph_system[i].setPressure();
```

```
369     }
370 }
371
372 void CheckConservativeVariables(const PS::ParticleSystem<FP>& sph_system){
373     PS::F64vec Mom=0.0; // total momentum
374     PS::F64    Eng=0.0; // total enegry
375     for(PS::S32 i = 0; i < sph_system.getNumberOfParticleLocal(); ++ i){
376         Mom += sph_system[i].vel * sph_system[i].mass;
377         Eng += (sph_system[i].eng + 0.5 * sph_system[i].vel * sph_system[i].
                vel)
378                * sph_system[i].mass;
379     }
380     Eng = PS::Comm::getSum(Eng);
381     Mom = PS::Comm::getSum(Mom);
382      if(PS::Comm::getRank() == 0){
383          printf("%.16e\n", Eng);
384          printf("%.16e\n", Mom.x);
385          printf("%.16e\n", Mom.y);
386          printf("%.16e\n", Mom.z);
387      }
388 }
389
390 int main(int argc, char* argv[]){
391     // Initialize FDPS
392     PS::Initialize(argc, argv);
393     // Make a directory
394     char dir_name[1024];
395     sprintf(dir_name,"./result");
396     makeOutputDirectory(dir_name);
397     // Display # of MPI processes and threads
398     const PS::S32 n_proc = PS::Comm::getNumberOfProc();
399     const PS::S32 my_rank = PS::Comm::getRank();
400     const PS::S32 n_thread = PS::Comm::getNumberOfThread();
401     if (my_rank == 0) {
402         std::cout << "=========================================" << std::
                endl
403                   << "␣This␣is␣a␣sample␣program␣of␣"              << std::
                      endl
404                   << "␣Smoothed␣Particle␣Hydrodynamics␣on␣FDPS!"   << std::
                      endl
405                   << "␣#␣of␣processes␣is␣" << n_proc               << std::
                      endl
406                   << "␣#␣of␣thread␣is␣␣␣␣" << n_thread             << std::
                      endl
407                   << "=========================================" << std::
                      endl;
408     }
409     // Make an instance of ParticleSystem and initialize it
410     PS::ParticleSystem<FP> sph_system;
411     sph_system.initialize();
412     // Define local variables
413     PS::F64 dt, end_time;
414     boundary box;
415     // Make an initial condition and initialize the particle system
416     SetupIC(sph_system, &end_time, &box);
```

```
417     Initialize(sph_system);
418     // Make an instance of DomainInfo and initialize it
419     PS::DomainInfo dinfo;
420     dinfo.initialize();
421     // Set the boundary condition
422     dinfo.setBoundaryCondition(PS::BOUNDARY_CONDITION_PERIODIC_XYZ);
423     dinfo.setPosRootDomain(PS::F64vec(0.0, 0.0, 0.0),
424                            PS::F64vec(box.x, box.y, box.z));
425     // Perform domain decomposition
426     dinfo.decomposeDomainAll(sph_system);
427     // Exchange the SPH particles between the (MPI) processes
428     sph_system.exchangeParticle(dinfo);
429     PS::S32 n_loc = sph_system.getNumberOfParticleLocal();
430     // Make two tree structures
431     // (one is for the density calculation and
432     //  another is for the force calculation.)
433     PS::TreeForForceShort<Dens, EP, EP>::Gather dens_tree;
434     dens_tree.initialize(n_loc);
435
436     PS::TreeForForceShort<Hydro, EP, EP>::Symmetry hydr_tree;
437     hydr_tree.initialize(n_loc);
438     // Compute density, pressure, acceleration due to pressure gradient
439     dens_tree.calcForceAllAndWriteBack(CalcDensity(), sph_system, dinfo);
440     setPressure(sph_system);
441     hydr_tree.calcForceAllAndWriteBack(CalcHydroForce(), sph_system, dinfo)
            ;
442     // Get timestep
443     dt = getTimeStepGlobal(sph_system);
444     // Main loop for time integration
445     PS::S32 step = 0;
446     for(PS::F64 time = 0 ; time < end_time ; time += dt, ++ step){
447        // Leap frog: Initial Kick & Full Drift
448        InitialKick(sph_system, dt);
449        FullDrift(sph_system, dt);
450        // Adjust the positions of the SPH particles that run over
451        // the computational boundaries.
452        sph_system.adjustPositionIntoRootDomain(dinfo);
453        // Leap frog: Predict
454        Predict(sph_system, dt);
455        // Perform domain decomposition again
456        dinfo.decomposeDomainAll(sph_system);
457        // Exchange the SPH particles between the (MPI) processes
458        sph_system.exchangeParticle(dinfo);
459        // Compute density, pressure, acceleration due to pressure gradient
460        dens_tree.calcForceAllAndWriteBack(CalcDensity(), sph_system, dinfo)
              ;
461        setPressure(sph_system);
462        hydr_tree.calcForceAllAndWriteBack(CalcHydroForce(), sph_system,
              dinfo);
463        // Get a new timestep
464        dt = getTimeStepGlobal(sph_system);
465        // Leap frog: Final Kick
466        FinalKick(sph_system, dt);
467        // Output result files
468        if(step % OUTPUT_INTERVAL == 0){
```

```
469             FileHeader header;
470             header.time = time;
471             header.Nbody = sph_system.getNumberOfParticleGlobal();
472             char filename[256];
473             sprintf(filename, "result/%04d.txt", step);
474             sph_system.writeParticleAscii(filename, header);
475             if (my_rank == 0){
476                std::cout << "================================" << std::endl;
477                std::cout << "output␣" << filename << "." << std::endl;
478                std::cout << "================================" << std::endl;
479             }
480          }
481          // Output information to STDOUT
482          if (my_rank == 0){
483             std::cout << "================================" << std::endl;
484             std::cout << "time␣=␣" << time << std::endl;
485             std::cout << "step␣=␣" << step << std::endl;
486             std::cout << "================================" << std::endl;
487          }
488          CheckConservativeVariables(sph_system);
489       }
490       // Finalize FDPS
491       PS::Finalize();
492       return 0;
493    }
```

6 Extensions

6.1 P^3M code

In this section, we explain the usage of a FDPS extension "Particle Mesh" (hereafter PM) using a sample program for P^3M(Particle-Particle-Particle-Mesh) method. The sample code calculates the crystal energy of sodium chloride (NaCl) crystal using the P^3M method and compares the result with the analytical solution. In the P^3M method, the calculation of force and potential energy is performed by splitting into Particle-Particle(PP) part and Particle-Mesh(PM) part. In this sample code, the PP part is calculated by using FDPS standard features and the PM part is computed by using a FDPS extension "PM". Note that the detail of the extension "PM" is described in § 9.2 of the specification of FDPS and please see it for detail.

6.1.1 Location of sample code and working directory

The sample code is placed at `$(FDPS)/sample/c++/p3m`. Change the current directory to there.

```
$ cd (FDPS)/sample/c++/p3m
```

The sample code consists of `main.cpp` and a Makefile for GCC, `Makefile`.

6.1.2 User-defined types

In this section, we describe classes that you need to define in order to perform P^3M calculation using FDPS.

6.1.2.1 FullParticle type

You must define a FullParticle type. Listing 33 shows the implementation of FullParticle type in the sample code. FullParticle type must have all physical quantities required to perform a calculation with P^3M method. It must have the following member functions:

`getCharge()`
: required for FDPS to get the charge of particles

`getChargeParticleMesh()`
: required for the PM module of FDPS to get the charge of particles

`getPos()`
: required for FDPS to get the position of particles

`getRSearch()`
: required for FDPS to get the cutoff radius

`setPos()`
: required for FDPS to write the positions of particles recorded in FullParticle object

`copyFromForce()`
: required for FDPS to copy data form Force object

`copyFromForceParticleMesh()`
: required for the PM module to write the result of Force calculation to FullParticle object

Note that `copyFromForce()` and `copyFromForceParticleMesh()` are empty functions in this sample code. This is because the sample code explicitly copy data from Force objects to FullParticle object using APIs such as `getForce()` (explained later).

Listing 33: FullParticle type

```
1  class Nbody_FP
2  {
3     public:
4        PS::S64 id;
5        PS::F64 m;
6        PS::F64 rc;
7        PS::F64vec x;
8        PS::F64vec v,v_half;
9        PS::F64vec agrv;
10       PS::F64 pot;
11       // Member functions required by FDPS
12       PS::F64 getCharge() const {
13          return m;
14       };
15       PS::F64 getChargeParticleMesh() const {
16          return m;
17       };
18       PS::F64vec getPos() const {
19          return x;
20       };
21       PS::F64 getRSearch() const {
22          return rc;
23       };
24       void setPos(const PS::F64vec& x) {
25          this->x = x;
26       };
27       void copyFromForce(const Nbody_PP_Results& result) {};
28       void copyFromForceParticleMesh(const PS::F64 apm) {};
29 };
```

6.1.2.2 EssentialParticleI type

You must define a EssentialParticleI type. EssentialParticleI type must have member variables that store all physical quantities necessary for an i particle to perform the PP part of the Force calculation. In the sample code, it is also used as EssentialParticleJ type. Therefore, it should have member variables that store all physical quantities necessary for a j particle to perform the PP part of the Force calculation. Listing 34 shows the implementation of EssentialParticleI type in the sample code. EssentialParticleI type needs to have member function `copyFromFP()` to copy data from FullParticle object described above. In addition, it must have `getCharge()` (returns the charges of particles), `getPos()` (returns the positions of particles), `getRSearch()` (returns the cutoff radius of particles), and `setPos()` (sets the positions of particles).

Listing 34: EssentialParticleI type

```
1  class Nbody_EP
2  {
```

```
3     public:
4        PS::S64 id;
5        PS::F64 m;
6        PS::F64 rc;
7        PS::F64vec x;
8        // Member functions required by FDPS
9        PS::F64 getCharge() const {
10          return m;
11       };
12       PS::F64vec getPos() const {
13          return x;
14       };
15       PS::F64 getRSearch() const {
16          return rc;
17       };
18       void setPos(const PS::F64vec& x) {
19          this->x = x;
20       };
21       void copyFromFP(const Nbody_FP& FP) {
22          id  = FP.id;
23          m   = FP.m;
24          rc  = FP.rc;
25          x   = FP.x;
26       };
27 };
```

6.1.2.3 Force type

You must define a Force type. Force type must have member variables that store the results of the PP part of the Force calculation. Listing 35 shows the implementation of Force type in this sample code. Because we consider Coulomb interaction only, one Force type is defined. Force type needs to have member function `clear()` to zero-clear or initialize member variables that store the results of accumulation operation.

Listing 35: Force type

```
1  class Nbody_PP_Results
2  {
3     public:
4        PS::F64 pot;
5        PS::F64vec agrv;
6        void clear() {
7           pot  = 0.0;
8           agrv = 0.0;
9        }
10 };
```

6.1.2.4 calcForceEpEp

You must define an interaction function calcForceEpEp. calcForceEpEp must contain actual code for the PP part of the Force calculation and must be implemented as `void` function or functor (function object) . Its arguments is an array of EssentialParticleI objects, the

number of EssentialParticleI objects, an array of EssentialParticleJ objects, the number of EssentialParticleJ objects, and an array of Force objects. Listing 36 shows the implementation of calcForceEpEp in this sample code. In the code, it is implemented as a functor.

Listing 36: Interaction function calcForceEpEp

```
1  class Calc_force_ep_ep{
2     public:
3        void operator () (const Nbody_EP* const ep_i,
4                          const PS::S32 Nip,
5                          const Nbody_EP* const ep_j,
6                          const PS::S32 Njp,
7                          Nbody_PP_Results* const result) {
8           for (PS::S32 i=0; i<Nip; i++) {
9              for (PS::S32 j=0; j<Njp; j++) {
10                PS::F64vec dx = ep_i[i].x - ep_j[j].x;
11                PS::F64 rij = std::sqrt(dx * dx);
12                if ((ep_i[i].id == ep_j[j].id) && (rij == 0.0)) continue;
13                PS::F64 rinv = 1.0/rij;
14                PS::F64 rinv3 = rinv*rinv*rinv;
15                PS::F64 xi = 2.0*rij/ep_i[i].rc;
16                result[i].pot  += ep_j[j].m * S2_pcut(xi) * rinv;
17                result[i].agrv += ep_j[j].m * S2_fcut(xi) * rinv3 * dx;
18             }
19             //* Self-interaction term
20             result[i].pot -= ep_i[i].m * (208.0/(70.0*ep_i[i].rc));
21          }
22
23       };
24 };
```

The PP part in the P^{3}M method is a two-body interaction with cutoff (i.e. the interaction is truncated if the distance between the particles is larger than the cutoff distance). Hence, cutoff functions (`S2_pcut()`, `S2_fcut()`) appears in the calculations of potential and acceleration. These cutoff functions must be the ones that are constructed assuming that the particle shape function is $S2(r)$, which is introduced by Hockney & Eastwood (1988)(Eq.(8.3)) and takes the form of

$$S2(r) = \begin{cases} \dfrac{48}{\pi a^4}\left(\dfrac{a}{2} - r\right) & r < a/2, \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

where r is the distance from the center of a particle, a is the scale length of the shape function. When assuming this shape function, the charge density distribution due to a particle, $\rho(r)$, is expressed as $\rho(r) = q\,S2(r)$, where q is the charge of the particle. Thus, $S2(r)$ shape function gives linear density distribution. The reason why we have to use the cutoff functions that correspond to $S2(r)$ shape function is that the cutoff functions used in the PM part also assumes the $S2(r)$ shape function (the cutoff functions in the PM and PP parts should be consistent with each other).

The cutoff functions must be defined by a user. Possible implementations for `S2_pcut()` and `S2_fcut()` are given at the beginning of the sample code (see the lines 22-72 in `main.cpp`). In these examples, we used Eqs.(8-72) and (8-75) in Hockney & Eastwood

(1988) and we define them such that the PP interaction takes of the form:

$$\Phi_{\rm PP}(\boldsymbol{r}) = \frac{m}{|\boldsymbol{r}-\boldsymbol{r}'|}\texttt{S2_pcut}(\xi) \tag{2}$$

$$\boldsymbol{f}_{\rm PP}(\boldsymbol{r}) = \frac{m(\boldsymbol{r}-\boldsymbol{r}')}{|\boldsymbol{r}-\boldsymbol{r}'|^3}\texttt{S2_fcut}(\xi) \tag{3}$$

where $\xi = 2|\boldsymbol{r}-\boldsymbol{r}'|/a$. In this sample code, a is expressed as a variable `rc` .

As is clear from Eq.(8-75) in Hockney & Eastwood (1988), the mesh potential ϕ^m has a finite value at $r = 0$ (we omit a factor $1/4\pi\varepsilon_0$ here):

$$\phi^m(0) = \frac{208}{70a} \tag{4}$$

This term is taken into account the last line in the i-particle loop:

```
result[i].pot -= ep_i[i].m * (208.0/(70.0*ep_i[i].rc));
```

Note that this term is necessary to match the numerical result with the analytical solution.

6.1.2.5 calcForceEpSp

You must define an interaction function calcForceEpSp[2)]. calcForceEpSp must contain actual code for particle-superparticle interaction and must be implemented as `void` function or functor (function object) . Its arguments is an array of EssentialParticleI objects, the number of EssentialParticleI objects, an array of SuperParticleJ objects, the number of SuperParticleJ objects, and an array of Force objects. Listing 37 shows the implementation of calcForceEpSp in the sample code.

Listing 37: Interaction function calcForceEpSp

```
class Calc_force_ep_sp{
   public:
      void operator () (const Nbody_EP* const ep_i,
                        const PS::S32 Nip,
                        const PS::SPJMonopoleCutoff* const ep_j,
                        const PS::S32 Njp,
                        Nbody_PP_Results* const result) {
         for (PS::S32 i=0; i<Nip; i++) {
            for (PS::S32 j=0; j<Njp; j++) {
               PS::F64vec dx = ep_i[i].x - ep_j[j].pos;
               PS::F64 rij = std::sqrt(dx * dx);
               PS::F64 rinv = 1.0/rij;
               PS::F64 rinv3 = rinv*rinv*rinv;
               PS::F64 xi = 2.0*rij/ep_i[i].rc;
               result[i].pot  += ep_j[j].mass * S2_pcut(xi) * rinv;
               result[i].agrv += ep_j[j].mass * S2_fcut(xi) * rinv3 * dx;
            }
```

[2)] As describe at the beginning of this section, the sample code uses P^{3}M for the calculation of interaction. In order to realize it using FDPS, we perform the calculation of interaction with the opening angle criterion θ of 0. Hence, particle-superparticle interaction should not occur. However, API `calcForceAllAndWriteBack` requires a function pointer of a `void` function that calculates particle-superparticle interaction. Therefore, we defined calcForceEpSp here.

```
18            }
19
20        };
21 };
```

6.1.3 Main body of the sample code

In this section, we explain the main body of the sample code. Before going into details, we first give a simple explanation about the content and the structure of the sample code. As described in § 6.1, this code computes the crystal energy of NaCl crystal using the P^{3}M method and compares the result with the analytical solution. The NaCl crystal is expressed as an uniform grid of particles in this sample code. Na and Cl are placed in the staggered layout. Particles corresponding to Na has a positive charge, while those corresponding to Cl has a negative charge. We place a crystal expressed as an grid of charged particles into a periodic computational box of the sizes $[0, 1)^3$ and calculates the crystal energy. The computational accuracy of the crystal energy should depend on the number of particles and the configuration of particles (to the grid used in the PM calculation). Hence, in the sample code, we measure the relative energy errors for a different set of these parameters and output the result of the comparisons into a file.

The structure of the sample code is as follows:

(1) Create and initialize FDPS objects
(2) Create a NaCl crystal for given number of particles and configuration (in `void` function `NaCl_IC()`)
(3) Compute the potential energy of each particle by the P^{3}M method (in `void` function `Nbody_objs.calc_gravity()`)
(4) Compute the total energy of the crystal and compare it with the analytical solution (`void` function `calc_energy_error()`)
(5) Repeat (2)-(4)

In the following, we explain in detail each steps described above.

6.1.4 Include header files

In order to use the FDPS extension "PM", we must include the header files `particle_mesh.hpp` and `param_fdps.h` as well as `particle_simulator.hpp` (the last one is needed to use FDPS standard features). In addition, `param.h` is included because the sample code accesses a non-public constant `CUTOFF_RADIUS`.

Listing 38: Include FDPS's header files

```
1 #include <particle_simulator.hpp>
2 #include <particle_mesh.hpp>
3 #include <param.h>
4 #include <param_fdps.h>
```

6.1.4.1 Initialization and Termination of FDPS

First, you must initialize FDPS by the following code.

Listing 39: Initialization of FDPS

```
1 PS::Initialize(argc, argv);
```

Once started, FDPS should be terminated explicitly. In this sample, FDPS is terminated just before the termination of the program. Hence, you need to write the following code at the end of the main function.

Listing 40: Termination of FDPS

```
1 PS::Finalize();
```

6.1.4.2 Creation and initialization of FDPS objects

After the initialization of FDPS, a user need to create the objects used to talk to FDPS. In this section, we describe how to create and initialize these objects.

6.1.4.2.1 Creation of necessary FDPS objects

In the calculation using the P^3M method, we must create objects of the ParticleSystem class and the DomainInfo class. In addition, objects of the TreeForForceLong class and the ParticleMesh class are need to calculate the PP and PM parts of the force calculation. In this sample code, these objects are grouped into `Nbody_Objects` class. The following code is the implementation of the `Nbody_Objects` class.

Listing 41: `Nbody_Objects` class

```
1 class Nbody_Objects {
2    public:
3       PS::ParticleSystem<Nbody_FP> system;
4       PS::DomainInfo dinfo;
5       PS::TreeForForceLong<Nbody_PP_Results, Nbody_EP, Nbody_EP>::
              MonopoleWithCutoff pp_tree;
6       PS::PM::ParticleMesh pm;
7 }
```

In this sample, a object of the `Nbody_Objects` class is created as a local variable in the main function:

Listing 42: Creation of a `Nbody_Objects`-class object

```
1 Nbody_Objects Nbody_objs;
```

6.1.4.2.2 Initialization of FDPS objects

After the creation of FDPS objects, you must initialize these objects before you use them in a user code. In the following, we explain how to initialize each object.

(i) Initialization of a ParticleSystem object A ParticleSystem object is initialized as follows:

Listing 43: Initialization of a ParticleSystem object

```
1 Nbody_objs.system.initialize();
```

This is done in the main function in the sample code.

(ii) Initialization of a DomainInfo object A DomainInfo object is initialized as follows:

Listing 44: Initialization of a DomainInfo object

```
1 Nbody_objs.dinfo.initialize();
```

This is done in the main function in the sample code.

After the initialization, you need to specify the boundary condition and the size of the simulation box through APIs `setBoundaryCondition` and `setPosRootDomain` . In the sample code, these procedures are performed in `void` function `NaCl_IC()` that sets up the distribution of particles:

```
1 dinfo.setBoundaryCondition(PS::BOUNDARY_CONDITION_PERIODIC_XYZ);
2 dinfo.setPosRootDomain(PS::F64vec(0.0,0.0,0.0),
3                        PS::F64vec(1.0,1.0,1.0));
```

(iii) Initialization of a TreeForForceLong object A TreeForForceLong object is initialized by API `initialize` :

Listing 45: Initialization of a TreeForForceLong object

```
1 void init_tree() {
2    PS::S32 numPtclLobal = system.getNumberOfParticleLobal();
3    PS::U64 ntot = 3 * numPtclLobal;
4    pp_tree.initialize(ntot,0.0);
5 };
```

You need to give a rough number of particles to this API as the first argument. Here, we set three times the number of local particles at the time of calling. The second argument of this API is an optional argument and represents the opening angle criterion θ for the tree method. In the sample, we do not use the tree method in the PP part of the force calculation. Therefore, we set $\theta = 0$.

In this sample code, a TreeForForceLong object is initialized within the function `Nbody_objs.init_tree()` (see the main function).

```
1 if (is_tree_initialized == false) {
2    Nbody_objs.init_tree();
3    is_tree_initialized = true;
4 }
```

where the `if` statement above is necessary because the initialization should be done only once in the program (otherwise, the program will fail).

(iv) Initialization of a ParticleMesh object No explicit initialization is needed.

6.1.4.3 Generation of a distribution of particles

In this section, we explain `void` function `NaCl_IC` that generates a distribution of particles, and FDPS APIs called within it. Given the number of particles per one space dimension and the position of the particle that is nearest to the origin $(0, 0, 0)$, `void` function `NaCl_IC` makes a three-dimensional uniform grid of particles. These parameters are specified through an object of class `Crystal_Parameters` , `NaCl_params`:

```
1  class Crystal_Parameters
2  {
3     public:
4        PS::S32 numPtcl_per_side;
5        PS::F64vec pos_vertex;
6  };
7  /* In main function */
8  Crystal_Parameters NaCl_params;
9  NaCl_IC(Nbody_objs.system,
10         Nbody_objs.dinfo,
11         NaCl_params);
```

In the first half of `void` function `NaCl_IC` , it makes an uniform grid of particles based on the value of `NaCl_params`. In this process, we scale the particle charge m to satisfy the relation

$$\frac{2Nm^2}{R_0} = 1, \tag{5}$$

where N is the total number of molecules (the total number of atomic particles is $2N$) and R_0 is the distance to the nearest particle. This scaling is introduced just for convenience: The crystal energy can be written analytically as

$$E = -\frac{N\alpha m^2}{R_0}, \tag{6}$$

where α is the Madelung constant and $\alpha \approx 1.747565$ for the NaCl crystal (e.g. see Kittel (2004) "Introduction to Solid State Physics"). Thus, the crystal energy depends on the total number of particles. This is inconvenient when comparing the calculation result with the analytical solution. By scaling the particle charge as described above, the crystal energy becomes independent from N.

After generating a particle distribution, this function performs domain decomposition and particle exchange using FDPS APIs. In the following, we explain these APIs.

6.1.4.3.1 Domain Decomposition

API `decomposeDomainAll` of the DomainInfo object is used to perform domain decomposition based on the current distribution of particles:

Listing 46: Domain Decomposition

```
1  dinfo.decomposeDomainAll(system);
```

Note that this API needs a ParticleSystem object as the argument to get the information of particle distribution.

6.1.4.3.2 Particle Exchange

API `exchangeParticle` of the ParticleSystem object is used to exchange particles based on the current decomposed domains:

Listing 47: Particle Exchange

```
1 system.exchangeParticle(dinfo);
```

Note that this API needs a DomainInfo object as the argument to get the domain information.

6.1.4.4 Interaction Calculation

After these procedures are completed, we must perform the interaction calculation. In the sample code, it is performed in the main function by calling the function `Nbody_objs.calc_gravity()`:

Listing 48: Execution of interaction calculation

```
1 Nbody_objs.calc_gravity();
```

The function `Nbody_objs.calc_gravity()` consists of (i) zero-clear of potential energy and acceleration of each particle, (ii) calculation of the PM part, and (iii) calculation of the PP part:

Listing 49: Interaction calculation

```
1  void calc_gravity() {
2     //* Local variables
3     PS::S32 numPtclLocal = system.getNumberOfParticleLocal();
4
5     //* Reset potential and accelerations
6     for (PS::S32 i=0; i<numPtclLocal; i++) {
7        system[i].pot  = 0.0;
8        system[i].agrv = 0.0;
9     }
10
11    //=================
12    //* [1] PM part
13    //=================
14    pm.setDomainInfoParticleMesh(dinfo);
15    pm.setParticleParticleMesh(system);
16    pm.calcMeshForceOnly();
17    for (PS::S32 i=0; i<numPtclLocal; i++) {
18       PS::F32vec x32 = system[i].x;
19       system[i].pot  -= pm.getPotential(x32);
20       system[i].agrv -= pm.getForce(x32);
21    }
22
23    //=================
24    //* [2] PP part
25    //=================
26    pp_tree.calcForceAll(Calc_force_ep_ep(),
27                         Calc_force_ep_sp(),
28                         system, dinfo);
29    for (PS::S32 i=0; i<numPtclLocal; i++) {
```

```
30        Nbody_PP_Results result = pp_tree.getForce(i);
31        system[i].pot  += result.pot;
32        system[i].agrv += result.agrv;
33     }
34  };
```

The code shown below is the PM part of the Force calculation (hereafter PM calculation). In order to perform the PM calculation, the ParticleMesh object `pm` must have information about the domain and the particles in advance. Therefore, `setDomainInfoParticleMesh` and `setParticleParticleMesh` methods are first called to give these information to the object `pm`. Now, the object `pm` are ready to perform the PM calculation. In the sample code, the PM calculation is performed by `calcMeshForceOnly` method. Then, `getPotential` and `getForce` methods are called to obtain the potential and acceleration at the particle position. They are stored to the FullParticle type object `system`. ***Note that the accumulation operation is done by the operator*** `-=`. The reason why we use `-=` instead of `+=` is that the FDPS extension "PM" computes the potential energy assuming gravity. In other words, the FDPS extension "PM" treats a charge with $m(> 0)$ creates negative potential. Hence, ***we need to invert the signs of potential energy and acceleration*** in order to use the FDPS extension "PM" for the Coulomb interaction calculation.

Listing 50: PM part of Force calculation

```
1  pm.setDomainInfoParticleMesh(dinfo);
2  pm.setParticleParticleMesh(system);
3  pm.calcMeshForceOnly();
4  for (PS::S32 i=0; i<numPtclLocal; i++) {
5     PS::F32vec x32 = system[i].x;
6     system[i].pot  -= pm.getPotential(x32);
7     system[i].agrv -= pm.getForce(x32);
8  }
```

Next, we shows the PP part of the Force calculation in the following. The `calcForceAll` method of the TreeForForceLong class is used to calculation the PP part (we do not use the `calcForceAllAndWriteBack` method because this method zero-clears the results of the PM calculation stored the FullParticle object). Then, the `getForce` method is used to obtain the potential energy and the acceleration at the particle position and they are accumulated to the FullParticle-type object `system`.

Listing 51: PP part of Force calculation

```
1  pp_tree.calcForceAll(Calc_force_ep_ep(),
2                       Calc_force_ep_sp(),
3                       system, dinfo);
4  for (PS::S32 i=0; i<numPtclLocal; i++) {
5     Nbody_PP_Results result = pp_tree.getForce(i);
6     system[i].pot  += result.pot;
7      system[i].agrv += result.agrv;
8  }
```

6.1.4.5 Calculation of relative energy error

The relative error of the crystal energy is computed in the function `calc_energy_error()`, where we assume that the analytical solution is $E_0 \equiv 2E = -1.7475645946332$, which is numerically evaluated by the PM^3(Particle-Mesh Multipole Method).

6.1.5 Compile

Before compiling your program, you need to install the FFTW(Fast Fourier Transform in the West) library. Then, edit the file `Makefile` in the working directory to set the PATHs of the locations of FFTW and FDPS to the variables `FFTW_LOC` and `FDPS_LOC`. After that, run `make`.

```
$ make
```

The execution file `p3m.x` will be created in the directory `work` if the compilation is succeeded.

6.1.6 Run

You must run your program using MPI with the number of MPI processes is equal to or greater than 2, because of the specification of FDPS extensions. Therefore, you should run the following command:

```
$ MPIRUN -np NPROC ./p3m.x
```

where "MPIRUN" represents the command to run your program using MPI such as `mpirun` or `mpiexec`, and "NPROC" is the number of MPI processes.

6.1.7 Check the result

After the program ended, a file that records the relative error of the crystal energy is output in the directory `work`. Figure 3 shows the dependency of the relative error on the number of particles used.

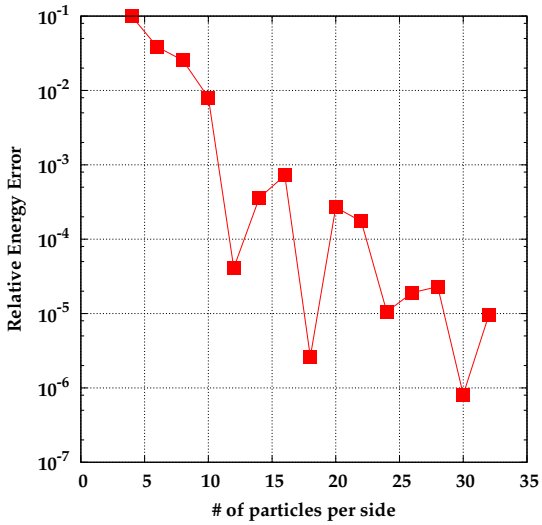


Figure 3: The relative error of the crystal energy as a function of the number of particles per side, where we assume that the number of the PM grids is 16^3 and the cutoff radius is 3/16.

6.2 TreePM code

In this section, we explain the usage of a FDPS extension "Particle Mesh" (hereafter PM) using a sample program for TreePM(Tree-Particle-Mesh) method. This sample code performs cosmological N-body simulation using the TreePM method. In the TreePM method, the calculation of gravity is performed by splitting into the PP part and the PM part as in the P^3M method. Therefore, functions of FDPS used in the sample code is almost the same as the sample code for the P^3M method. The difference between the two is that the PP part is computed by the Tree method in the TreePM method while the P^3M method uses the direct summation for the PP part.

6.2.1 Location of the sample code and the working directory

The sample code is placed at `$(FDPS)/sample/c++/treepm`. Change the current directory to there. As shown below, the sample code consists of several source files, of which the main body of the program is implemented in the files `treepm.hpp` and `treepm.cpp`.

```
$ cd (FDPS)/sample/c++/treepm
$ ls | awk '{print $0}'
IC/
Makefile
README_en.txt
README_ja.txt
constants.hpp
cosmology.hpp
fig/
make_directory.c
param_file_for_test.txt
prototype.h
result/
run_param.hpp
test.py*
timing.c
treepm.cpp
treepm.hpp
utils/
```

6.2.2 Required header files

In order to use the FDPS extension "PM", we must include the header file `particle_mesh.hpp` as well as `particle_simulator.hpp`. These are described in the file `treepm.cpp`:

Listing 52: Include FDPS

```
1 #include <particle_simulator.hpp>
2 #include <particle_mesh.hpp>
```

6.2.3 User-defined classes

In this section, we describe classes that you need to define in order to perform TreePM calculation using FDPS.

6.2.3.1 FullParticle type

You must define a FullParticle type. FullParticle type must have all physical quantities required to perform a calculation with the TreePM method. Listing 53 shows the implementation of FullParticle type in the sample code. In the code, it has member variables necessary for usual N-body simulations (`id`, `mass`, `eps`, `pos`, `vel`, `acc`). In addition, it has the following member variables: `acc_pm` (stores the acceleration of the PM part), `H0` (stores the Hubble constant)、and `Lbnd` (stores the size of the simulation box in the unit of Mpc h^{-1}). FullParticle type must have the following member functions to use the FDPS standard functions and the FDPS extension "PM":

`getCharge()`
: required for FDPS to get the mass of particle

`getChargeParticleMesh()`
: required for the PM module to get the mass of particle

`getPos()`
: required for FDPS to get the position of particle

`getRSearch()`
: required for FDPS to get the cutoff radius

`setPos()`
: required for FDPS to write the position of particle recorded in FullParticle object

`copyFromForce()`
: required for FDPS to copy data from a Force object

`copyFromForceParticleMesh()`
: required for the PM module to write the result of Force calculation to FullParticle object

In addition, the sample uses file I/O functions of FDPS, which requires a user to define the following member functions:

- `readBinary()`
- `writeBinary()`

Note that the use of these file I/O functions is not necessary and user-defined I/O can be used.

Listing 53: FullParticle type

```
1 class FPtreepm {
2 private:
3     template<class T>
4     T reverseEndian(T value){
5         char * first = reinterpret_cast<char*>(&value);
6         char * last = first + sizeof(T);
7         std::reverse(first, last);
8         return value;
9     }
```

```
10 public:
11     PS::S64    id;
12     PS::F32    mass;
13     PS::F32    eps;
14     PS::F64vec pos;
15     PS::F64vec vel;
16     PS::F64vec acc;
17     PS::F64vec acc_pm;
18
19     //static PS::F64vec low_boundary;
20     //static PS::F64vec high_boundary;
21     //static PS::F64 unit_l;
22     //static PS::F64 unit_m;
23     static PS::F64 H0;
24     static PS::F64 Lbnd;
25
26     PS::F64vec getPos() const {
27         return pos;
28     }
29
30     PS::F64 getCharge() const {
31         return mass;
32     }
33
34     void copyFromForce(const Result_treepm & force) {
35         this->acc = force.acc;
36     }
37
38     PS::F64 getRSearch() const {
39         PS::F64 rcut = 3.0/SIZE_OF_MESH;
40         return rcut;
41     }
42
43     void setPos(const PS::F64vec pos_new) {
44         this->pos = pos_new;
45     }
46
47     PS::F64 getChargeParticleMesh() const {
48         return this->mass;
49     }
50
51     void copyFromForceParticleMesh(const PS::F64vec & acc_pm) {
52         this->acc_pm = acc_pm;
53     }
54
55     /*
56     void writeParticleBinary(FILE *fp) {
57         int count;
58         count = 0;
59
60         count += fwrite(&mass,   sizeof(PS::F32),1,fp);
61         count += fwrite(&eps,    sizeof(PS::F32),1,fp);
62         count += fwrite(&pos[0], sizeof(PS::F64),1,fp);
63         count += fwrite(&pos[1], sizeof(PS::F64),1,fp);
64         count += fwrite(&pos[2], sizeof(PS::F64),1,fp);
```

```
65          count += fwrite(&vel[0], sizeof(PS::F64),1,fp);
66          count += fwrite(&vel[1], sizeof(PS::F64),1,fp);
67          count += fwrite(&vel[2], sizeof(PS::F64),1,fp);
68      }
69      */
70      /*
71      int readParticleBinary(FILE *fp) {
72          int count;
73          count = 0;
74
75          count += fread(&mass,   sizeof(PS::F32),1,fp);
76          count += fread(&eps,    sizeof(PS::F32),1,fp);
77          count += fread(&pos[0], sizeof(PS::F64),1,fp);
78          count += fread(&pos[1], sizeof(PS::F64),1,fp);
79          count += fread(&pos[2], sizeof(PS::F64),1,fp);
80          count += fread(&vel[0], sizeof(PS::F64),1,fp);
81          count += fread(&vel[1], sizeof(PS::F64),1,fp);
82          count += fread(&vel[2], sizeof(PS::F64),1,fp);
83
84          return count;
85      }
86      */
87
88      void writeParticleBinary(FILE *fp) {
89          PS::F32 x = pos[0];
90          PS::F32 y = pos[1];
91          PS::F32 z = pos[2];
92          PS::F32 vx = vel[0];
93          PS::F32 vy = vel[1];
94          PS::F32 vz = vel[2];
95          PS::S32 i = id;
96          PS::S32 m = mass;
97          fwrite(&x, sizeof(PS::F32),1,fp);
98          fwrite(&vx, sizeof(PS::F32),1,fp);
99          fwrite(&y, sizeof(PS::F32),1,fp);
100         fwrite(&vy, sizeof(PS::F32),1,fp);
101         fwrite(&z, sizeof(PS::F32),1,fp);
102         fwrite(&vz, sizeof(PS::F32),1,fp);
103         //fwrite(&mass,   sizeof(PS::F32),1,fp);
104         fwrite(&m,    sizeof(PS::F32),1,fp);
105         fwrite(&i,    sizeof(PS::F32),1,fp);
106         //fwrite(&id,   sizeof(PS::F32),1,fp);
107     }
108
109
110     // for API of FDPS
111     // in snapshot, L unit is Mpc/h, M unit is Msun, v unit is km/s
112     void readBinary(FILE *fp){
113         static PS::S32 ONE = 1;
114         static bool is_little_endian = *reinterpret_cast<char*>(&ONE) ==
                ONE;
115         static const PS::F64 Mpc_m = 3.08567e22; // unit is m
116         static const PS::F64 Mpc_km = 3.08567e19; // unit is km
117         static const PS::F64 Msun_kg = 1.9884e30; // unit is kg
118         static const PS::F64 G = 6.67428e-11; // m^3*kg^-1*s^-2
```

```
119         static const PS::F64 Cl = 1.0 / FPtreepm::Lbnd;
120         static const PS::F64 Cv = 1.0 / (FPtreepm::Lbnd * FPtreepm::H0);
121         static const PS::F64 Cm = 1.0 / (pow(Mpc_m*FPtreepm::Lbnd, 3.0) /
                pow(Mpc_km/FPtreepm::H0, 2.0) / G / Msun_kg);
122         PS::F32 x, y, z, vx, vy, vz, m;
123         PS::S32 i;
124         fread(&x, 4, 1, fp);
125         fread(&vx, 4, 1, fp);
126         fread(&y, 4, 1, fp);
127         fread(&vy, 4, 1, fp);
128         fread(&z, 4, 1, fp);
129         fread(&vz, 4, 1, fp);
130         fread(&m,  4, 1, fp);
131         fread(&i,  4, 1, fp);
132         if( is_little_endian){
133             pos.x = x * Cl;
134             pos.y = y * Cl;
135             pos.z = z * Cl;
136             vel.x = vx * Cv;
137             vel.y = vy * Cv;
138             vel.z = vz * Cv;
139             mass = m * Cm;
140             //mass = m / 1.524e17;
141             id = i;
142         }
143         else{
144             pos.x = reverseEndian(x) * Cl;
145             pos.y = reverseEndian(y) * Cl;
146             pos.z = reverseEndian(z) * Cl;
147             vel.x = reverseEndian(vx) * Cv;
148             vel.y = reverseEndian(vy) * Cv;
149             vel.z = reverseEndian(vz) * Cv;
150             mass = reverseEndian(m) * Cm;
151             //mass = reverseEndian(m) / 1.524e17;
152             id = reverseEndian(i);
153         }
154     }
155
156     // for API of FDPS
157     void writeBinary(FILE *fp){
158         static const PS::F64 Mpc_m = 3.08567e22; // unit is m
159         static const PS::F64 Mpc_km = 3.08567e19; // unit is km
160         static const PS::F64 Msun_kg = 1.9884e30; // unit is kg
161         static const PS::F64 G = 6.67428e-11; // m^3*kg^-1*s^-2
162         static const PS::F64 Cl = FPtreepm::Lbnd;
163         static const PS::F64 Cv = (FPtreepm::Lbnd * FPtreepm::H0);
164         static const PS::F64 Cm = (pow(Mpc_m*FPtreepm::Lbnd, 3.0) / pow(
                Mpc_km/FPtreepm::H0, 2.0) / G / Msun_kg);
165         PS::F32vec x = pos * Cl;
166         PS::F32vec v = vel * Cv;
167         PS::F32 m = mass * Cm;
168         PS::S32 i = id;
169         fwrite(&x.x,   sizeof(PS::F32), 1, fp);
170         fwrite(&v.x,   sizeof(PS::F32), 1, fp);
171         fwrite(&x.y,   sizeof(PS::F32), 1, fp);
```

```
172         fwrite(&v.y,    sizeof(PS::F32), 1, fp);
173         fwrite(&x.z,    sizeof(PS::F32), 1, fp);
174         fwrite(&v.z,    sizeof(PS::F32), 1, fp);
175         fwrite(&m,    sizeof(PS::F32), 1, fp);
176         fwrite(&i,    sizeof(PS::S32), 1, fp);
177     }
178
179     PS::F64 calcDtime(run_param &this_run) {
180     PS::F64 dtime_v, dtime_a, dtime;
181     PS::F64 vnorm, anorm;
182     vnorm = sqrt(SQR(this->vel))+TINY;
183     anorm = sqrt(SQR(this->acc+this->acc_pm))+TINY;
184
185     dtime_v = this->eps/vnorm;
186     dtime_a = sqrt(this->eps/anorm)*CUBE(this_run.anow);
187
188     dtime = fmin(0.5*dtime_v, dtime_a);
189
190     return dtime;
191   }
192 };
```

6.2.3.2 EssentialParticleI type

You must define a EssentialParticleI type and it must have all physical quantities as member variables that i particle should have. Listing 54 shows the implementation of EssentialParticleI type in the sample code. It must have member functions `copyFromFP()` (to copy data from a FullParticle object described above) and `getPos()` (to get the position of particle).

Listing 54: EssentialParticleI type

```
1  class EPItreepm {
2  public:
3    PS::S64    id;
4    PS::F32    eps;
5    PS::F64vec pos;
6
7    PS::F64vec getPos() const {
8      return this->pos;
9    }
10
11   void copyFromFP(const FPtreepm & fp) {
12     this->id = fp.id;
13     this->eps = fp.eps;
14     this->pos = fp.pos;
15   }
16
17 };
```

6.2.3.3 EssentialParticleJ type

You must define a EssentialParticleJ type and it must have all physical quantities as member variables that j particle should have when the PP part of Force calculation is performed. Note that it is possible to define EssentialParticleI type so that it operates as EssentialParticleJ type as in the sample code for P^3M method (see § 6.1). Listing 55 shows the implementation of EssentialParticleJ type in this sample code. EssentialParticleJ type should have the following member functions:

`getPos()`
: required for FDPS to get the position of particle

`getCharge()`
: required for FDPS to get the mass of particle

`copyFromFP()`
: required for FDPS to copy data from FullParticle type to EssentialParticleJ type

`getRSearch()`
: required for FDPS to get the cutoff radius

`setPos()`
: required for FDPS to write the position of particle

Listing 55: EssentialParticleJ type

```
1  class EPJtreepm {
2  public:
3    PS::S64    id;
4    PS::F64vec pos;
5    PS::F64    mass;
6    //  PS::F64    rcut;
7
8    PS::F64vec getPos() const {
9      return this->pos;
10   }
11
12   PS::F64 getCharge() const {
13     return this->mass;
14   }
15
16   void copyFromFP(const FPtreepm & fp) {
17     this->id = fp.id;
18     this->mass = fp.mass;
19     this->pos = fp.pos;
20   }
21
22   PS::F64 getRSearch() const {
23     PS::F64 rcut = 3.0/SIZE_OF_MESH;
24     return rcut;
25   }
26
27   void setPos(const PS::F64vec pos_new) {
28     this->pos = pos_new;
29   }
30 };
```

6.2.3.4 Force type

You must define a Force type and it must have all physical quantities that obtained as the results of the PP part of Force calculation. Listing 56 shows the implementation of Force type. It must have a member function `clear()` in order to initialize or zero-clear the member variables that stored the results of accumulation operations.

Listing 56: Force type

```
class Result_treepm {
public:
  PS::F32vec acc;
  PS::F32    pot;

  void clear() {
    acc = 0.0;
    pot = 0.0;
  }
};
```

6.2.3.5 calcForceEpEp type

You must define a calcForceEpEp type and it must contain actual code for the PP part of Force calculation. Listing 57 shows the implementation of calcForceEpEp type. In the sample code, calcForceEpEp type is implemented as a template function. Depending on the value of the macro `ENABLE_PHANTOM_GRAPE_X86`, which determines whether to use the Phantom-GRAPE library, a different template function is used. In both cases, the arguments of the functions is an array of EssentialParticleI variables, the number of EssentialParticleI variables, an array of EssentialParticleJ variables, the number of EssentialParticleJ variables, an array of Force variables.

Listing 57: calcForceEpEp type

```
template <class TPJ>
class calc_pp_force {
public:
    void operator () (EPItreepm *iptcl,
                      const PS::S32 ni,
                      TPJ *jptcl,
                      const PS::S32 nj,
                      Result_treepm *ppforce) {
        for (PS::S32 i=0;i < ni;i++) {
            PS::F64 eps2 = SQR(iptcl[i].eps);
            for (PS::S32 j=0; j < nj;j++) {
                PS::F64vec dr = iptcl[i].pos - jptcl[j].pos;
                PS::F64 rsq = dr*dr;
                PS::F64 rad = sqrt(rsq+eps2);
                PS::F64 gfact = gfactor_S2(rad, 3.0/SIZE_OF_MESH);
                PS::F64 rinv  = 1.0/rad;
                PS::F64 mrinv3 = jptcl[j].mass*CUBE(rinv);
                ppforce[i].acc -= dr*gfact*mrinv3;
            }
        }
    }
```

```
22 };
```

The PP part of the TreePM method is a two-body interaction with cutoff as in the P^3M method. Hence, a cutoff function is involved in the calculation of gravitational acceleration. As explained in § 6.1.2.4, the cutoff function must be the one that is constructed assuming that the particle shape function is $S2(r)$ (Hockney & Eastwood 1988). The cutoff function is implemented as the function `gfactor_S2()` for the case where the Phantom-GRAPE library is not used. In using the Phantom-GRAPE library, you do not have to implement the cutoff function because the library computes the interaction taking into account cutoff. In this case, you must call the library's API `pg5_gen_s2_force_table()` before the Force calculation in order to give the value of the cutoff radius to the library. In the sample code, the call of the API is performed at the main function:

```
1 #ifdef ENABLE_PHANTOM_GRAPE_X86
2     //g5_open();
3     pg5_gen_s2_force_table(EPS_FOR_PP, 3.0/SIZE_OF_MESH);
4 #endif
```

6.2.4 Main body of the program

In this section, we explain in detail the main body of the program. Before going into details, we first give a simple explanation about the content and the structure of the sample code. As described in the beginning of § 6.2, this code performs a cosmological N-body simulation using the TreePM method. The code supports three different types of initial condition files:

(a) Initial condition files used in the Santa Barbara Cluster Comparison Test (Frenk et al.[1999, ApJ, 525, 554]). These initial condition files are available at https://v2.jmlab.jp/owncloud/index.php/s/XnzvW5XAYwfqZYQ?path=%2Fsb, where `ic_sb128.tar` for $N = 128^3$ and `ic_sb256.tar` for $N = 256^3$.
(b) Initial condition files described in the same format as the above test
(c) Random distribution of particles

You must pass the absolute PATH of an initial condition file as the runtime command-line argument. Then, the program reads the given initial condition file and automatically identifies the type of initial condition ((a)-(c)). After setting the initial condition, the code numerically integrates the motions of particles to the finish time (specified by the redshift z) described in the file using the TreePM method. For the details of the format of initial condition file, please see `$(FDPS)/sample/c++/treepm/README_en.txt`. Note that an example of the initial condition file for the case (a) is given at `$(FDPS)/sample/c++/treepm/result/input.para`.

The structure of the sample code is as follows::

(1) Create and initialize FDPS objects
(2) Initialize the Phantom-GRAPE library (if needed)
(3) Read the initial condition file
(4) Integrate the motions of particles in time to the finish time

In the followings, we explain in detail each step described above.

6.2.4.1 Initialization and Termination of FDPS

First, you must initialize FDPS by the following code.

Listing 58: Initialization of FDPS

```
1 PS::Initialize(argc, argv);
```

Once started, FDPS should be terminated explicitly. In this sample, FDPS is terminated just before the termination of the program. Hence, you need to write the following code at the end of the main function.

Listing 59: Termination of FDPS

```
1 PS::Finalize();
```

6.2.4.2 Creation and Initialization of FDPS objects

After the initialization of FDPS, a user need to create the objects used to talk to FDPS. In this section, we describe how to create and initialize these objects.

6.2.4.2.1 Creation of necessary FDPS objects

In the calculation using the TreePM method, we must create objects of the FullParticle class, the DomainInfo class, the TreeForForceLong class (used in the PP part), and the ParticleMesh class (used in the PM part). In the sample code, these objects are created in the main function described in `treepm.cpp`:

Listing 60: Creation of FDPS objects

```
1 int main(int argc, char **argv)
2 {
3     PS::PM::ParticleMesh pm;
4     PS::ParticleSystem<FPtreepm> ptcl;
5     PS::DomainInfo domain_info;
6     PS::TreeForForceLong<Result_treepm, EPItreepm, EPJtreepm>::
            MonopoleWithCutoff treepm_tree;
7
8 }
```

Note that the above code is the one that is constructed by collecting the parts of object creation.

6.2.4.2.2 Initialization of FDPS objects

Almost all of FDPS objects must be initialized before they are used in a user code. Of four objects described in previous section, the ParticleMesh object is the only object that does not require an explicit initialization. The initialization of the other objects is done by the `initialize` method. In the following, we show an excerpt of the sample code where the initializations are performed:

Listing 61: Initialization of FDPS objects

```
1  int main(int argc, char **argv)
2  {
3     // Initialize ParticleSystem
4     ptcl.initialize();
5
6     // Initialize DomainInfo
7     domain_info.initialize();
8     domain_info.setBoundaryCondition(PS::BOUNDARY_CONDITION_PERIODIC_XYZ);
9     domain_info.setPosRootDomain(PS::F64vec(0.0, 0.0, 0.0),
10                                 PS::F64vec(1.0, 1.0, 1.0));
11
12    // Initialize Tree
13    treepm_tree.initialize(3*ptcl.getNumberOfParticleGlobal(),
14                           this_run.theta);
15 }
```

- The initialization of a ParticleSystem object is simply done by calling `initialize` method without any arguments.
- As for a DomainInfo object, we must set the boundary condition and the size of the simulation box after calling `initialize` method. These are done by calling `setBoundaryCondition` and `setPosRootDomain` methods.
- We must pass a rough number of particles to the `initialize` method of a TreeForForceLong object as the first argument. In this sample code, we pass a value three times larger than the total number of the particles. We can specify the value of the opening angle criterion θ used in the force calculation of the tree method via the second argument. Note that the object `this_run` is used to store a set of parameters such as θ that control the simulation.

6.2.4.3 Initial Condition

An parameter file that specifies an initial condition is read in the function `read_param_file()` described in the main function:

```
1  read_param_file(ptcl, this_run, argv[1]);
```

This function reads the parameter file specified by the command line arguments and sets the particle information such as mass and position to the ParticleSystem object based on the parameter file. After that, the sample code performs domain decomposition and particle exchange using FDPS APIs. In the following, we explain these APIs in detail.

6.2.4.3.1 Domain Decomposition

The sample code first perform domain decomposition, which is done by calling the `decomposeDomainAll` method (see the `main` function):

Listing 62: Domain Decomposition

```
1  domain_info.decomposeDomainAll(ptcl);
```

This method divides the entire of the domain based on a given particle distribution. Hence, we need to pass a ParticleSystem object to this method.

6.2.4.3.2 Particle Exchange

Then, the code performs particle exchange, which is done by calling the `exchangeParticle` method (see the `main` function):

Listing 63: Particle Exchange

```
1 ptcl.exchangeParticle(domain_info);
```

where we pass a DomainInfo object to this method because the method needs to know the information of domain decomposition in advance.

6.2.4.4 Interaction Calculation

After that, the code performs interaction calculation to determine the accelerations at the initial time. In the following, we show our implementation of interaction calculation. In this code, the `calcForceAllAndWriteBack` method of the TreeForForceLong object is used to calculate the PP part of the force calculation. This method automatically stores the results into the member variable `acc` of the ParticleSystem object. As for the PM part of the force calculation, we use the `calcForceAllAndWriteBack` method of the ParticleMesh object. Likewise, this method automatically stores the results of the PM part into the member variable `acc_pm` of the ParticleSystem object.

Listing 64: Interaction Calculation

```
1 //* PP part
2 treepm_tree.calcForceAllAndWriteBack
3     (calc_pp_force<EPJtreepm>(),
4      calc_pp_force<PS::SPJMonopoleCutoff>(),
5      ptcl,
6      domain_info);
7
8 //* PM part
9 pm.calcForceAllAndWriteBack(ptcl, domain_info);
```

6.2.4.5 Time Integration

The sample code uses the Leapfrog time integrator to perform time integration (the details of the method is described in § 4.1.3.4.4). The $D(\cdot)$ operator, which integrates the positions of particles in time, is implemented as the function `drift_ptcl`, while the $K(\cdot)$ operator, which integrates the velocities of particles in time, is implemented as the function `kick_ptcl`. The effects of cosmic expansion is taken into account in the function `kick_ptcl`. The time evolution of the scale factor and the Hubble parameter is done by the `update_expansion` method of the `this_run` object.

6.2.5 Compile

As explained in README.txt, you must edit `Makefile` in `src` directory appropriately to adapt to your computer environment. Then, run `make` command to compile the sample code. Note that this code uses FFTW library and therefore you have to install it in advance. The execution file `treepm` will be created if the compilation is succeeded.

6.2.6 Execution

We must run the program using MPI with the number of MPI processes is equal to or greater than 2, because of the specification of FDPS extension "ParticleMesh" module. Therefore, you should run the following command:

```
$ MPIRUN -np NPROC ./treepm
```

where "MPIRUN" represents the command to run a program using MPI such as `mpirun` or `mpiexec`, and "NPROC" is the number of MPI processes.

6.2.7 Confirmation of the result

After the simulation is completed, the results will be output at the directory specified in the parameter file. Figure 4 shows the time evolution of column density distribution of dark matter in a Santa Barbara Cluster Comparison test with 256^3 particles.

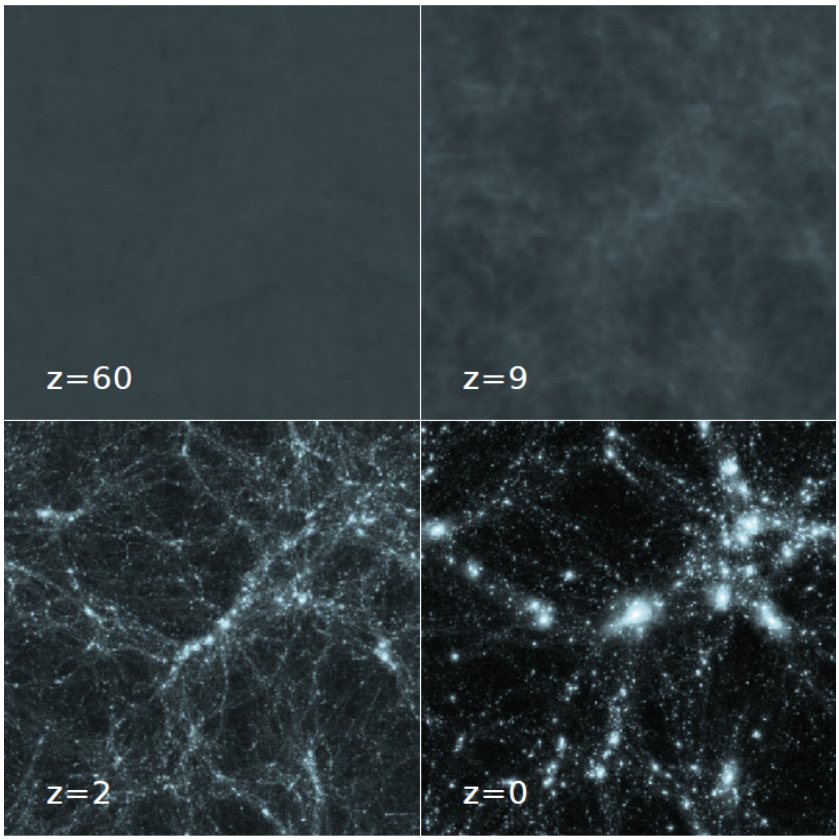


Figure 4: Time evolution of particle density of Santa Barbara Cluster Comparison test (the number of particles is 256^3)

7 Practical Applications

In previous sections, we have explained fundamental features of FDPS using relatively simple application codes. However, we need to develop a more complex application in actual research, in which for example we need to treat different types of particles. In this section, we will explain advanced features of FDPS using practical applications. To keep the explanations short and simple, we require the readers understand the contents of the previous sections in this document.

7.1 N-body/SPH code

In this section, we explain the accompanying sample code for N-body/SPH simulation of a disk galaxy. In this code, dark matter and stars, which perform gravitational interaction only, are represented by N-body particles, while interstellar gas, which performs both gravitational and hydrodynamic interactions, is represented by SPH particles. The tree method is used for the gravity calculation. The SPH scheme adopted in this code is the one proposed by Springel & Hernquist [2002, MNRAS, 333, 649] and Springel [2005, MNRAS, 364, 1105] (hereafter, we call it Springel's SPH scheme). The readers can understand how to treat different types of particles using FDPS by reading this section.

Below, we first explain the usage of the code. Next, we give a brief explanation of the Springel's SPH scheme. Then, we explain the contents of the sample source codes in detail.

7.1.1 How to run the sample code

As we described, this code simulates the dynamical evolution of a disk galaxy. This code sets the initial distributions of dark matter and stars by reading a file created by MAGI (Miki & Umemura [2018, MNRAS, 475, 2269]), which is a software to make an initial condition of a galaxy simulation. On the other hand, the initial gas distribution is set inside the code. Therefore, the following procedures are required to use the code.

- Move to directory `$(FDPS)/sample/c++/nbody+sph`
- Edit `Makefile` in the current directory
- Create particle data using MAGI and place it under directory `./magi_data/dat`
- Run the `make` command to create the executable `nbodysph.out`
- Run `nbodysph.out`
- Check the output

Below, we explain each procedure.

7.1.1.1 Move to the directory the sample code is placed

Move to `$(FDPS)/sample/c++/nbody+sph`.

7.1.1.2 File structure of the sample code

The following is the file structure of the sample code.

```
$ ls | awk '{print $0}'
Makefile
Makefile.K
Makefile.ofp
ic.hpp
job.K.sh
job.ofp.sh
leapfrog.hpp
macro_defs.hpp
magi_data/
main.cpp
mathematical_constants.cpp
mathematical_constants.h
physical_constants.cpp
physical_constants.h
test.py*
user_defined.hpp
```

We explain briefly the content of each source file. In `ic.hpp`, functions to create initial conditions are implemented. Users can choose an initial condition other than that for a disk galaxy (described later). In `leapfrog.hpp`, we implement functions necessary to integrate the orbits of particles based on the Leapfrog method. In `macro_defs.hpp`, we define macros that are used to control numerical simulation. In `main.cpp`, the main routine is implemented. In `mathematical_constants.h` and `mathematical_constants.cpp`, we define some mathematical constants. In `physical_constants.h` and `physical_constants.cpp`, we define some physical constants. In `user_defined.hpp`, we define user-defined classes and interaction functions.

Directory `magi_data` stores a parameter file input to the software MAGI (`magi_data/cfg/*`) and a script file used to run MAGI (`magi_data/sh/run.sh`).

7.1.1.3 Edit Makefile

Edit `Makefile` following the description below.

- Set the variable `CXX` the command to run your C++ compiler.
- Set the variable `CXXFLAGS` compile options of the C++ compiler.
- In this code, several macros are used to control numerical simulations. Table 1 lists the names of the macros and their definitions. In addition, there are macros whose states (i.e. value or defined/undefined states) are automatically set according to the value of macro `INITIAL_CONDITION`. Generally, users do not have to change them. Please see `macro_defs.h` directly for detail.

- Phantom-GRAPE library for x86 can be used for the gravity calculation. To use it, set the variable `use_phantom_grape_x86 yes`.

As for the way to specify the use/non-use of OpenMP and MPI, see § 3.

Macro name	Defintion
INITIAL_CONDITION	It specifies the type of initial condition or the operation mode of the code. It must take a value from 0 to 3. According to its value, the code operates as follows. 0: an initial condition for a disk galaxy is used, 1: an initial condition for cold collapse test problem is used, 2: an initial condition for Evrard test is used, 3: the code operates in the mode to make a glass-like distribution of SPH particles.
ENABLE_VARIABLE_SMOOTHING_LENGTH	It specifies that smoothing length of SPH particles is variable or not. If it is defined, variable smoothing length is used and the SPH calculation is performed according to the Springel's SPH scheme. If it is not defined, the fixed smoothing length is used and the SPH calculation is done in almost the same way as the sample code described in § 3-4.
USE_ENTROPY	It specifies whether to use entropy or specific internal energy as an independent variable to describe the thermodynamic state of SPH particle. If defined, entropy is used. But, if macro ISOTHERMAL_EOS described below is defined, specific internal energy is forcibly used (specific internal energy is used to calculate pressure).
USE_BALSARA_SWITCH	It specifies whether Balsara switch (Balsara [1995, JCP, 121, 357]) is used or not. If defined, the Balsara switch is used.
USE_PRESCR_OF_THOMAS_COUCHMAN_1992	It specifies whether a simple prescription proposed by Thomas & Couchman [1992, MNRAS,257, 11] to prevent the tensile instability is used or not. If defined, this prescription is used.
ISOTHERMAL_EOS	It specifies whether isothermal process is assumed or not. If defined, isothermal process is assumed (specific internal energy is assumed to be constant). If not defined, the code solve the entropy equation or the internal energy equation.
READ_DATA_WITH_BYTESWAP	It specifies whether the program reads particle data with performing byte swap (byte swap is applied for each variable of basic data type). If defined, byte swap is performed.

Table 1: Compile-time macros and their definitions

7.1.1.4 Create particle data using MAGI

As described earlier, users need to create particle data using the software MAGI before simulation according to the procedures described below. For users who cannot use `MAGI` for some reasons, we prepared sample particle data in web sites described below. In the following, we explain each case in detail.

Create particle data using MAGI Create particle data as follows.

1. Download the source file of MAGI from the web side https://bitbucket.org/ymiki/magi and install it in appropriate PATH according to the descriptions in Section "How to compile MAGI" in the above web side. But, our N-body/SPH sample code supports TIPSY file format only. Therefore, please build MAGI with `USE_TIPSY_FORMAT=ON`.
2. Edit `./magi_data/sh/run.sh` and set the variable `MAGI_INSTALL_DIR` the PATH of the directory where the `magi` command is stored. Also, set the variable `NTOT` the number of N-body particles (MAGI automatically assigns the numbers of dark matter particles and star particles).
3. Edit `./magi_data/cfg/*` to specify a galaxy model. For detail of the format of input file for MAGI, please see the web side above or Section 2.4 in the original paper Miki & Umemura [2018, MNRAS, 475, 2269]. In the default, galaxy model consists of the following four components (hereafter, we call this **default galaxy model**):
   (i) Dark matter halo (NFW profile, $M = 10^{12}$ M$_\odot$, $r_s = 21.5$ kpc, $r_c = 200$ kpc, $\Delta_c = 10$ kpc)
   (ii) Stellar bulge (King model, $M = 5 \times 10^{10}$ M$_\odot$, $r_s = 0.7$ kpc, $W_0 = 5$)
   (iii) Thick stellar disk (Sérsic profile, $M = 2.5 \times 10^{10}$ M$_\odot$, $r_s = 3.5$ kpc, $n = 1.5$, $z_d = 1$ kpc, $Q_{T,\mathrm{min}} = 1.0$)
   (iv) Thin stellar disk (exponential disk, $M = 2.5 \times 10^{10}$ M$_\odot$, $r_s = 3.5$ kpc, $z_d = 0.5$ kpc, $Q_{T,\mathrm{min}} = 1.0$)

   In the default galaxy model, two stellar disks are marginally unstable to a bar-mode in view of the Ostriker-Peebles criterion. Therefore, a simulated galaxy is expected to evolve into a spiral galaxy having a weak bar. In the latest release of MAGI (version 1.1.1 [as of July 19th, 2019]), its default operation mode is changed from previous releases. With this demand, we have replaced parameter f in thick and thin disks by $Q_{T,\mathrm{min}}$, where f is a parameter controlling the velocity dispersion of disk and is used in the previous releases of MAGI to specify the stability of a disk component. $Q_{T,\mathrm{min}}$ is the minimum of Toomre Q value in the disk. (In the sample code in FDPS 5.0d or earlier, we used $f = 0.125$).
4. Move to directory `magi_data` and run the following command:

```
$ ./sh/run.sh
```

5. If MAGI stops successfully, particle data whose extension is `tipsy` will be created in directory `magi_data/dat`.

Download sample particle data form our web sites Download a particle data file from one of the following URLs and place it under directory `./magi_data/dat/`. All of particle data is made with the default galaxy model. Only the number of particles is different for each data.

- $N = 2^{21}$: https://v2.jmlab.jp/owncloud/index.php/s/XnzvW5XAYwfqZYQ/download?path=%2Fmagi_data%2FGalaxy%2F21&files=Galaxy.tipsy
- $N = 2^{22}$: https://v2.jmlab.jp/owncloud/index.php/s/XnzvW5XAYwfqZYQ/download?path=%2Fmagi_data%2FGalaxy%2F22&files=Galaxy.tipsy
- $N = 2^{23}$: https://v2.jmlab.jp/owncloud/index.php/s/XnzvW5XAYwfqZYQ/download?path=%2Fmagi_data%2FGalaxy%2F23&files=Galaxy.tipsy
- $N = 2^{24}$: https://v2.jmlab.jp/owncloud/index.php/s/XnzvW5XAYwfqZYQ/download?path=%2Fmagi_data%2FGalaxy%2F24&files=Galaxy.tipsy

7.1.1.5 Run make

Type "make" to run the `make` command.

7.1.1.6 Run the sample code

- If you are not using MPI, run the following in CLI (terminal)

```
$ ./nbodysph.out
```

- If you are using MPI, run the following in CLI (terminal)

```
$ MPIRUN -np NPROC ./nbodysph.out
```

where `MPIRUN` should be `mpirun` or `mpiexec` depending on your MPI configuration, and `NPROC` is the number of processes you will use.

7.1.1.7 Analysis of the result

In the directory `result`, data of N-body and SPH particles are output as files "nbody0000x.dat" and "sph0000x.dat", where x is an integer representing time. The output file format of N-body particle data is that in each line, index of particle, mass, position (x, y, z), velocity (vx, vy, vz) are listed. The output file format of SPH particle data is that in each line, index of particle, mass, positon (x, y, z), velocity (vx, vy, vz), density, specific internal energy, entropy, pressure are listed.

Figure 5 shows the distribution of star and SPH particles at $T = 0.46$ for a disk galaxy simulation with the number of N-body particles is 2^{21} and the number of SPH particles is 2^{18}.

Below, we briefly explain the Springel's SPH scheme and then explain the implementation of the sample code.

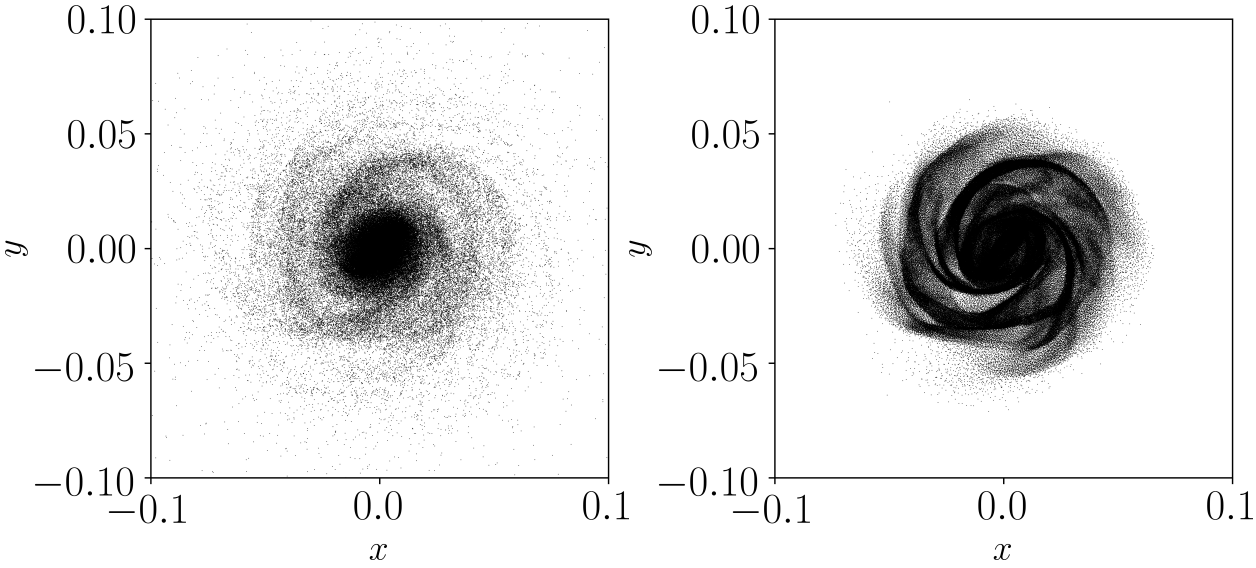


Figure 5: Face-on view of distributions of stars (left) and gas (right) (simulation configuration: the simulation is performed the number of N-body particles is 2^{21}, the number of SPH particles is 2^{18}, isothermal, gas temperature is 10^4 K, mean molecular weight to the mass of hydrogen $\mu = 0.5$)

7.1.2 Springel's SPH scheme

Springel & Hernquist [2002, MNRAS, 333, 649] proposed a formulation of SPH (actually, equation of motion[EoM]) where the total energy and entropy of a system are conserved even if smoothing length changes with time. In this section, we briefly explain their formulation. The outline of the derivation is as follows. Construct a Lagrangian of the system assuming that smoothing length is also independent variable, then solve the Euler-Lagrange equations under N constraints, where N is the number of particles.

More specifically, they consider the Lagrangian

$$L(\boldsymbol{q}, \dot{\boldsymbol{q}}) = \frac{1}{2}\sum_{i=1}^{N} m_i \dot{\boldsymbol{r}}_i^2 - \frac{1}{\gamma - 1}\sum_{i=1}^{N} m_i A_i \rho_i^{\gamma - 1} \tag{7}$$

where $\boldsymbol{q} = (\boldsymbol{r}_1, ..., \boldsymbol{r}_N, h_1, ...h_N)$ is the generalized coordinates (the subscripts represent the indice of particles), $\boldsymbol{r}_i$ is the position, h_i is smoothing length, m_i is mass, γ is the ratio of specific heats, ρ_i is density, A_i is called entropy function and it is related with specific internal energy u_i and ρ_i through the equation

$$u_i = \frac{A_i}{\gamma - 1}\rho_i^{\gamma - 1} \tag{8}$$

The first and second terms of Eq.(7) represents the kinetic energy and the internal energy of the system, respectively. Because solving the Euler-Lagrangian equation directly using this Lagrangian results in $4N$ equations, which is not undesirable, they introduce the following N constraints.

$$\phi_i = \frac{4\pi}{3} h_i^3 \rho_i - \overline{m} N_{\mathrm{neigh}} = 0 \tag{9}$$

where $\overline{m}$ is the average mass of SPH particles[3)], $N_{\rm neigh}$ is the number of neighbor particles (constant). Under these constraints, using the method of Lagrange multiplier, they solve the Euler-Lagrange equations to obtain the following equations of motion:

$$\frac{\mathrm{d}\boldsymbol{v}_i}{\mathrm{d}t} = -\sum_{j=1}^{N} m_j \left[f_i \frac{P_i}{\rho_i^2} \nabla_i W(r_{ij}, h_i) + f_j \frac{P_j}{\rho_j^2} \nabla_i W(r_{ij}, h_j) \right] \tag{10}$$

where P_i is pressure, $r_{ij} = |\boldsymbol{r}_i - \boldsymbol{r}_j|$, W is the kernel function, f_i is the so-called ∇h term, defined by

$$f_i = \left(1 + \frac{h_i}{3\rho_i} \frac{\partial \rho_i}{\partial h_i} \right)^{-1} \tag{11}$$

The thermodynamic state of the system is described by the independent variable A_i, the entropy. If the flow is adiabatic, the entropy is constant along the flow except for locations of shock waves where the entropy is increased. Springel [2005, MNRAS, 364, 1105] modeled the increase of the entropy by passing shock waves using the method of artificial viscosity:

$$\frac{\mathrm{d}A_i}{\mathrm{d}t} = \frac{1}{2} \frac{\gamma - 1}{\rho_i^{\gamma - 1}} \sum_{j=1}^{N} m_j \Pi_{ij} \boldsymbol{v}_{ij} \cdot \nabla_i \overline{W}_{ij} \tag{12}$$

$$\left. \frac{\mathrm{d}\boldsymbol{v}_i}{\mathrm{d}t} \right|_{\rm visc} = -\sum_{j=1}^{N} m_j \Pi_{ij} \nabla_i \overline{W}_{ij} \tag{13}$$

where $\boldsymbol{v}_{ij} = \boldsymbol{v}_i - \boldsymbol{v}_j$, $\boldsymbol{v}_i$ is velocity, $\overline{W}_{ij} = \frac{1}{2}(W(r_{ij}, h_i) + W(r_{ij}, h_j))$. For Π_{ij}, please see the original papers.

The procedures of SPH calculation is summarized as follows:

(1) Solve Eq.(9) and the following equation self-consistently to determine the density ρ_i and the smoothing length h_i.

$$\rho_i = \sum_{j=1}^{N} m_j W(r_{ij}, h_i) \tag{14}$$

(2) Calculate ∇h term defined by Eq.(11).
(3) Calculate the right-hand side of Eqs.(10), (12)、(13).
(4) Update the positions, velocities, entropies of SPH particles.

In the remaining sections, we first explain the implementations of user-defined classes and interaction functions. Then, we explain the implementation of the main routine where we explain how to treat different types of particles in FDPS.

7.1.3 User-defined types

All user-defined types are defined in `user_defined.hpp` . Here, we explain the types of user-defined types used in this code. As described earlier, this code use two types of particles,

[3)] This must be treated as constant.

N-body and SPH particles. Thus, this code defines **two** FullParticle types (`FP_nbody` class for N-body particles and `FP_sph` class for SPH particles). The number of types of *physical* interactions are two, the gravitational and hydrodynamic interactions. But, as explained in § 4, we need to perform (at least) two interaction calculations (for density and acceleration) in SPH calculations. Therefore, the code defines **three** Force types (`Force_grav` class for the gravity calculation, `Force_dens` class for the density calculation, and `Force_hydro` class for the calculation of acceleration due to pressure gradient (hereafter we call it pressure-gradient acceleration for simplicity)). For simplicity, this code uses one class for both EssentialParticleI type and EssentialParticleJ type (hereafter, we call them together EssentialParticle type). Also this code uses the same EssentialParticle type for the calculations of density and pressure-gradient acceleration. Therefore, the number of types of EssentialParticle types is **two** (`EP_grav` class for the gravity calculation and `EP_hydro` class for SPH calculation).

Below, we explain the implementation of each user defined type.

7.1.3.1 FullParticle type

First, we explain `FP_nbody` class , which is used to store the information of N-body particles. This data type contains all physical quantities that a N-body particle should have as member variables. Listing 65 shows the implementation of `FP_nbody` class . The definitions of the member variables and member functions are almost the same as those of N-body sample code introduced in § 3-4. Thus, please see the corresponding section for detail.

Listing 65: FullParticle type (`FP_nbody` class)

```
class FP_nbody{
public:
    PS::S64    id;
    PS::F64    mass;
    PS::F64vec pos;
    PS::F64vec vel;
    PS::F64vec acc;
    PS::F64    pot;

    PS::F64vec getPos() const {
        return pos;
    }
    PS::F64 getCharge() const {
        return mass;
    }
    void setPos(const PS::F64vec& pos){
       this->pos = pos;
    }
    void copyFromForce(const Force_grav & f) {
        this->acc = f.acc;
        this->pot = f.pot;
    }
    void writeAscii(FILE* fp) const {
        fprintf(fp, "%lld\t%g\t%g\t%g\t%g\t%g\t%g\t%g\n",
                this->id, this->mass,
                this->pos.x, this->pos.y, this->pos.z,
                this->vel.x, this->vel.y, this->vel.z);
```

```
28      }
29      void readAscii(FILE* fp) {
30          fscanf(fp, "%lld\t%lf\t%lf\t%lf\t%lf\t%lf\t%lf\t%lf\n",
31                  &this->id, &this->mass,
32                  &this->pos.x, &this->pos.y, &this->pos.z,
33                  &this->vel.x, &this->vel.y, &this->vel.z);
34      }
35  };
```

Next, we explain FP_sph class , which is used to store the information of SPH particles. This data type contains all physical quantities that a SPH particle should have as member variables. Listing 66 shows the implementation of FP_sph class . The definitions of main member variables are as follows: id (identification number), mass (mass), pos (position[$\boldsymbol{r}_i$]), vel (velocity[$\boldsymbol{v}_i$]), acc_grav (gravitational acceleration), pot_grav (gravitational potential), acc_hydro (pressure-gradient acceleration), dens (density[ρ_i]), eng (specific internel energy[u_i]), ent (entropy function [hereafter, entropy][A_i]), pres (pressure[P_i]), smth (smoothing length[4)][h_i]), gradh (∇h term[f_i]), divv ($(\nabla \cdot \boldsymbol{v})_i$, where the subscript i means that the derivative is performed at particle position), rotv ($(\nabla \times \boldsymbol{v})_i$), BalSW (coefficient for Balsara switch and its definition is the same as $f(a)$ in Balsara [1995, JCP, 121, 357]), snds (sound speed), eng_dot (time rate of change of eng), ent_dot (time rate of change of ent), dt (the maximum allowable time step to integrate the orbit of this particle).

The configuration of member functions are almost the same as those of the SPH sample code introduced in § 3-4, but there are the following differences:

- SPH particles are involved with three types of interaction calculations (gravity, density, pressure-gradient acceleration). Thus, **three** types of member function copyFromForce are defined.
- The existence of member function writeBinaryPos. This function is only used in the case of INITIAL_CONDITION=3.
- The existence of member function setEntropy. This function is used to set the initial value of entropy.

For the other member functions, please see § 3-4.

Listing 66: FullParticle type (FP_sph class)

```
1  class FP_sph {
2  public:
3      PS::S64    id;
4      PS::F64    mass;
5      PS::F64vec pos;
6      PS::F64vec vel;
7      PS::F64vec acc_grav; // gravitational acceleration
8      PS::F64    pot_grav; // gravitational potential
9      PS::F64vec acc_hydro; // acceleration due to pressure-gradient
10     PS::S32    flag;
11     PS::F64    dens; // mass density
12     PS::F64    eng; // specific internal energy
13     PS::F64    ent; // entropy
```

[4)] It is defined as the distance from the center of a particle where the value of the SPH kernel function is 0.

```
14      PS::F64    pres; // pressure
15      PS::F64    smth; // smoothing length
16      PS::F64    gradh; // grad-h term
17      PS::F64    divv; // divergence of velocity
18      PS::F64vec rotv; // rotation of velocity
19      PS::F64    BalSW; // Balsara switch
20      PS::F64    snds; // sound speed
21      PS::F64    eng_dot; // time rate of change of ‘eng‘
22      PS::F64    ent_dot; // time rate of change of ‘ent‘
23      PS::F64    dt; // hydrodynamic time step for this particle
24      PS::F64vec vel_half;
25      PS::F64    eng_half;
26      PS::F64    ent_half;
27
28      void copyFromForce(const Force_grav& f) {
29          this->acc_grav = f.acc;
30          this->pot_grav = f.pot;
31      }
32      void copyFromForce(const Force_dens& f){
33          this->flag  = f.flag;
34          this->dens  = f.dens;
35          this->smth  = f.smth;
36          this->gradh = f.gradh;
37          this->divv  = f.divv;
38          this->rotv  = f.rotv;
39
40      }
41      void copyFromForce(const Force_hydro& f){
42          this->acc_hydro = f.acc;
43          this->eng_dot   = f.eng_dot;
44          this->ent_dot   = f.ent_dot;
45          this->dt        = f.dt;
46      }
47      PS::F64 getCharge() const{
48          return this->mass;
49      }
50      PS::F64vec getPos() const{
51          return this->pos;
52      }
53      PS::F64 getRSearch() const{
54          return this->smth;
55      }
56      void setPos(const PS::F64vec& pos){
57         this->pos = pos;
58      }
59      void writeAscii(FILE* fp) const{
60          fprintf(fp,
61                  "%lld\t%lf\t%lf\t%lf\t%lf\t%lf\t"
62                  "%lf\t%lf\t%lf\t%lf\t%lf\t%lf\n",
63                  this->id, this->mass,
64                  this->pos.x, this->pos.y, this->pos.z,
65                  this->vel.x, this->vel.y, this->vel.z,
66                  this->dens, this->eng, this->ent, this->pres);
67      }
68      void readAscii(FILE* fp){
```

```
69          fscanf(fp,
70                 "%lld\t%lf\t%lf\t%lf\t%lf\t%lf\t"
71                 "%lf\t%lf\t%lf\t%lf\t%lf\t%lf\n",
72                 &this->id, &this->mass,
73                 &this->pos.x, &this->pos.y, &this->pos.z,
74                 &this->vel.x, &this->vel.y, &this->vel.z,
75                 &this->dens, &this->eng, &this->ent, &this->pres);
76      }
77      void writeBinaryPos(FILE* fp) const {
78          fwrite(&this->pos, sizeof(this->pos), 1, fp);
79      }
80      void setEntropy(){
81          ent = (specific_heat_ratio - 1.0) * eng / std::pow(dens,
                   specific_heat_ratio - 1.0);
82      }
83      void setPressure(){
84  #if defined(ISOTHERMAL_EOS)
85          // In this case, eng = const.
86          pres = (specific_heat_ratio - 1.0) * dens * eng;
87          ent  = pres / std::pow(dens, specific_heat_ratio);
88  #else
89  #if defined(USE_ENTROPY)
90          pres = ent * std::pow(dens, specific_heat_ratio);
91          eng  = pres / ((specific_heat_ratio - 1.0) * dens);
92  #else
93          pres = (specific_heat_ratio - 1.0) * dens * eng;
94          ent  = pres / std::pow(dens, specific_heat_ratio);
95  #endif
96  #endif
97          snds = std::sqrt(specific_heat_ratio * pres / dens);
98  #if defined(USE_BALSARA_SWITCH)
99          BalSW = std::fabs(divv) / (std::fabs(divv) + std::sqrt(rotv * rotv
                   ) + 1.0e-4 * snds / smth);
100 #else
101         BalSW = 1.0;
102 #endif
103     }
104 };
```

7.1.3.2 EssentialParticle type

First, we explain `EP_grav` class , which is used for the gravity calculation. This data type has all physical quantities that i- and j-particles should have in order to perform gravity calculation as member variables. Listing 67 shows the implementation of `EP_grav` class . EssentialParticle type should have member function(s) `copyFromFP()` to copy data from a FullParticle type. In this code, there are two FullParticle types and hence **two** `copyFromFP` functions are defined.

Listing 67: EssentialParticle type (`EP_grav` class)

```
1  class EP_grav {
2  public:
3      PS::S64 id;
4      PS::F64 mass;
```

```
5     PS::F64vec pos;
6
7     void copyFromFP(const FP_nbody& fp) {
8         this->id   = fp.id;
9         this->mass = fp.mass;
10        this->pos  = fp.pos;
11    }
12    void copyFromFP(const FP_sph& fp) {
13        this->id   = fp.id;
14        this->mass = fp.mass;
15        this->pos  = fp.pos;
16    }
17    PS::F64 getCharge() const {
18        return this->mass;
19    }
20    PS::F64vec getPos() const {
21        return this->pos;
22    }
23 };
```

Next, we explain `EP_hydro` class , which is used for the calculations of density and pressure-gradient acceleration. This data type has all physical quantities that i- and j-partiles should have in order to perform the calculations of density and pressure-gradient acceleration. Listing 68 shows the implementation of `EP_hydro` class . Note that member function `getRSearch` returns `smth` multiplied by a coefficient `SCF_smth` (it has a value larger than 1, but nearly equal to 1) instead of `smth` itself. This gimmick is introduced to perform the density calculation efficiently. For detail, please see § 7.1.4.2.

Listing 68: EssentialParticle type (`EP_hydro` class)

```
1  class EP_hydro {
2  public:
3     PS::S64    id;
4     PS::F64vec pos;
5     PS::F64vec vel;
6     PS::F64    mass;
7     PS::F64    smth;
8     PS::F64    dens;
9     PS::F64    pres;
10    PS::F64    gradh;
11    PS::F64    snds;
12    PS::F64    BalSW;
13
14    void copyFromFP(const FP_sph& fp){
15       this->id    = fp.id;
16       this->pos   = fp.pos;
17       this->vel   = fp.vel;
18       this->mass  = fp.mass;
19       this->smth  = fp.smth;
20       this->dens  = fp.dens;
21       this->pres  = fp.pres;
22       this->gradh = fp.gradh;
23       this->snds  = fp.snds;
24       this->BalSW = fp.BalSW;
25    }
```

```
26     PS::F64vec getPos() const{
27        return this->pos;
28     }
29     PS::F64 getRSearch() const{
30        return SCF_smth * this->smth;
31     }
32     void setPos(const PS::F64vec& pos){
33        this->pos = pos;
34     }
35 };
```

7.1.3.3 Force type

First, we explain `Force_grav` class , which is a Force type used for the gravity calculation. This data type must have all physical quantities that are obtained as the result of the gravity calculation. Listing 69 shows the implementation of `Force_grav` class .

Listing 69: Force type (`Force_grav` class)

```
1 class Force_grav {
2 public:
3     PS::F64vec acc;
4     PS::F64 pot;
5     void clear() {
6         acc = 0.0;
7         pot = 0.0;
8     }
9 };
```

Next, we explain `Force_dens` class , which is a Force type used for the density calculation. This data type must have all physical quantities that are obtained as the result of the density calculation. Listing 70 shows the implementation of `Force_dens` class . In the Springel's SPH scheme, the smoothing length h_i changes depending on the density at the position of a particle, ρ_i. In other words, h_i is also updated with ρ_i. Therefore, there is member variable `smth` to store updated smoothing length. In this code, we calculate ∇h term, $(\nabla \cdot \boldsymbol{v})_i$ $(\nabla \times \boldsymbol{v})_i$ at the same time (if `USE_BALSARA_SWITCH` is defined). Thus, there are member variables `gradh`, `divv`, `rotv` to store them. Member variable `flag` is used to store the result of iteration calculation of ρ_i and h_i (for detail, see § 7.1.4.2)。

Listing 70: Force type (`Force_dens` class)

```
1 class Force_dens{
2 public:
3    PS::S32 flag;
4    PS::F64 dens;
5    PS::F64 smth;
6    PS::F64 gradh;
7    PS::F64 divv;
8    PS::F64vec rotv;
9    void clear(){
10       flag  = 0;
11       dens  = 0.0;
12       gradh = 0.0;
```

```
13        divv   = 0.0;
14        rotv   = 0.0;
15    }
16 };
```

Finally, we explain `Force_hydro` class , which is a Force type used for the calculation of pressure-gradient acceleration. This data type must have all physical quantities that are obtained as the result of the calculation of pressure-gradient acceleration. Listing 71 shows the implementation of `Force_hydro` class .

Listing 71: Force type (`Force_hydro` class)

```
1  class Force_hydro{
2  public:
3     PS::F64vec acc;
4     PS::F64 eng_dot;
5     PS::F64 ent_dot;
6     PS::F64 dt;
7     void clear(){
8        acc = 0.0;
9        eng_dot = 0.0;
10       ent_dot = 0.0;
11    }
12 };
```

7.1.4 Interaction functions

All interaction functions are implemented in `user_defined.hpp` . There are **three** types of interaction functions. Below, we explain them.

7.1.4.1 Interaction function for the gravity calculation

Interaction functions for the gravity calculation are implemented as function template `CalcGravity` . Listing 72 shows the implementation. The implementation is almost the same as that of the N-body sample code introduced in § 3-4. For detail, please the corresponding section.

Listing 72: Interaction function for the gravity calculation

```
1  #if defined(ENABLE_PHANTOM_GRAPE_X86)
2  template <class TParticleJ>
3  void CalcGravity(const EP_grav * iptcl,
4                   const PS::S32 ni,
5                   const TParticleJ * jptcl,
6                   const PS::S32 nj,
7                   Force_grav * force) {
8      const PS::S32 nipipe = ni;
9      const PS::S32 njpipe = nj;
10     PS::F64 (*xi)[3] = (PS::F64 (*)[3])malloc(sizeof(PS::F64) * nipipe *
           PS::DIMENSION);
11     PS::F64 (*ai)[3] = (PS::F64 (*)[3])malloc(sizeof(PS::F64) * nipipe *
           PS::DIMENSION);
12     PS::F64  *pi     = (PS::F64  *   )malloc(sizeof(PS::F64) * nipipe);
```

```
13      PS::F64 (*xj)[3] = (PS::F64 (*)[3])malloc(sizeof(PS::F64) * njpipe *
            PS::DIMENSION);
14      PS::F64  *mj     = (PS::F64  *   )malloc(sizeof(PS::F64) * njpipe);
15      for(PS::S32 i = 0; i < ni; i++) {
16          xi[i][0] = iptcl[i].getPos()[0];
17          xi[i][1] = iptcl[i].getPos()[1];
18          xi[i][2] = iptcl[i].getPos()[2];
19          ai[i][0] = 0.0;
20          ai[i][1] = 0.0;
21          ai[i][2] = 0.0;
22          pi[i]    = 0.0;
23      }
24      for(PS::S32 j = 0; j < nj; j++) {
25          xj[j][0] = jptcl[j].getPos()[0];
26          xj[j][1] = jptcl[j].getPos()[1];
27          xj[j][2] = jptcl[j].getPos()[2];
28          mj[j]    = jptcl[j].getCharge();
29      }
30      PS::S32 devid = PS::Comm::getThreadNum();
31      g5_set_xmjMC(devid, 0, nj, xj, mj);
32      g5_set_nMC(devid, nj);
33      g5_calculate_force_on_xMC(devid, xi, ai, pi, ni);
34      for(PS::S32 i = 0; i < ni; i++) {
35          force[i].acc[0] += ai[i][0];
36          force[i].acc[1] += ai[i][1];
37          force[i].acc[2] += ai[i][2];
38          force[i].pot    -= pi[i];
39      }
40      free(xi);
41      free(ai);
42      free(pi);
43      free(xj);
44      free(mj);
45  }
46  #else
47  template <class TParticleJ>
48  void CalcGravity (const EP_grav * ep_i,
49                    const PS::S32 n_ip,
50                    const TParticleJ * ep_j,
51                    const PS::S32 n_jp,
52                    Force_grav * force) {
53      const PS::F64 eps2 = eps_grav * eps_grav;
54      for(PS::S32 i = 0; i < n_ip; i++){
55          PS::F64vec xi = ep_i[i].getPos();
56          PS::F64vec ai = 0.0;
57          PS::F64 poti = 0.0;
58          for(PS::S32 j = 0; j < n_jp; j++){
59              PS::F64vec rij    = xi - ep_j[j].getPos();
60              PS::F64    r3_inv = rij * rij + eps2;
61              PS::F64    r_inv  = 1.0/sqrt(r3_inv);
62              r3_inv  = r_inv * r_inv;
63              r_inv  *= ep_j[j].getCharge();
64              r3_inv *= r_inv;
65              ai     -= r3_inv * rij;
66              poti   -= r_inv;
```

```
67          }
68          force[i].acc += ai;
69          force[i].pot += poti;
70      }
71  }
72  #endif
```

7.1.4.2 Interaction function for the density calculation

Interaction function for the density calculation is implemented as function object `CalcDensity` . Listing 73 shows its implementation. The implementation actually used differs depending on the state of macro `ENABLE_VARIABLE_SMOOTHING_LENGTH`. If this macro is not defined, an implementation for fixed smoothing length is used. Its source code is almost the same as the interaction function for the density calculation of the SPH sample code described in § 3-4. Thus, we omit explanation for this case. Below, we explain an implementation used for the case that the above macro is defined.

As described in § 7.1.2, we need to determine the density ρ_i and smoothing length h_i at the same time by solving Eqs.(14) and (9) self-consistently. For this, we need to perform an iterative calculation. This calculation is performed in the infinite `for` loop in the code. As you'll see by reading the source code of member function `getRSearch() of EP_hydro class` , this sample code performs the density calculation after multiplying the smoothing lengths of all particles by a constant `SCF_smth` in order to make the density calculation efficiently. By this, we can change h_i between 0 and $h_{\mathrm{max,alw}} \equiv$ `SCF_smth` $\times h_{i,0}$, during the iteration, where $h_{i,0}$ is the value of the smoothing length of particle i before we multiply by `SCF_smth`. This is because all of particles that is eligible to be j-particles are contained in the current j-particle list (`ep_j`). If the iteration does not converge for some particle i, we cannot determine ρ_i and h_i for this particle by using the current j particle list because the value of the smoothing length we want to obtain will be larger than $h_{\mathrm{max,alw}}$. In this case, we need to perform the density calculation again after increasing $h_{i,0}$. This "outer" iteration is performed in `void` function `calcDensity`(note that the first letter of the function name is lower case) in `main.cpp` . We will describe this `void` function in § 7.1.5.

After the infinite `for` loop, this `void` function performs the calculations of ∇h, $(\nabla \cdot \boldsymbol{v})_i$, and $(\nabla \times \boldsymbol{v})_i$.

Listing 73: Interaction function for the density calculation

```
1  class CalcDensity{
2  public:
3      void operator () (const EP_hydro * ep_i,
4                        const PS::S32 n_ip,
5                        const EP_hydro * ep_j,
6                        const PS::S32 n_jp,
7                        Force_dens * force){
8  #if defined(ENABLE_VARIABLE_SMOOTHING_LENGTH)
9          const PS::F64 eps = 1.0e-6;
10         const PS::F64 M_trgt = mass_avg * N_neighbor;
11         for (PS::S32 i = 0; i < n_ip; i++) {
12             PS::F64 dens = 0.0;
13             PS::F64 h = ep_i[i].smth;
14             const PS::F64 h_max_alw = SCF_smth * h; // maximum allowance
```

```
15              PS::F64 h_L = 0.0;
16              PS::F64 h_U = h_max_alw;
17              PS::F64 dh_prev = 0.0;
18              PS::S32 n_unchanged = 0;
19              // Software caches
20              PS::F64 * mj  = (PS::F64 *)malloc(sizeof(PS::F64) * n_jp);
21              PS::F64 * rij = (PS::F64 *)malloc(sizeof(PS::F64) * n_jp);
22              for (PS::S32 j = 0; j < n_jp; j++) {
23                  mj[j] = ep_j[j].mass;
24                  const PS::F64vec dr = ep_j[j].pos - ep_i[i].pos;
25                  rij[j] = std::sqrt(dr * dr);
26              }
27              for (;;) {
28                  // Calculate density
29                  dens = 0.0;
30                  for (PS::S32 j = 0; j < n_jp; j++) {
31                     dens += mj[j] * W(rij[j], h);
32                  }
33                  // Check if the current value of the smoohting length
                        satisfies
34                  // Eq.(5) in Springel (2005).
35                  const PS::F64 M = 4.0 * math_const::pi * h * h * h * dens
                        / 3.0;
36                  if ((h < h_max_alw) && (std::abs(M/M_trgt - 1.0) < eps)) {
37                      // In this case, Eq.(5) holds within a specified
                            accuracy.
38                      force[i].flag = 1;
39                      force[i].dens = dens;
40                      force[i].smth = h;
41                      break;
42                  }
43                  if (((h == h_max_alw) && (M < M_trgt)) || (n_unchanged ==
                        4)) {
44                      // In this case, we skip this particle forcibly.
45                      // In order to determine consistently the density
46                      // and the smoohting length for this particle,
47                      // we must re-perform calcForceAllAndWriteBack().
48                      force[i].flag = 0;
49                      force[i].dens = dens;
50                      force[i].smth = h_max_alw;
51                      break;
52                  }
53                  // Update h_L & h_U
54                  if (M < M_trgt) {
55                     if (h_L < h) h_L = h;
56                  }
57                  else if (M_trgt < M) {
58                     if (h < h_U) h_U = h;
59                  }
60                  const PS::F64 dh = h_U - h_L;
61                  if (dh == dh_prev) {
62                     n_unchanged++;
63                  }
64                  else {
65                     dh_prev = dh;
```

```
66                      n_unchanged = 0;
67                  }
68                  // Update smoothing length
69                  h = std::pow((3.0 * M_trgt)/(4.0 * math_const::pi * dens),
                            1.0/3.0);
70                  if ((h <= h_L) || (h == h_U)) {
71                      // In this case, we switch to the bisection search.
72                      // The inclusion of '=' in the if statement is very
73                      // important to escape a limit cycle.
74                      h = 0.5 * (h_L + h_U);
75                  }
76                  else if (h_U < h) {
77                      h = h_U;
78                  }
79              }
80              // Calculate grad-h term
81              if (force[i].flag == 1) {
82                  PS::F64 drho_dh = 0.0;
83                  for (PS::S32 j = 0; j < n_jp; j++) {
84                      drho_dh += mj[j] * dWdh(rij[j], h);
85                  }
86                  force[i].gradh = 1.0 / (1.0 + (h * drho_dh) / (3.0 * dens)
                            );
87              }
88              else {
89                  force[i].gradh = 1.0; // dummy value
90              }
91 #if defined(USE_BALSARA_SWITCH)
92              // Compute \div v & \rot v for Balsara switch
93              for (PS::S32 j = 0; j < n_jp; j++) {
94                  const PS::F64vec dr = ep_i[i].pos - ep_j[j].pos;
95                  const PS::F64vec dv = ep_i[i].vel - ep_j[j].vel;
96                  force[i].divv -= mj[j] * dv * gradW(dr, force[i].smth);
97                  force[i].rotv -= mj[j] * dv ^ gradW(dr, force[i].smth);
98              }
99              force[i].divv /= force[i].dens;
100             force[i].rotv /= force[i].dens;
101 #endif
102             // Release memory
103             free(mj);
104             free(rij);
105         }
106 #else
107         for (PS::S32 i = 0; i < n_ip ; i++){
108             // Compute density
109             for (PS::S32 j = 0; j < n_jp; j++){
110                 const PS::F64vec dr = ep_j[j].pos - ep_i[i].pos;
111                 const PS::F64 rij = std::sqrt(dr * dr);
112                 force[i].dens += ep_j[j].mass * W(rij, ep_i[i].smth);
113             }
114             force[i].smth = ep_i[i].smth;
115             force[i].gradh = 1.0;
116 #if defined(USE_BALSARA_SWITCH)
117             // Compute \div v & \rot v for Balsara switch
118             for (PS::S32 j = 0; j < n_jp; j++) {
```

```
119                 const PS::F64vec dr = ep_i[i].pos - ep_j[j].pos;
120                 const PS::F64vec dv = ep_i[i].vel - ep_j[j].vel;
121                 force[i].divv -= ep_j[j].mass * dv * gradW(dr, force[i].
                          smth);
122                 force[i].rotv -= ep_j[j].mass * dv ^ gradW(dr, force[i].
                          smth);
123             }
124             force[i].divv /= force[i].dens;
125             force[i].rotv /= force[i].dens;
126 #endif
127         }
128 #endif
129     }
130 };
```

7.1.4.3 Interaction function for the calculation of pressure-gradient acceleration

Interaction function for the calculation of pressure-gradient acceleration is implemented as function object `CalcHydroForce` . Listing 74 shows its implementation. This performs the calculations of the right hand sides of Eqs.(10), (12), and (13), and `dt` according to Eq.(16) in Springel [2005, MNRAS, 364, 1105] (for `dt`, see the definition of `FP_sph` class)。

Listing 74: Interaction function for the calculation of pressure-gradient acceleration

```
1  class CalcHydroForce{
2  public:
3      void operator () (const EP_hydro * ep_i,
4                        const PS::S32 n_ip,
5                        const EP_hydro * ep_j,
6                        const PS::S32 n_jp,
7                        Force_hydro * force){
8          for (PS::S32 i = 0; i < n_ip; i++){
9              const PS::F64vec pos_i = ep_i[i].pos;
10             const PS::F64vec vel_i = ep_i[i].vel;
11             const PS::F64 smth_i   = ep_i[i].smth;
12             const PS::F64 dens_i   = ep_i[i].dens;
13             const PS::F64 pres_i   = ep_i[i].pres;
14             const PS::F64 f_i      = ep_i[i].gradh;
15             const PS::F64 snds_i   = ep_i[i].snds;
16             const PS::F64 povrho2_i = pres_i / (dens_i * dens_i);
17             PS::F64 v_sig_max = 0.0;
18             for (PS::S32 j = 0; j < n_jp; j++){
19                 const PS::F64vec dr = pos_i - ep_j[j].pos;
20                 const PS::F64vec dv = vel_i - ep_j[j].vel;
21                 const PS::F64 w_ij = (dv * dr < 0) ? dv * dr / std::sqrt(dr
                           * dr) : 0;
22                 const PS::F64 v_sig = snds_i + ep_j[j].snds - 3.0 * w_ij;
23                 v_sig_max = std::max(v_sig_max, v_sig);
24                 const PS::F64 AV = - 0.5 * alpha_AV * v_sig * w_ij / (0.5 *
                           (dens_i + ep_j[j].dens))
25                                  * 0.5 * (ep_i[i].BalSW + ep_j[j].BalSW);
26                 const PS::F64vec gradW_i  = gradW(dr, smth_i);
27                 const PS::F64vec gradW_j  = gradW(dr, ep_j[j].smth);
28                 const PS::F64vec gradW_ij = 0.5 * (gradW_i + gradW_j);
```

```
29                const PS::F64 povrho2_j = ep_j[j].pres / (ep_j[j].dens *
                        ep_j[j].dens);
30                const PS::F64 f_j = ep_j[j].gradh;
31                force[i].acc     -= ep_j[j].mass * (f_i * povrho2_i *
                        gradW_i
32                                                    +f_j * povrho2_j *
                                                            gradW_j
33                                                    +AV * gradW_ij);
34                force[i].eng_dot += ep_j[j].mass * (f_i * povrho2_i *
                        gradW_i
35                                                    +0.5 * AV * gradW_ij) *
                                                            dv;
36                force[i].ent_dot += 0.5 * ep_j[j].mass * AV * gradW_ij * dv;
37             }
38             const PS::F64 p = specific_heat_ratio - 1.0;
39             force[i].ent_dot *= p/std::pow(dens_i, p);
40             force[i].dt = CFL_hydro * 2.0 * ep_i[i].smth / v_sig_max;
41          }
42       }
43 };
```

7.1.5 Main body of the sample code

In this section, we describe the main body of the sample code implemented mainly in `main.cpp`. Before entering a detailed explanation, we describe here the overall structure of the code. As described in the beginning of § 7.1, this code performs a N-body/SPH simulation of a disk galaxy. Thus, in the default, the code sets an initial condition for a disk galaxy. But, initial conditions for simple test calculations are also prepared in the code. More specifically, the code supports the following four types of initial conditions:

(a) Initial condition for a disk galaxy simulation. It is selected when `-DINITIAL_CONDITION=0` is specified at the compile-time. The initial condition is created in `void` function `GalaxyIC` in `ic.hpp`. The initial distributions of dark matter and star particles are set by reading a file created by MAGI. The initial distribution of gas (SPH) particles is determined in the subroutine. In the default, an exponential disk ($M = 10^{10}$ M$_\odot$, $R_s = 7$ kpc [scale radius], $R_t = 12.5$ kpc [truncation radius], $z_d = 0.4$ kpc [scale height], $z_t = 1$ kpc [truncation height]) is created with the number of SPH particles of 2^{18}.

(b) Initial condition for cold collapse test. It is selected when `-DINITIAL_CONDITION=1` is specified at the compile-time. The initial condition is created in `void` function `ColdCollapseTestIC` in `ic.hpp`.

(c) Initial condition for the Evrard test (§ 3.3 in Evrard [1988,MNRAS,235,911]). It is selected when `-DINITIAL_CONDITION=2` is specified at the compile-time. This initial condition is created in `void` function `EvrardTestIC` in `ic.hpp`. There are two options for the way of creating an initial condition. We can specify the way by manually set the value of the last argument of the function 0 or 1. If 0 is given, the function creates the density profile of the Evrard gas sphere by rescaling the positions of particles which are placed in a grid. If 1 is specified, it creates the density profile by rescaling the positions of particles which are distributed glass-like. In order to use the second option, we have to create particle data by executing the code with the mode described in the next item.

(d) Operation mode to create a glass-like distribution of SPH particles in a box of $[-1, 1)^3$. This mode is selected when `-DINITIAL_CONDITION=3` is specified at the compile-time. The initial condition is created in `void` function `MakeGlassIC` in `ic.hpp` .

The structure of the sample code is as follows:

(1) Create and initialize FDPS objects
(2) Initialize the Phantom-GRAPE library for x86 if needed
(3) Read a data file of N-body particles and make an initial condition
(4) Calculate the motions of particles until the end time we specify

Below, we explain each item in detail.

7.1.5.1 Include the header file of FDPS

In order to use the features of FDPS, `particle_simulator.hpp` is included in the beginning part of `main.cpp`.

Listing 75: Include the header file of FDPS

```
1 #include <particle_simulator.hpp>
```

7.1.5.2 Initialization and and termination of FDPS

We need first to initialize FDPS by calling API `Initialize` :

Listing 76: Initialize FDPS

```
1 PS::Initialize(argc, argv);
```

Once started, FDPS should be explicitly terminated by calling API `Finalize` . This sample code terminates FDPS just before the termination of the program. You can find the following code at the last part of `main.cpp`.

Listing 77: Finalize FDPS

```
1 PS::Finalize();
```

7.1.5.3 Creation and initialization of FDPS objects

After the initialization of FDPS, a user need to create the objects used to talk to FDPS. In this section, we describe how to create and initialize these objects.

7.1.5.3.1 Creation and initialization of ParticleSystem objects

This sample code uses different ParticleSystem objects to manage N-body and SPH particles. More specifically, the code uses objects of names of `psys_nbody` and `psys_sph` for N-body and SPH particles, respectively. The creation and the initialization of these objects are done as follows.

Listing 78: Creation and initialization of ParticleSystem objects

```
1 PS::ParticleSystem<FP_nbody> psys_nbody;
2 PS::ParticleSystem<FP_sph> psys_sph;
3 psys_nbody.initialize();
4 psys_sph.initialize();
```

7.1.5.3.2 Creation and initialization of DomainInfo object

This sample code decomposes the computational domain so that the *total* (N-body + SPH) particle distribution is divided equally. In this case, we need one DomainInfo object. Thus, the creation and initialization of DomainInfo object are performed as follows.

Listing 79: Creation and initialization of DomainInfo object

```
1 PS::DomainInfo dinfo;
2 dinfo.initialize();
```

7.1.5.3.3 Creation and initialization of TreeForForce objects

The code uses three types of TreeForForce objects and they are used for the gravity calculation, the density calculation, and the calculation of pressure-gradient acceleration. When initializing a TreeForForce object, we must pass a typical number of particles used in the interaction calculation as the first argument of API `initialize` . For TreeForForce object `tree_grav` , the value that is three times of the number of local particles (N-body + SPH) is passed. On the other hand, for TreeForForce objects `tree_dens` and `tree_hydro` , the value that is three times of the number of local SPH particles is passed.

Listing 80: Creation and initialization of TreeForForce objects

```
1  const PS::S64 numPtclSPH = std::max(psys_sph.getNumberOfParticleGlobal()
       ,1);
2  const PS::S64 numPtclAll = psys_nbody.getNumberOfParticleGlobal()
3                           + numPtclSPH;
4
5  const PS::F32 theta_grav = 0.5;
6  PS::TreeForForceLong<Force_grav, EP_grav, EP_grav>::Monopole tree_grav;
7  tree_grav.initialize(3 * numPtclAll, theta_grav);
8
9  PS::TreeForForceShort<Force_dens, EP_hydro, EP_hydro>::Gather tree_dens;
10 tree_dens.initialize(3 * numPtclSPH);
11
12 PS::TreeForForceShort<Force_hydro, EP_hydro, EP_hydro>::Symmetry
       tree_hydro;
13 tree_hydro.initialize(3 * numPtclSPH);
```

7.1.5.4 Setting initial condition

The initial condition is set in `void` function `setupIC` , which internally calls a different `void` function depending on the value of macro `INITIAL_CONDITION`. The correspondence relation between the name of a internally-called `void` function and the value of the macro has

been described already in the beginning part of § 7.1.5. The arguments `time_dump`, `dt_dump`, `time_end` represents the initial time of data output, the time interval of data output, and the end time of the simulation, respectively. These must be set in this `void` function. Also, the boundary condition, the gravitational softening (`eps_grav`), the maximum allowable time step of the system (`dt_max`) are set in this `void` function (a user does not necessarily set `dt_max`).

Listing 81: Setting initial condition

```
1 setupIC(psys_nbody, psys_sph, dinfo, time_dump, dt_dump, time_end);
```

In what follows, we describe some of points to remember for `void` function `GalaxyIC` .

- MAGI outputs particle data in its code unit. The information about the MAGI's code unit is described in file `./magi_data/doc/unit.txt` (see section "Computational unit"). This file is created when executing MAGI. The variables `magi_unit_mass`, `magi_unit_leng`, `magi_unit_time` in the `void` function must be consistent with the MAGI's code unit.

- The `void` function reads particle data from file of the name of `./magi_data/dat/Galaxy.tipsy` in the default. If you make the code read a different file, please change the source code manually.

- The `void` function generates an initial gas distribution which has exponential profile along both R ($\equiv \sqrt{x^2+y^2}$) and z directions. The variables `Rs` and `zd` represents the scale lengths. The variables `Rt` and `zt` represents the truncation (cutoff) lengths.

- The initial thermodynamic state is specified by both the initial gas temperature `temp` and the mean molecular weight relative to the mass of hydrogen atom `mu`. Regardless of the state of the macro `USE_ENTROPY`, a user must specify the thermodynamic state of SPH particles via the specific internal energy (member variable `eng` in `FP_sph` class)[the sample code automatically does this]. If the macro `USE_ENTROPY` is defined, the initial value of the entropy is automatically set by `void` function `setEntropy` called in the `void` function `main()`, using the initial value of the specific internal energy and the calculated density. On the other hand, if the macro is not defined, the value of `eng` set in the `void` function `GalaxyIC` is treated as the initial value of the specific internal energy.

7.1.5.5 Domain decomposition

When there are different types of ParticleSystem objects, the domain decomposition based on the combined distribution of particles can be realized by using APIs `collectSampleParticle` and `decomposeDomain` . First, a user have to collect sample particles from each ParticleSystem object using API `collectSampleParticle` . **Here, we must pass `.false.` to the second argument of this API for the second or later ParticleSystem object because the previous information is cleared without this**. After collecting sample particles from all of ParticleSystem objects, call API `decomposeDomain` to perform domain decomposition.

Listing 82: Domain decomposition

```
dinfo.collectSampleParticle(psys_nbody);
dinfo.collectSampleParticle(psys_sph,false);
dinfo.decomposeDomain();
```

7.1.5.6 Particle exchange

In order to perform particle exchange based on the previous-calculated domain information, it is only necessary to call API `exchangeParticle` .

Listing 83: Particle exchange

```
psys_nbody.exchangeParticle(dinfo);
psys_sph.exchangeParticle(dinfo);
```

7.1.5.7 Interaction calculations

After the domain decomposition and particle exchange, interaction calculations are done. Below, we show the implementation of the interaction calculations just after setting the initial condition. At first, the code performs the gravity calculation. Then, it performs the calculations of density and pressure-gradient acceleration.

Listing 84: Interaction calculations

```
    //- Gravity calculations
#if defined(ENABLE_GRAVITY_INTERACT)
    tree_grav.setParticleLocalTree(psys_nbody);
    tree_grav.setParticleLocalTree(psys_sph,false);
    tree_grav.calcForceMakingTree(CalcGravity<EP_grav>,
                                  CalcGravity<PS::SPJMonopole>,
                                  dinfo);
    for (PS::S32 i = 0; i < psys_nbody.getNumberOfParticleLocal(); i++) {
        psys_nbody[i].copyFromForce(tree_grav.getForce(i));
    }
    const PS::S32 offset = psys_nbody.getNumberOfParticleLocal();
    for (PS::S32 i = 0; i < psys_sph.getNumberOfParticleLocal(); i++) {
        psys_sph[i].copyFromForce(tree_grav.getForce(i+offset));
    }
#endif

    //- SPH calculations
#if defined(ENABLE_HYDRO_INTERACT)
    calcDensity(psys_sph, dinfo, tree_dens);
#if defined(USE_ENTROPY)
    setEntropy(psys_sph);
#endif
    setPressure(psys_sph);
    tree_hydro.calcForceAllAndWriteBack(CalcHydroForce(), psys_sph, dinfo)
          ;
#endif
```

First, we explain the part of the implementation for the gravity calculation. In the gravity calculation, both N-body and SPH particles are involved. In order to perform an

interaction calculation between different types of particles, we must use in combination TreeForForce object's APIs `setParticleLocalTree` and `calcForceMakingTree` . We first pass the particle information stored in each ParticleSystem object to a TreeForForce object using API `setParticleLocalTree` . **Here, we must pass false to the second argument of this API for the second or later ParticleSystem objects because all of the previously-passed information is cleared without this**. After finishing calling this API for all of ParticleSystem objects that are involved in the gravity calculation, call API `calcForceMakingTree` to perform the interaction calculation. In order to obtain the result of the interaction calculation, we need to use API `getForce` . This API takes an integral argument i, and it returns the force of the ith particle read by API `setParticleLocalTree` . Hence, we must use appropriate offset to obtain the results of the interaction calculation of the second or later ParticleSystem.

Next, we explain the part of the implementation for the calculations of density and pressure-gradient acceleration. These interaction calculations involves only single type of particles, SPH particles. Therefore, we can use API `calcForceAllAndWriteBack` , which is frequently used in the sample code introduced in this document. For the calculation of pressure-gradient acceleration, the code performs this API in the `void` function `main()`. On the other hand, we need to handle the case that the iteration calculation of ρ_i and h_i does not converge for some particles as described in § 7.1.4. This handling is done in the `void` function `calcDensity` . The implementation of this `void` function is shown below. The implementation actually used differs depending on the state of the macro `ENABLE_VARIABLE_SMOOTHING_LENGTH`. If it is not defined, the code calls API `calcForceAllAndWriteBack` only once because in this case the code performs SPH calculation as the fixed smoothing length SPH code. If the macro is defined, the code calls the API repeatedly until ρ_i and h_i of all the particles are self-consistently determined. The member variable `flag` stores the result of the iteration calculation and the value of 1 means that the iteration converges successfully. So, the code stops the infinite `for` loop when the number of SPH particles whose `flag` has the value of 1 agrees with the total number of SPH particles.

Listing 85: Function `calcDensity`

```
1  void calcDensity(PS::ParticleSystem<FP_sph> & psys,
2                   PS::DomainInfo & dinfo,
3                   PS::TreeForForceShort<Force_dens, EP_hydro, EP_hydro>::
                          Gather & tree) {
4  #if defined(ENABLE_VARIABLE_SMOOTHING_LENGTH)
5      const PS::S32 n_loc = psys.getNumberOfParticleLocal();
6      const PS::S64 n_glb = psys.getNumberOfParticleGlobal();
7      // Determine the density and the smoothing length so that Eq.(6) in
             Springel (2005)
8      // holds within a specified accuracy.
9      SCF_smth = 1.25;
10     PS::S32 iter = 0;
11     for (;;) {
12         iter++;
13         if (PS::Comm::getRank() == 0) std::cout << "iter␣=␣" << iter <<
                 std::endl;
14         // Compute density, etc.
15         tree.calcForceAllAndWriteBack(CalcDensity(), psys, dinfo);
16         // Check convergence
17         PS::S32 n_compl_loc = 0;
```

```
18          for (PS::S32 i = 0; i < n_loc; i++) {
19              if (psys[i].flag == 1) n_compl_loc++;
20          }
21          const PS::S64 n_compl = PS::Comm::getSum(n_compl_loc);
22          if (n_compl == n_glb) break;
23      }
24      // Reset SCF_smth
25      SCF_smth = 1.0;
26  #else
27      SCF_smth = 1.0;
28      tree.calcForceAllAndWriteBack(CalcDensity(), psys, dinfo);
29  #endif
30  }
```

`void` function `setEntropy` is called only once just after setting an initial condition. As described earlier, this `void` function is used to set the initial value of the entropy. Because we need the initial density to set the initial value of the entropy using Eq. (8), this `void` function is placed just after `void` function `calcDensity` . After this, the entropy becomes the independent variable to describe the thermodynamic state of gas if the macro `USE_ENTROPY` is defined.

7.1.5.8 Time integration

This code performs the time integration using the Leapfrog method (see § 4.1.3.4.4 for this method). In this code, $D(\cdot)$ operator is implemented as the `void` function `full_drift`, while $K(\cdot)$ operator is implemented as `void` functions `initial_kick` and `final_kick`.

8 User Supports

We accept questions and comments on FDPS at the following mail address:
fdps-support@mail.jmlab.jp
Please provide us with the following information.

8.1 Compile-time problem

- Compiler environment (version of the compiler, compile options etc)
- Error message at the compile time
- (if possible) the source code

8.2 Run-time problem

- Run-time environment
- Run-time error message
- (if possible) the source code

8.3 Other cases

For other problems, please do not hesitate to contact us. We sincerely hope that you'll find FDPS useful for your research.

9 License

This software is MIT licensed. Please cite Iwasawa et al. (2016, Publications of the Astronomical Society of Japan, 68, 54) and Namekata et al. (2018, Publications of the Astronomical Society of Japan, 70, 70) if you use the standard functions only.

The extended feature "Particle Mesh" is implemented by using a module of GreeM code (Developers: Tomoaki Ishiyama and Keigo Nitadori) (Ishiyama, Fukushige & Makino 2009, Publications of the Astronomical Society of Japan, 61, 1319; Ishiyama, Nitadori & Makino, 2012 SC'12 Proceedings of the International Conference on High Performance Computing, Networking Stroage and Analysis, No. 5). GreeM code is developed based on the code in Yoshikawa & Fukushige (2005, Publications of the Astronomical Society of Japan, 57, 849). Please cite these three literatures if you use the extended feature "Particle Mesh".

Please cite Tanikawa et al.(2012, New Astronomy, 17, 82) and Tanikawa et al.(2012, New Astronomy, 19, 74) if you use the extended feature "Phantom-GRAPE for x86".

Copyright (c) <2015-> <FDPS developer team>

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.